

ỦY BAN NHÂN DÂN TỈNH AN GIANG
TRƯỜNG CAO ĐẲNG NGHỀ AN GIANG



GIÁO TRÌNH
MÔ-ĐUN: LẬP TRÌNH WINDOWS
NGHỀ : LẬP TRÌNH MÁY TÍNH – TIN HỌC
ỨNG DỤNG
TRÌNH ĐỘ : CAO ĐẲNG - TRUNG CẤP

(Ban hành theo Quyết định số: /QĐ-CDN, ngày tháng năm 20 của Hiệu trưởng trường Cao đẳng nghề An Giang)

An Giang, Năm: 2026

TUYÊN BỐ BẢN QUYỀN

Tài liệu này thuộc loại sách giáo trình nên các nguồn thông tin có thể được phép dùng nguyên bản hoặc trích dùng cho các mục đích về đào tạo và tham khảo.

Nghiêm cấm mọi mục đích khác mang tính lệch lạc hoặc sử dụng với mục đích kinh doanh thiếu lành mạnh

LỜI GIỚI THIỆU

Lập trình Windows với C# là quá trình xây dựng các ứng dụng chạy trên hệ điều hành Windows bằng ngôn ngữ lập trình C# và nền tảng .NET của Microsoft. Đây là một trong những hướng phát triển phần mềm phổ biến nhờ giao diện trực quan, dễ sử dụng và khả năng xử lý mạnh mẽ.

Trong lập trình Windows, người học thường sử dụng các công nghệ như Windows Forms (WinForms) hoặc WPF (Windows Presentation Foundation) để thiết kế giao diện người dùng. Với C#, lập trình viên có thể tạo ra nhiều loại ứng dụng như: Phần mềm quản lý bán hàng, ứng dụng quản lý sinh viên, phần mềm kế toán, công cụ hỗ trợ học tập và làm việc

Lập trình Windows bằng C# rất phù hợp cho người mới bắt đầu học lập trình desktop cũng như sinh viên ngành công nghệ thông tin muốn xây dựng các phần mềm quản lý thực tế.

An Giang, ngày tháng năm 2026

Tham gia biên soạn

Trần Thị Kim Ngọc

MỤC LỤC

LỜI GIỚI THIỆU	1
MỤC LỤC	2
Bài 1: NỀN TẢNG LẬP TRÌNH	7
1. Cơ bản về C#	7
1.1. Kiểu dữ liệu	7
1.2. Biến và hằng	11
1.3. Biểu thức	14
1.4. Khoảng trắng	15
1.5. Câu lệnh	16
1.6. Toán tử	27
1.7. Namespace	31
1.8. Các chỉ dẫn biên dịch	31
1.9. Xử lý ngoại lệ	32
2. Xây dựng lớp – Đối tượng	43
2.1. Lớp và đối tượng	43
2.2. Sử dụng các thành viên static	51
2.3. Hủy đối tượng	55
2.4. Truyền tham số và nạp chồng phương thức	56
2.5. Đóng gói dữ liệu với thuộc tính	65
3. Kế thừa – Đa hình	69
3.1. Sự kế thừa	69
3.2. Đa hình	69
3.3. Lớp trừu tượng	70
3.4. Các lớp lồng nhau	71
4. Nạp chồng toán tử	71
4.1. Sử dụng từ khóa operator	72
4.2. Toán tử so sánh bằng	72
4.3. Toán tử chuyển đổi	73
5. Cấu trúc	73
6. Thực thi giao diện	76
7. Mảng	80
7.1. Mảng	80
7.2. Danh sách mảng	81
7.3. Câu lệnh foreach	81
8. Xử lý chuỗi	82
Bài 2: Xây dựng giao diện người dùng	91
1. Các đối tượng	91
1.1. Các nhãn và hộp nhập văn bản, lớp nút nhấn	91

1.2. Các sự kiện bàn phím và chuột	93
1.3. ListBox, ComboBox, Calendar	97
1.4. Image, ImageList.....	101
1.5. Toolbar, Tab, Tabpage, Tooltip	102
1.6. ListView, TreeView	103
2. Các hộp thoại	109
2.1. Hộp thoại thông điệp	109
2.2. Các hộp thoại tập tin.....	111
3. Thực đơn	114
4. Các ứng dụng MDI	118
Bài 3: Lưu trữ và quản lý dữ liệu	125
1. Tổng quan về ADO.NET	125
2. Các công cụ kết nối dữ liệu	126
3. Hiển thị dữ liệu, lọc dữ liệu	129
4. Thao tác với dữ liệu	133
5. Lập báo cáo	134
TÀI LIỆU THAM KHẢO	139

CHƯƠNG TRÌNH MÔ ĐUN

Tên mô đun: Lập trình window

Mã mô đun: MĐ 20

Thời gian thực hiện mô đun: 90 giờ; (*Lý thuyết: 34 giờ; thực hành, thí nghiệm, thảo luận, bài tập: 49 giờ; Thi/Kiểm tra: 7 giờ*)

I. Vị trí, tính chất của mô đun

- Vị trí: Mô đun này thuộc nhóm mô đun chuyên môn trong chương trình đào.
- Tính chất: Đây là mô đun chứa đựng kiến thức nền tảng về ngôn ngữ lập trình C#, chứa đựng kiến thức nền tảng về lập trình Windows Form, lập trình kết nối cơ sở dữ liệu. Mô đun này là nền tảng giúp người học xây dựng được một ứng dụng quản lý trên nền Windows Form

II. Mục tiêu mô đun

- Kiến thức:
 - + Trình bày được các kiến thức về nền tảng Microsoft .NET.
 - + Trình bày được kiến thức về lập trình hướng đối tượng, xử lý mảng, chuỗi và sử dụng các lớp cơ sở của .NET
 - + Trình bày được quy trình: Tạo các ứng dụng, tạo được các ứng dụng cơ sở dữ liệu trên nền Window.
 - + Trình bày được các đối tượng của .NET
 - + Trình bày được cơ sở dữ liệu với các báo cáo..
- Kỹ năng:
 - + Tạo các ứng dụng cơ sở dữ liệu trên nền Windows.
 - + Tạo được ứng dụng cơ sở dữ liệu với các báo cáo.
 - + Tạo ra các ứng dụng MDI..
- Năng lực tự chủ và trách nhiệm:
 - + Hình thành kỹ năng làm việc cẩn thận, nghiêm túc.
 - + Có thể làm việc độc lập hoặc theo nhóm trong việc ứng dụng năng lực số cơ bản vào học tập, lao động và các hoạt động khác.

III. Nội dung mô đun

1. Nội dung tổng quát và phân bổ thời gian

Số TT	Tên các bài trong mô đun	Thời gian (giờ)			
		Tổng số	Lý thuyết	Thực hành, thí nghiệm, thảo luận, bài tập	Thi/Kiểm tra

1	Bài 1: Nền tảng lập trình	28	11	16	1
	1. Cơ bản về C#		2	2	
	1.1. Kiểu dữ liệu				
	1.2. Biến và hằng				
	1.3. Biểu thức				
	1.4. Khoảng trắng				
	1.5. Câu lệnh				
	1.6. Toán tử				
	1.7. Namespace				
	1.8. Các chỉ dẫn biên dịch				
	1.9. Xử lý ngoại lệ				
	2. Xây dựng lớp – Đối tượng		3	5	
	2.1. Lớp và đối tượng				
	2.2. Sử dụng các thành viên static				
	2.3. Hủy đối tượng				
	2.4. Truyền tham số và nạp chồng phương thức				
	2.5. Đóng gói dữ liệu với thuộc tính				
	3. Kế thừa – Đa hình		2	2	
	3.1. Sự kế thừa				
	3.2. Đa hình				
	3.3. Lớp trừu tượng				
	3.4. Các lớp lồng nhau				
	4. Nạp chồng toán tử		2	2	
	4.1. Sử dụng từ khóa operator				
	4.2. Toán tử so sánh bằng				
	4.3. Toán tử chuyển đổi				
	5. Cấu trúc		0.5	1	
	6. Thực thi giao diện		0.5	2	
	7. Mảng		0.5	1	
	7.1. Mảng				
	7.2. Danh sách mảng				
	7.3. Câu lệnh foreach				

	8. Xử lý chuỗi		0.5	1	
	Kiểm tra				1
2	Bài 2: Xây dựng giao diện người dùng	30	12	17	1
	1. Các đối tượng		5	8	
	1.1. Các nhãn và hộp nhập văn bản, lớp nút nhấn				
	1.2. Các sự kiện bàn phím và chuột				
	1.3. ListBox, ComboBox, Calendar				
	1.4. Image, ImageList				
	1.5. Toolbar, Tab, Tabpage, Tooltip				
	1.6. ListView, TreeView				
	2. Các hộp thoại		2	2	
	2.1. Hộp thoại thông điệp				
	2.2. Các hộp thoại tập tin				
	3. Thực đơn		2	2	
	4. Các ứng dụng MDI		3	5	
	Kiểm tra				1
3	Bài 3: Lưu trữ và quản lý dữ liệu	28	11	16	1
	1. Tổng quan về ADO.NET				
	2. Các công cụ kết nối dữ liệu		2	2	
	3. Hiển thị dữ liệu, lọc dữ liệu		3	5	
	4. Thao tác với dữ liệu		4	5	
	5. Lập báo cáo		2	4	
	Kiểm tra				1
4	Thi kết thúc môn	4			4
	Cộng	90	34	49	7

BÀI 1: NỀN TẢNG LẬP TRÌNH

Giới thiệu:

Bài học cung cấp cho sinh viên kiến thức cơ bản và nâng cao về lập trình C# theo hướng lập trình hướng đối tượng. Nội dung môn học bao gồm các kiến thức nền tảng như kiểu dữ liệu, biến, hằng, biểu thức, câu lệnh, toán tử, namespace, xử lý ngoại lệ và các chỉ dẫn biên dịch. Bên cạnh đó, sinh viên được tìm hiểu cách xây dựng lớp và đối tượng, sử dụng thành viên static, truyền tham số, nạp chồng phương thức và đóng gói dữ liệu với thuộc tính.

Đồng thời giới thiệu các đặc trưng quan trọng của lập trình hướng đối tượng như kế thừa, đa hình, lớp trừu tượng, giao diện và nạp chồng toán tử. Ngoài ra, sinh viên còn được học về cấu trúc dữ liệu mảng, danh sách mảng, câu lệnh foreach và kỹ thuật xử lý chuỗi trong C#. Qua đó, sinh viên có khả năng xây dựng các chương trình ứng dụng cơ bản, vận dụng hiệu quả các kỹ thuật lập trình và phát triển tư duy logic trong giải quyết bài toán thực tế.

Mục tiêu:

Học xong bài này, sinh viên có khả năng:

- Trình bày được các kiểu dữ liệu dựng sẵn của C#.
- Trình bày được biến, hằng và các biểu thức, các toán tử.
- Trình bày được các chỉ dẫn biên dịch.
- Trình bày được tham số và các phương thức nạp chồng.
- Vận dụng được đóng gói dữ liệu với thuộc tính.
- Tạo được lớp, tạo đối tượng.
- Trình bày được tính thừa kế, tính đa hình, lớp trừu tượng, lớp giao diện.
- Vận dụng được các câu lệnh trong bài toán cụ thể.
- Vận dụng được tính thừa kế, đa hình và trừu tượng.
- Nghiêm túc trong học tập và thực hiện tốt các yêu cầu được giao.

Nội dung chính:

1. Cơ bản về C#

1.1. Kiểu dữ liệu

C# chia thành hai tập hợp kiểu dữ liệu chính: Kiểu xây dựng sẵn (built-in) mà ngôn ngữ cung cấp cho người lập trình và kiểu được người dùng định nghĩa (user-defined) do người lập trình tạo ra. C# phân tập hợp kiểu dữ liệu này thành hai loại: Kiểu dữ liệu giá trị (value) và kiểu dữ liệu tham chiếu (reference). Bảng các kiểu dữ liệu xây dựng sẵn

Kiểu C#	Số byte	Kiểu .NET	Mô tả
byte	1	Byte	Số nguyên dương không dấu từ 0-255
char	2	Char	Ký tự Unicode
bool	1	Boolean	Giá trị logic true/ false
sbyte	1	Sbyte	Số nguyên có dấu (từ -128 đến 127)
short	2	Int16	Số nguyên có dấu giá trị từ -32768 đến 32767.
ushort	2	UInt16	Số nguyên không dấu 0 – 65.535
int	4	Int32	Số nguyên có dấu –2.147.483.647 và 2.147.483.647
uint	4	UInt32	Số nguyên không dấu 0 – 4.294.967.295
float	4	Single	Kiểu dấu chấm động, giá trị xấp xỉ từ 3,4E-38 đến 3,4E+38, với 7 chữ số có nghĩa..
double	8	Double	Kiểu dấu chấm động có độ chính xác gấp đôi, giá trị xấp xỉ từ 1,7E-308 đến 1,7E+308, với 15,16 chữ số có nghĩa.
decimal	8	Decimal	Có độ chính xác đến 28 con số và giá trị thập phân, được dùng trong tính toán tài chính, kiểu này đòi hỏi phải có hậu tố “m” hay “M”
long	8	Int64	Kiểu số nguyên có dấu có giá trị trong khoảng : -9.223.370.036.854.775.808 đến 9.223.372.036.854.775.807
ulong	8	UInt64	Số nguyên không dấu từ 0 đến 0xffffffffffffffff

Ghi chú: Tất cả các kiểu dữ liệu xây dựng sẵn là kiểu dữ liệu giá trị ngoại trừ các đối tượng và chuỗi. Và tất cả các kiểu do người dùng định nghĩa ngoại trừ kiểu struct đều là kiểu dữ liệu tham chiếu. trong bài học này chúng ta sẽ tìm hiểu các kiểu xây dựng sẵn.

Kiểu liệt kê

Kiểu liệt kê đơn giản là tập hợp các tên hằng có giá trị không thay đổi (thường được gọi là danh sách liệt kê).

Ví dụ có hai biểu tượng hằng có quan hệ với nhau:

```
const int DoDong = 0;
```

```
const int DoSoi = 100;
```

Do mục đích mở rộng ta mong muốn thêm một số hằng số khác vào danh sách trên, như các hằng sau:

```
const int DoNong = 60;
const int DoAm = 40;
const int DoNguoi = 20;
```

Các biểu tượng hằng trên đều có ý nghĩa quan hệ với nhau, cùng nói về nhiệt độ của nước, khi khai báo từng hằng trên có vẻ cồng kềnh và không được liên kết chặt chẽ cho lắm. Thay vào đó C# cung cấp kiểu liệt kê để giải quyết vấn đề trên:

```
enum NhietDoNuoc
{
    DoDong = 0,
    DoNguoi = 20,
    DoAm = 40,
    DoNong = 60,
    DoSoi = 100,
}
```

Mỗi kiểu liệt kê có một kiểu dữ liệu cơ sở, kiểu dữ liệu có thể là bất cứ kiểu dữ liệu nguyên nào như int, short, long... tuy nhiên kiểu dữ liệu của liệt kê không chấp nhận kiểu ký tự. Để khai báo một kiểu liệt kê ta thực hiện theo cú pháp sau:

[thuộc tính] [bổ sung] enum <tên liệt kê> [:kiểu cơ sở] {danh sách các thành phần liệt kê};

Thành phần thuộc tính và bổ sung là tự chọn sẽ được trình bày trong phần sau của sách. Trong phần này chúng ta sẽ tập trung vào phần còn lại của khai báo. Một kiểu liệt kê bắt đầu với từ khóa enum, tiếp sau là một định danh cho kiểu liệt kê:

```
enum NhietDoNuoc
```

Thành phần kiểu cơ sở chính là kiểu khai báo cho các mục trong kiểu liệt kê. Nếu bỏ qua thành phần này thì trình biên dịch sẽ gán giá trị mặc định là kiểu nguyên int, tuy nhiên chúng ta có thể sử dụng bất cứ kiểu nguyên nào như ushort hay long,...ngoại trừ kiểu ký tự. Đoạn ví dụ sau khai báo một kiểu liệt kê sử dụng kiểu cơ sở là số nguyên không dấu uint:

```
enum KichThuoc :uint
{
    Nho = 1,
    Vua = 2,
    Lon = 3,
}
```

Lưu ý là khai báo một kiểu liệt kê phải kết thúc bằng một danh sách liệt kê, danh sách liệt kê này phải có các hằng được gán, và mỗi thành phần phải phân cách nhau dấu phẩy.

Ta viết lại ví dụ minh trên như sau.

□ Ví dụ 1.1: Sử dụng kiểu liệt kê để đơn giản chương trình.

```
-----
class MinhHoaC3
{
// Khai báo kiểu liệt kê
enum NhietDoNuoc
{
DoDong = 0,
DoNguoi = 20,
DoAm = 40,
DoNong = 60,
DoSoi = 100,
}
static void Main()
{
System.Console.WriteLine(    “Nhiet    do    dong:    {0}”,
NhietDoNuoc.DoDong);
System.Console.WriteLine(    “Nhiet    do    nguoi:    {0}”,
NhietDoNuoc.DoNguoi);
System.Console.WriteLine( “Nhiet do am: {0}”, NhietDoNuoc.DoAm);
System.Console.WriteLine(    “Nhiet    do    nong:    {0}”,
NhietDoNuoc.DoNong);
System.Console.WriteLine( “Nhiet do soi: {0}”, NhietDoNuoc.DoSoi);
}
}
-----
```

□ Kết quả:

Nhiet do dong: 0

Nhiet do nguoi: 20

Nhiet do am: 40

Nhiệt độ nóng: 60

Nhiệt độ sôi: 100

Mỗi thành phần trong kiểu liệt kê tương ứng với một giá trị số, trong trường hợp này là một số nguyên. Nếu chúng ta không khởi tạo cho các thành phần này thì chúng sẽ nhận các giá trị tiếp theo với thành phần đầu tiên là 0.

Ta xem thử khai báo sau:

```
enum Thutu{
    ThuNhat,
    ThuHai,
    ThuBa = 10,
    ThuTu
}
```

Khi đó giá trị của ThuNhat là 0, giá trị của ThuHai là 1, giá trị của ThuBa là 10 và giá trị của ThuTu là 11.

Kiểu liệt kê là một kiểu hình thức do đó bắt buộc phải thực hiện phép chuyển đổi tường minh với các kiểu giá trị nguyên:

```
int x = (int) ThuTu.ThuNhat;
```

Kiểu chuỗi ký tự

Kiểu dữ liệu chuỗi khá thân thiện với người lập trình trong bất cứ ngôn ngữ lập trình nào, kiểu dữ liệu chuỗi lưu giữ một mảng những ký tự.

Để khai báo một chuỗi chúng ta sử dụng từ khóa string tương tự như cách tạo một thể hiện của bất cứ đối tượng nào:

```
string chuoi;
```

Một hằng chuỗi được tạo bằng cách đặt các chuỗi trong dấu nháy đôi:

```
“Xin chào”
```

Đây là cách chung để khởi tạo một chuỗi ký tự với giá trị hằng:

```
string chuoi = “Xin chào”
```

1.2. Biến và hằng

Một biến là một vùng lưu trữ với một kiểu dữ liệu. Để tạo một biến chúng ta phải khai báo kiểu của biến và gán cho biến một tên duy nhất. Biến có thể được khởi tạo giá trị ngay khi được khai báo, hay nó cũng có thể được gán một giá trị mới vào bất cứ lúc nào trong chương trình.

Các biến trong C# được khai báo theo công thức như sau: AccessModifier DataType VariableName; Trong đó: AccessModifier: xác định ưu tiên truy xuất tới biến Datatype: định nghĩa kiểu lưu trữ dữ liệu của biến VariableName: là tên

biến Cấp độ truy xuất tới biến được mô tả như bảng dưới đây

Access Modifier	Mô tả
<code>public</code>	Truy cập tại bất kỳ nơi đâu
<code>protected</code>	Cho phép truy xuất bên trong một lớp nơi biến này được định nghĩa, hoặc từ các lớp con của lớp đó.
<code>private</code>	Chỉ truy xuất ở bên trong lớp nơi mà biến được định nghĩa.

Ví dụ bạn khai báo một biến kiểu `int`

```
int bien1;
```

Bạn có thể khởi gán ngay cho biến đó trong lúc khai báo `int bien1 = 9;` hoặc có thể gán giá trị sau khi khai báo như sau:

```
int bien1;
```

```
bien1 = 9;
```

Cách khai báo biến tương ứng với các kiểu dữ liệu:

C# Data Type	Ví dụ
<code>object</code>	<code>object obj = null;</code>
<code>string</code>	<code>string str = "Welcome";</code>
<code>int</code>	<code>int ival = 12;</code>
<code>byte</code>	<code>byte val = 12;</code>
<code>float</code>	<code>float val = 1.23F;</code>
<code>bool</code>	<code>bool val1 = false;</code> <code>bool val2 = true;</code>
<code>char</code>	<code>char cval = 'a';</code>

Ví dụ sau sẽ minh họa cách sử dụng biến:

```
namespace SuDungBien
{
    class MinhHoa
    {
        static void Main()
        {
            int bien1 = 9; // khai báo và khởi tạo giá trị cho một biến
            System.Console.WriteLine("Sau khi khai tạo: bien1 ={0}", bien1);
            bien1 = 15; // gán giá trị cho biến
            System.Console.WriteLine("Sau khi gán: bien1 ={0}", bien1);
            Console.ReadLine();
        }
    }
}
```

Các bạn hãy chú ý đến màu sắc của đoạn code trên. Các chữ có màu xanh dương là từ khóa, phần văn bản màu xanh lục sau dấu sổ chéo “//” là các chú thích, phần text nằm trong dấu “” có màu đỏ là các kí tự. Lệnh Write và WriteLine có phân biệt việc in ra màn hình biến và kí tự

Hằng

Hằng cũng là một biến nhưng giá trị của hằng không thay đổi. Biến là công cụ rất mạnh, tuy nhiên khi làm việc với một giá trị được định nghĩa là không thay đổi, ta phải đảm bảo giá trị của nó không được thay đổi trong suốt chương trình. Ví dụ, khi lập một chương trình thí nghiệm hóa học liên quan đến nhiệt độ sôi, hay nhiệt độ đông của nước, chương trình cần khai báo hai biến là DoSoi và DoDong, nhưng không cho phép giá trị của hai biến này bị thay đổi hay bị gán. Để ngăn ngừa việc gán giá trị khác, ta phải sử dụng biến kiểu hằng.

Hằng được phân thành ba loại: giá trị hằng (literal), biểu tượng hằng (symbolic constants), kiểu liệt kê (enumerations).

Giá trị hằng: ta có một câu lệnh gán như sau:

```
x = 100;
```

Giá trị 100 là giá trị hằng. Giá trị của 100 luôn là 100. Ta không thể gán giá trị khác cho 100 được.

Biểu tượng hằng: gán một tên cho một giá trị hằng, để tạo một biểu tượng hằng dùng từ khóa const và cú pháp sau:

```
<const> <type> <tên hằng> = <giá trị>;
```

Một biểu tượng hằng phải được khởi tạo khi khai báo, và chỉ khởi tạo duy nhất một lần trong suốt chương trình và không được thay đổi.

Ví dụ:

```
const int DoSoi = 100;
```

Trong khai báo trên, 32 là một hằng số và DoSoi là một biểu tượng hằng có kiểu nguyên.

□ Ví dụ 1.2: Sử dụng biểu tượng hằng.

```
-----
class MinhHoaC3{
static void Main(){
    const int DoSoi = 100;
    const int DoDong = 0;
    System.Console.WriteLine( “Do dong cua nuoc {0}”, DoDong );
    System.Console.WriteLine( “Do soi cua nuoc {0}”, DoSoi );
}
}
```

□ Kết quả:

Do dong cua nuoc 0

Do soi cua nuoc 100

Ví dụ tạo ra hai biểu tượng hằng chứa giá trị nguyên: DoSoi và DoDong, theo qui tắc đặt tên hằng thì tên hằng thường được đặt theo cú pháp Pascal, nhưng điều này không đòi hỏi bởi ngôn ngữ nên ta có thể đặt tùy ý.

Việc dùng biểu thức hằng này sẽ làm cho chương trình được viết tăng thêm phần ý nghĩa cùng với sự dễ hiểu. Thật sự chúng ta có thể dùng hằng số là 0 và 100 thay thế cho hai biểu tượng hằng trên, nhưng khi đó chương trình không được dễ hiểu và không được tự nhiên lắm. Trình biên dịch không bao giờ chấp nhận một lệnh gán giá trị mới cho một biểu tượng hằng.

Ví dụ trên có thể được viết lại như sau

```
class MinhHoaC3
{
static void Main()
{
    const int DoSoi = 100;
    const int DoDong = 0;
    // Độ C
    // Độ C
    System.Console.WriteLine( "Do dong cua nuoc {0}", DoDong );
    System.Console.WriteLine( "Do soi cua nuoc {0}", DoSoi );
    DoSoi = 200;
}
}
```

Khi đó trình biên dịch sẽ phát sinh một lỗi sau:

error CS0131: The left-hand side of an assignment must be a variable, property or indexer.

1.3. Biểu thức

Những câu lệnh mà thực hiện việc đánh giá một giá trị gọi là biểu thức. Một phép gán một giá trị cho một biến cũng là một biểu thức:

```
var1 = 24;
```

Trong câu lệnh trên phép đánh giá hay định lượng chính là phép gán có giá

trị là 24 cho biến `var1`. Lưu ý là toán tử gán (`=`) không phải là toán tử so sánh. Do vậy khi sử dụng toán tử này thì biến bên trái sẽ nhận giá trị của phần bên phải.

Do `var1 = 24` là một biểu thức được định giá trị là 24 nên biểu thức này có thể được xem như phần bên phải của một biểu thức gán khác:

```
var2 = var1 = 24;
```

Lệnh này sẽ được thực hiện từ bên phải sang khi đó biến `var1` sẽ nhận được giá trị là 24 và tiếp sau đó thì `var2` cũng được nhận giá trị là 24. Do vậy cả hai biến đều cùng nhận một giá trị là 24. Có thể dùng lệnh trên để khởi tạo nhiều biến có cùng một giá trị như:

```
a = b = c = d = 24;
```

1.4. Khoảng trắng

Trong ngôn ngữ C#, những khoảng trắng, khoảng tab và các dòng được xem như là khoảng trắng (whitespace), giống như tên gọi vì chỉ xuất hiện những khoảng trắng để đại diện cho các ký tự đó. C# sẽ bỏ qua tất cả các khoảng trắng đó, do vậy chúng ta có thể viết như sau:

```
var1 = 24;
```

hay

```
var1= 24 ;
```

và trình biên dịch C# sẽ xem hai câu lệnh trên là hoàn toàn giống nhau.

Tuy nhiên lưu ý là khoảng trắng trong một chuỗi sẽ không được bỏ qua. Nếu chúng ta viết:

```
System.WriteLine("Xin chào!");
```

mỗi khoảng trắng ở giữa hai chữ "Xin" và "chào" đều được đối xử bình thường như các ký tự khác trong chuỗi.

Tuy nhiên, cũng cần lưu ý khi sử dụng khoảng trắng như sau:

```
int x = 24;
```

tương tự như:

```
int x=24;
```

nhưng không giống như:

```
intx=24;
```

Trình biên dịch nhận biết được các khoảng trắng ở hai bên của phép gán là phụ và có thể bỏ qua, nhưng khoảng trắng giữa khai báo kiểu và tên biến thì không phải phụ hay thêm mà bắt buộc phải có tối thiểu một khoảng trắng. Điều này không có gì bất hợp lý, vì khoảng trắng cho phép trình biên dịch nhận biết được từ khoá `int` và không thể nào nhận được `intx`.

Tương tự như C/C++, trong C# câu lệnh được kết thúc với dấu chấm phẩy `;`. Do vậy có thể một câu lệnh trên nhiều dòng, và một dòng có thể nhiều câu

lệnh nhưng nhất thiết là hai câu lệnh phải cách nhau một dấu chấm phẩy.

1.5. Câu lệnh

Trong C# một chỉ dẫn lập trình đầy đủ được gọi là câu lệnh. Chương trình bao gồm nhiều câu lệnh tuần tự với nhau. Mỗi câu lệnh phải kết thúc với một dấu chấm phẩy, ví dụ như:

```
int x; // một câu lệnh
x = 32; // câu lệnh khác
int y = x; // đây cũng là một câu lệnh
```

Những câu lệnh này sẽ được xử lý theo thứ tự. Đầu tiên trình biên dịch bắt đầu ở vị trí đầu của danh sách các câu lệnh và lần lượt đi từng câu lệnh cho đến lệnh cuối cùng, tuy nhiên chỉ đúng cho trường hợp các câu lệnh tuần tự không phân nhánh.

Khi trình biên dịch xử lý đến tên của một hàm, thì sẽ ngưng thực hiện hàm hiện thời mà bắt đầu phân nhánh để tạo một gọi hàm mới. Sau khi hàm vừa tạo thực hiện xong và trả về một giá trị thì trình biên dịch sẽ tiếp tục thực hiện dòng lệnh tiếp sau của hàm ban đầu. ví dụ 1.3 minh họa cho việc phân nhánh khi gọi hàm.

□ Ví dụ 1.3: Gọi một hàm.

```
-----
using System;
class GoiHam
{
    static void Main()
    {
        Console.WriteLine( "Ham Main chuan bi gọi ham Func()..." );
        Func();
        Console.WriteLine( "Tro lai ham Main()" );
    }
    static void Func()
    {
        Console.WriteLine( "---->Toi la ham Func()..." );
    }
}
-----
```

□ Kết quả:

Ham Main chuan bi gọi ham Func()...

---->Toi la ham Func()...

Tro lai ham Main()

Luồng chương trình thực hiện bắt đầu từ hàm Main xử lý đến dòng lệnh Func(), lệnh Func() thường được gọi là một lời gọi hàm. Tại điểm này luồng chương trình sẽ rẽ nhánh để thực hiện hàm vừa gọi. Sau khi thực hiện xong hàm Func, thì chương trình quay lại hàm Main và thực hiện câu lệnh ngay sau câu lệnh gọi hàm Func.

C# cung cấp hai cấu trúc điều khiển thực hiện việc lựa chọn điều kiện thực thi chương trình đó là cấu trúc if và switch...case

Cấu trúc if trong C# được mô tả như sau:

```
if (biểu thức điều kiện)
{
    // câu lệnh thực thi nếu biểu thức điều kiện đúng
}
[else
{
    // câu lệnh thực thi nếu biểu thức điều kiện sai
}]
```

Ví dụ:

```
using System;
class ExIfElse
{
    static void Main()
    {
        int var1 = 10;
        int var2 = 20;
        if ( var1 > var2)
        {
            Console.WriteLine( "var1: {0} > var2:{1}", var1, var2);
        }
        else
        {
            Console.WriteLine( "var2: {0} > var1:{1}", var2, var1);
        }
    }
}
```

```

}
var1 = 30;
if ( var1 > var2)
{
var2 = var1++;
Console.WriteLine( "Gan gia tri var1 cho var2");
Console.WriteLine( "Tang bien var1 len mot ");
Console.WriteLine( "Var1 = {0}, var2 = {1}", var1, var2);
}
else
{
var1 = var2;
Console.WriteLine( "Thiet lap gia tri var1 = var2" );
Console.WriteLine( "var1 = {0}, var2 = {1}", var1, var2 );
}
}
}
}

```

□ Kết quả:

Gan gia tri var1 cho var2

Tang bien var1 len mot

Var1 = 31, var2 = 30

Cấu trúc switch ... case

Khi có quá nhiều điều kiện để chọn thực hiện thì dùng câu lệnh if sẽ rất rối rắm và dài dòng, Các ngôn ngữ lập trình cấp cao đều cung cấp một dạng câu lệnh switch liệt kê các giá trị và chỉ thực hiện các giá trị thích hợp. C# cũng cung cấp câu lệnh nhảy switch có cú pháp sau:

switch (biểu thức điều kiện)

{

case <giá trị>:

<Các câu lệnh thực hiện>

<lệnh nhảy>

[default:

<Các câu lệnh thực hiện mặc định>]

}

Cũng tương tự như câu lệnh if, biểu thức để so sánh được đặt sau từ khóa switch, tuy nhiên giá trị so sánh lại được đặt sau mỗi các từ khóa case. Giá trị sau từ khóa case là các giá trị hằng số nguyên như đã đề cập trong phần trước.

Nếu một câu lệnh case được thích hợp tức là giá trị sau case bằng với giá trị của biểu thức sau switch thì các câu lệnh liên quan đến câu lệnh case này sẽ được thực thi. Tuy nhiên phải có một câu lệnh nhảy như break, goto để điều khiển nhảy qua các case khác. Vì nếu không có các lệnh nhảy này thì khi đó chương trình sẽ thực hiện tất cả các case theo sau. Để dễ hiểu hơn ta sẽ xem xét ví dụ 1.4 dưới đây.

□ Ví dụ 1.4: Câu lệnh switch.

```
-----
using System;
class MinhHoaSwitch
{
static void Main()
{
const int mauDo = 0;
const int mauCam = 1;
const int mauVang = 2;
const int mauLuc = 3;
const int mauLam = 4;
const int mauCham = 5;
const int mauTim = 6;
int chonMau = mauLuc;
switch ( chonMau )
{
case mauDo:
Console.WriteLine( "Ban cho mau do" );
break;
case mauCam:
Console.WriteLine( "Ban cho mau cam" );
break;
case mauVang:
```

```
//Console.WriteLine( "Ban chon mau vang");
case mauLuc:
Console.WriteLine( "Ban chon mau luc");
break;
case mauLam:
Console.WriteLine( "Ban chon mau lam");
goto case mauCham;
case mauCham:
Console.WriteLine( "Ban cho mau cham");
goto case mauTim;
case mauTim:
Console.WriteLine( "Ban chon mau tim");
goto case mauLuc;
default:
Console.WriteLine( "Ban khong chon mau nao het");
break;
}
Console.WriteLine( "Xin cam on!");
}
}
```

□ Ghi chú: Đối với người lập trình C/C++, trong C# chúng ta không thể nhảy xuống một trường hợp case tiếp theo nếu câu lệnh case hiện tại không rỗng. Vì vậy chúng ta phải viết như sau:

```
case 1:
// nhảy xuống
case 2:
```

Như minh họa trên thì trường hợp xử lý case 1 là rỗng, tuy nhiên chúng ta không thể viết như sau:

```
case 1:
DoAnything();
// Trường hợp này không thể nhảy xuống case 2
case 2:
```

trong đoạn chương trình thứ hai trường hợp case 1 có một câu lệnh nên không thể nhảy xuống được. Nếu muốn trường hợp case1 nhảy qua case 2 thì ta

phải sử dụng câu lệnh goto một các trường minh:

case 1:

DoAnything();

goto case 2;

case 2:

Do vậy khi thực hiện xong các câu lệnh của một trường hợp nếu muốn thực hiện một trường hợp case khác thì ta dùng câu lệnh nhảy goto với nhãn của trường hợp đó:

goto case <giá trị>

Khi gặp lệnh thoát break thì chương trình thoát khỏi switch và thực hiện lệnh tiếp sau khối switch đó.

Nếu không có trường hợp nào thích hợp và trong câu lệnh switch có dùng câu lệnh default thì các câu lệnh của trường hợp default sẽ được thực hiện. Ta có thể dùng default để cảnh báo một lỗi hay xử lý một trường hợp ngoài tất cả các trường hợp case trong switch.

Trong ví dụ minh họa câu lệnh switch trước thì giá trị để kiểm tra các trường hợp thích hợp là các hằng số nguyên. Tuy nhiên C# còn có khả năng cho phép chúng ta dùng câu lệnh switch với giá trị là một chuỗi, có thể viết như sau:

switch (chuoi1)

{

case “mau do”:

....

break;

case “mau cam”:

Câu lệnh lặp

C# cung cấp một bộ mở rộng các câu lệnh lặp, bao gồm các câu lệnh lặp for, foreach, while và do... while. Ngoài ra ngôn ngữ C# còn bổ sung thêm một câu lệnh lặp foreach, lệnh này mới đối với người lập trình C/C++ nhưng khá thân thiện với người lập trình VB. Cuối cùng là các câu lệnh nhảy như goto, break, continue, và return.

Vòng lặp while

Ý nghĩa của vòng lặp while là: “Trong khi điều kiện đúng thì thực hiện các công việc này”.

Cú pháp sử dụng vòng lặp while như sau:

while (Biểu thức){

<Câu lệnh thực hiện>}

Biểu thức của vòng lặp while là điều kiện để các lệnh được thực hiện, biểu thức này bắt buộc phải trả về một giá trị kiểu bool là true/false. Nếu có nhiều câu lệnh cần được thực hiện trong vòng lặp while thì phải đặt các lệnh này trong khối lệnh. Ví dụ 1.5 minh họa việc sử dụng vòng lặp while.

□ Ví dụ 1.5: Sử dụng vòng lặp while.

```
using System;
public class UsingWhile
{
    public static int Main()
    {
        int i = 0;
        while ( i < 10 )
        {
            Console.WriteLine(" i: {0} ",i);
            i++;
        }
        return 0;
    }
}
```

□ Kết quả:

```
i:0
i:1
i:2
i:3
i:4
i:5
i:6
i:7
i:8
i:9
```

Đoạn chương trình 1.5 cũng cho kết quả tương tự như chương trình minh họa 1.4 dùng lệnh goto. Tuy nhiên chương trình 1.5 rõ ràng hơn và có ý nghĩa tự nhiên hơn. Có thể diễn giải ngôn ngữ tự nhiên đoạn vòng lặp while như sau: “Trong khi i nhỏ hơn 10, thì in ra giá trị của i và tăng i lên một đơn vị”.

Lưu ý rằng vòng lặp while sẽ kiểm tra điều kiện trước khi thực hiện các lệnh bên trong, điều này đảm bảo nếu ngay từ đầu điều kiện sai thì vòng lặp sẽ không bao giờ thực hiện. do vậy nếu khởi tạo biến i có giá trị là 11, thì vòng lặp sẽ không được thực hiện.

Vòng lặp do...while

Đôi khi vòng lặp while không thoả mãn yêu cầu trong tình huống sau, chúng ta muốn chuyển ngữ nghĩa của while là “chạy trong khi điều kiện đúng” thành ngữ nghĩa khác như “làm điều này trong khi điều kiện vẫn còn đúng”. Nói cách khác thực hiện một hành động, và sau khi hành động được hoàn thành thì kiểm tra điều kiện. Cú pháp sử dụng vòng lặp do...while như sau:

```
do
{<Câu lệnh thực hiện>
}
while ( điều kiện )
```

Ở đây có sự khác biệt quan trọng giữa vòng lặp while và vòng lặp do...while là khi dung vòng lặp do...while thì tối thiểu sẽ có một lần các câu lệnh trong do...while được thực hiện.

Vòng lặp for

Vòng lặp for bao gồm ba phần chính:

Khởi tạo biến đếm vòng lặp

Kiểm tra điều kiện biến đếm, nếu đúng thì sẽ thực hiện các lệnh bên trong vòng for

Thay đổi bước lặp.

Cú pháp sử dụng vòng lặp for như sau:

```
for ([ phần khởi tạo] ; [biểu thức điều kiện]; [bước lặp])
<Câu lệnh thực hiện>
```

Vòng lặp for được minh họa trong ví dụ sau:

□ Ví dụ 1.6: Sử dụng vòng lặp for.

```
using System;
public class UsingFor
{
```

```

public static int Main()
{
    for (int i = 0; i < 30; i++)
    {
        if (i % 10 == 0)
        {
            Console.WriteLine("{0} ", i);
        }
        else
        {
            Console.Write("{0} ", i);
        }
    }
    return 0;
}

```

□ Kết quả:

```

0
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29

```

Trong đoạn chương trình trên có sử dụng toán tử chia lấy dư modulo, toán tử này sẽ được đề cập đến phần sau. Ý nghĩa lệnh `i % 10 == 0` là kiểm tra xem `i` có phải là bội số của 10 không, nếu `i` là bội số của 10 thì sử dụng lệnh `WriteLine` để xuất giá trị `i` và sau đó đưa cursor về đầu dòng sau. Còn ngược lại chỉ cần xuất giá trị của `i` và không xuống dòng.

Đầu tiên biến `i` được khởi tạo giá trị ban đầu là 0, sau đó chương trình sẽ kiểm tra điều kiện, do 0 nhỏ hơn 30 nên điều kiện đúng, khi đó các câu lệnh bên trong vòng lặp `for` sẽ được thực hiện. Sau khi thực hiện xong thì biến `i` sẽ được tăng thêm một đơn vị (`i++`).

Có một điều lưu ý là biến `i` do khai báo bên trong vòng lặp `for` nên chỉ có phạm vi hoạt động bên trong vòng lặp.

Câu lệnh lặp foreach

Vòng lặp foreach cho phép tạo vòng lặp thông qua một tập hợp hay một mảng. Đây là một câu lệnh lặp mới không có trong ngôn ngữ C/C++. Câu lệnh foreach có cú pháp chung như sau:

foreach (<kiểu tập hợp> <tên truy cập thành phần > in < tên tập hợp>)
<Các câu lệnh thực hiện>

Do lặp dựa trên một mảng hay tập hợp nên toàn bộ vòng lặp sẽ duyệt qua tất cả các thành phần của tập hợp theo thứ tự được sắp. Khi duyệt đến phần tử cuối cùng trong tập hợp thì chương trình sẽ thoát ra khỏi vòng lặp foreach.

□ Ví dụ 1.7 minh họa việc sử dụng vòng lặp foreach.

```
-----
using System;
public class UsingForeach
{
    public static int Main()
    {
        int[] intArray = {1,2,3,4,5,6,7,8,9,10};
        foreach( int item in intArray)
        {
            Console.Write("{0} ", item);
        }
        return 0;
    }
}
-----
```

□ Kết quả:

1 2 3 4 5 6 7 8 9 10

Câu lệnh nhảy goto

Lệnh nhảy goto là một lệnh nhảy đơn giản, cho phép chương trình nhảy vô điều kiện tới một vị trí trong chương trình thông qua tên nhãn. Goto giúp chương trình của bạn được linh hoạt hơn nhưng trong nhiều trường hợp nó sẽ làm mất đi cấu trúc thuật toán và gây rối chương trình. Cách sử dụng lệnh goto: Tạo một nhãn goto đến nhãn Nhãn là một định danh theo sau bởi dấu hai chấm (:). Thường thường một lệnh goto gắn với một điều kiện nào đó.

Ví dụ 1.8:

```

public class UsingGoto
{
    public static void Main()
    {
        int i = 0;
        lap: // nhãn
            Console.WriteLine("i:{0}",i);
        i++;
        if ( i < 10 )
            goto lap; // nhảy về nhãn lap
        Console.ReadLine();
    }
}

```

Tương đương với vòng lặp for sau:

```
for (int i = 0; i < 10; i++)
```

```
Console.WriteLine("i: {0}", i);
```

Câu lệnh nhảy break và continue

Khi đang thực hiện các lệnh trong vòng lặp, có yêu cầu như sau: không thực hiện các lệnh còn lại nữa mà thoát khỏi vòng lặp, hay không thực hiện các công việc còn lại của vòng lặp hiện tại mà nhảy qua vòng lặp tiếp theo. Để đáp ứng yêu cầu trên C# cung cấp hai lệnh nhảy là break và continue để thoát khỏi vòng lặp. Break khi được sử dụng sẽ đưa chương trình thoát khỏi vòng lặp và tiếp tục thực hiện các lệnh tiếp ngay sau vòng lặp. Continue ngừng thực hiện các công việc còn lại của vòng lặp hiện thời và quay về đầu vòng lặp để thực hiện bước lặp tiếp theo.

Ví dụ 1.9: `public class UsingBreak_Continue`

```

{
    public static void Main()
    {
        for (int i = 0; i < 10; i++)
        {
            if (i % 2 == 0) continue;
            Console.WriteLine("i:{0}", i);
            if (i==7) break;
        }
        Console.ReadLine();
    }
}

```

Nếu không có lệnh break và continue vòng lặp sẽ lần lượt in ra các số từ 0 đến 9 nhưng khi gặp I chẵn ($i \% 2 == 0$) thì nó sẽ continue – tức là không thực hiện các lệnh tiếp theo mà quay trở lại đầu vòng lặp với giá trị của I được tăng lên 1. Lệnh break được thực hiện khi ($i == 7$) nó sẽ thoát khỏi vòng lặp ngay lập tức và cũng kết thúc chương trình và kết quả là chương trình trên chỉ in ra các số lẻ từ 1 đến 7

1.6. Toán tử

Toán tử được kí hiệu bằng một biểu tượng dùng để thực hiện một hành động. Các kiểu dữ liệu cơ bản của C# như kiểu nguyên hỗ trợ rất nhiều các toán tử như toán tử gán, toán tử toán học, logic....

a. Toán tử gán

Đến lúc này toán tử gán khá quen thuộc với chúng ta, hầu hết các chương trình minh họa từ đầu đều đã sử dụng phép gán. Toán tử gán hay phép gán làm cho toán hạng bên trái thay đổi giá trị bằng với giá trị của toán hạng bên phải. Toán tử gán là toán tử hai ngôi. Đây là toán tử đơn giản nhất thông dụng nhất và cũng dễ sử dụng nhất. ví dụ $a = b$

b. Toán tử toán học

Ngôn ngữ C# cung cấp năm toán tử toán học, bao gồm bốn toán tử đầu các phép toán cơ bản. Toán tử cuối cùng là toán tử chia nguyên lấy phần dư.

□ Các phép toán số học cơ bản (+, -, *, /)

Các phép toán này không thể thiếu trong bất cứ ngôn ngữ lập trình nào, C# cũng không ngoại lệ, các phép toán số học đơn giản nhưng rất cần thiết bao gồm: phép cộng (+), phép trừ (-), phép nhân (*), phép chia (/) nguyên và không nguyên.

Khi chia hai số nguyên, thì C# sẽ bỏ phần phân số, hay bỏ phần dư, tức là nếu ta chia $8/3$ thì sẽ được kết quả là 2 và sẽ bỏ phần dư là 2, do vậy để lấy được phần dư này thì C# cung cấp thêm toán tử lấy dư sẽ được trình bày trong phần kế tiếp.

Tuy nhiên, khi chia cho số thực có kiểu như float, double, hay decimal thì kết quả chia được trả về là một số thực.

□ Phép toán chia lấy dư

Để tìm phần dư của phép chia nguyên, chúng ta sử dụng toán tử chia lấy dư (%). Ví dụ, câu lệnh sau $8 \% 3$ thì kết quả trả về là 2 (đây là phần dư còn lại của phép chia nguyên). Thật sự phép toán chia lấy dư rất hữu dụng cho người lập trình. Khi chúng ta thực hiện một phép chia dư n cho một số khác, nếu số này là bội số của n thì kết quả của phép chia dư là 0.

Ví dụ $20 \% 5 = 0$ vì 20 là một bội số của 5. Điều này cho phép chúng ta ứng dụng trong vòng lặp, khi muốn thực hiện một công việc nào đó cách khoảng n lần, ta chỉ cần kiểm tra phép chia dư n, nếu kết quả bằng 0 thì thực hiện công việc. Cách sử dụng này đã áp dụng trong ví dụ minh họa sử dụng vòng lặp for bên trên.

```

using System;
class Tester
{
    public static void Main()
    {
        int i1, i2;
        float f1, f2;
        double d1, d2;
        decimal dec1, dec2;
        i1 = 17;
        i2 = 4;
        f1 = 17f;
        f2 = 4f;
        d1 = 17;
        d2 = 4;
        dec1 = 17;
        dec2 = 4;
        Console.WriteLine("Integer: \t{0}", i1/i2);
        Console.WriteLine("Float: \t{0}", f1/f2);
        Console.WriteLine("Double: \t{0}", d1/d2);
        Console.WriteLine("Decimal: \t{0}", dec1/dec2);
        Console.WriteLine("\nModulus: : \t{0}", i1%i2);
        Console.ReadLine();
    }
}

```

Kết quả: Integer: 4 float: 4.25 double: 4.25 decimal: 4.25 Modulus: 1

c. Toán tử tăng và giảm

Khi sử dụng các biến số ta thường có thao tác là cộng một giá trị vào biến, trừ đi một giá trị từ biến đó, hay thực hiện các tính toán thay đổi giá trị của biến sau đó gán giá trị mới vừa tính toán cho chính biến đó.

d. Tính toán và gán trở lại

Giả sử chúng ta có một biến tên Luong lưu giá trị lương của một người, biến Luong này có giá trị hiện thời là 1.500.000, sau đó để tăng thêm 200.000 ta có thể viết như sau:

$Luong = Luong + 200.000;$

Trong câu lệnh trên phép cộng được thực hiện trước, khi đó kết quả của vế phải là 1.700.000 và kết quả này sẽ được gán lại cho biến Luong, cuối cùng Luong có giá trị là 1.700.000.

Chúng ta có thể thực hiện việc thay đổi giá trị rồi gán lại cho biến với bất kỳ phép toán số học nào:

$Luong = Luong * 2;$

$Luong = Luong - 100.000;$

...

Do việc tăng hay giảm giá trị của một biến rất thường xảy ra trong khi tính

toán nên C# cung cấp các phép toán tự gán (self- assignment). Bảng sau liệt kê các phép toán tự gán.

Toán tử	Ý nghĩa	Tương đương
+=	Cộng rồi gán	$a = a + b$
-=	Trừ rồi gán	$a = a - b$
*=	Nhân rồi gán	$a = a * b$
/=	Chia rồi gán	$a = a / b$
&=	AND bit rồi gán	$a = a \& b$

Dựa trên các phép toán tự gán trong bảng ta có thể thay thế các lệnh tăng giảm lương như sau:

Luong += 200.000;

Luong *= 2;

Luong -= 100.000;

e. Toán tử tăng giảm tiền tố và tăng giảm hậu tố

Giả sử muốn kết hợp các phép toán như gia tăng giá trị của một biến và gán giá trị của biến cho biến thứ hai, ta viết như sau:

var1 = var2++;

Câu hỏi được đặt ra là gán giá trị trước khi cộng hay gán giá trị sau khi đã cộng. Hay nói cách khác giá trị ban đầu của biến var2 là 10, sau khi thực hiện ta muốn giá trị của var1 là 10, var2 là 11, hay var1 là 11, var2 cũng 11?

Để giải quyết yêu cầu trên C# cung cấp thứ tự thực hiện phép toán tăng/giảm với phép toán gán, thứ tự này được gọi là tiền tố (prefix) hay hậu tố (postfix). Do đó ta có thể viết:

var1 = var2++; // Hậu tố

Khi lệnh này được thực hiện thì phép gán sẽ được thực hiện trước tiên, sau đó mới đến phép toán tăng. Kết quả là var1 = 10 và var2 = 11. Còn đối với trường hợp tiền tố:

var1 = ++var2;

Khi đó phép tăng sẽ được thực hiện trước tức là giá trị của biến var2 sẽ là 11 và cuối cùng phép gán được thực hiện. Kết quả cả hai biến var1 và var2 đều có giá trị là 11.

f. Toán tử quan hệ

Những toán tử quan hệ được dùng để so sánh giữa hai giá trị, và sau đó trả về kết quả là một giá trị logic kiểu bool (true hay false). Ví dụ toán tử so sánh lớn hơn (>) trả về giá trị là true nếu giá trị bên trái của toán tử lớn hơn giá trị bên phải của toán tử. Do vậy $5 > 2$ trả về một giá trị là true, trong khi $2 > 5$ trả về giá trị false.

Các toán tử quan hệ trong ngôn ngữ C# được trình bày ở bảng 1.2 bên dưới. Các toán tử trong bảng được minh họa với hai biến là value1 và value2, trong đó value1 có giá trị là 100 và value2 có giá trị là 50.

Tên toán tử	Kí hiệu	Biểu thức so sánh	Kết quả so sánh
So sánh bằng	==	value1 == 100 value1 == 50	true false
Không bằng	!=	value2 != 100 value2 != 90	false true
Lớn hơn	>	value1 > value2 value2 > value1	true false
Lớn hơn hay bằng	>=	value2 >= 50	true
Nhỏ hơn	<	value1 < value2 value2 < value1	false true
Nhỏ hơn hay bằng	<=	value1 <= value2	false

Các toán tử so sánh (giả sử value1 = 100, và value2 = 50).

g. Toán tử logic

Trong câu lệnh **if** mà chúng ta đã tìm hiểu trong phần trước, thì khi điều kiện là true thì biểu thức bên trong **if** mới được thực hiện. Đôi khi chúng ta muốn kết hợp nhiều điều kiện với nhau như: bắt buộc cả hai hay nhiều điều kiện phải đúng hoặc chỉ cần một trong các điều kiện đúng là đủ hoặc không có điều kiện nào đúng...C# cung cấp một tập hợp các toán tử logic để phục vụ cho người lập trình.

Bảng liệt kê ba phép toán logic, bảng này cũng sử dụng hai biến minh họa là x, và y trong đó x có giá trị là 5 và y có giá trị là 7

Tên toán tử	Ký hiệu	Biểu thức logic	Giá trị	Logic
and	&&	(x == 3) && (y == 7)	false	Cả hai điều kiện phải đúng
or		(x == 3) (y == 7)	true	Chỉ cần một điều kiện đúng
not	!	! (x == 3)	true	Biểu thức trong ngoặc phải sai.

Các toán tử logic (giả sử x = 5, y = 7).

h. Toán tử ba ngôi

Hầu hết các toán tử đòi hỏi có một toán hạng như toán tử (++ , --) hay hai toán hạng như (+, -, *, /, ...). Tuy nhiên, C# còn cung cấp thêm một toán tử có ba toán hạng (? :). Toán tử này có cú pháp sử dụng như sau:

<Biểu thức điều kiện> ? <Biểu thức thứ 1> : <Biểu thức thứ 2>

Toán tử này sẽ xác định giá trị của một biểu thức điều kiện, và biểu thức

điều kiện này phải trả về một giá trị kiểu bool. Khi điều kiện đúng thì <biểu thức thứ 1> sẽ được thực hiện, còn ngược lại điều kiện sai thì <biểu thức thứ 2> sẽ được thực hiện. Có thể diễn giải theo ngôn ngữ tự nhiên thì toán tử này có ý nghĩa : *“Nếu điều kiện đúng thì làm công việc thứ nhất, còn ngược lại điều kiện sai thì làm công việc thứ hai”*.

1.7. Namespace

Mọi đối tượng của một chương trình C# đều phải đặt trong một class (hay lớp) và các class này sẽ được đặt trong một Namespace (dịch ra tiếng việt là không gian tên). Các Namespace phải có tên khác nhau, các class trong các namespace khác nhau thì có thể trùng tên. Khi đó tên các class sẽ được phân biệt bởi namespace chứa nó. Một ví dụ mô phỏng về Namespace và Lớp đó là: Hà Nội và Tp.HCM đều có đường mang tên Trần Hưng Đạo, nhưng rõ ràng là con đường Trần Hưng Đạo ở hai nơi hoàn toàn khác nhau mặc dù vẫn có chung tên Trần Hưng đạo. Vậy ta nói Hà Nội và Tp.HCM là hai namespace chứa hai lớp có cùng tên "Trần Hưng Đạo".

C# đưa ra từ khóa **using** để khai báo sử dụng namespace trong chương trình:

```
using < Tên namespace >
```

Để tạo một namespace dùng cú pháp sau:

```
namespace <Tên namespace>
```

```
{
```

```
< Định nghĩa lớp A>
```

```
< Định nghĩa lớp B >
```

```
.....
```

```
}
```

1.8. Các chỉ dẫn biên dịch

Đối với các ví dụ minh họa trong các phần trước, khi biên dịch thì toàn bộ chương trình sẽ được biên dịch. Tuy nhiên, có yêu cầu thực tế là chúng ta chỉ muốn một phần trong chương trình được biên dịch độc lập, ví dụ như khi debug chương trình hoặc xây dựng các ứng dụng...

Trước khi một mã nguồn được biên dịch, một chương trình khác được gọi là chương trình tiền xử lý sẽ thực hiện trước và chuẩn bị các đoạn mã nguồn để biên dịch. Chương trình tiền xử lý này sẽ tìm trong mã nguồn các kí hiệu chỉ dẫn biên dịch đặc biệt, tất cả các chỉ dẫn biên dịch này đều được bắt đầu với dấu rào (#). Các chỉ dẫn cho phép chúng ta định nghĩa các định danh và kiểm tra các sự tồn tại của các định danh đó.

Định nghĩa định danh

Câu lệnh tiền xử lý sau:

```
#define DEBUG
```

Lệnh trên định nghĩa một định danh tiền xử lý có tên là DEBUG. Mặc dù những chỉ thị tiền xử lý khác có thể được đặt bất cứ ở đâu trong chương trình, nhưng với chỉ thị định nghĩa định danh thì phải đặt trước tất cả các lệnh khác, bao gồm cả câu lệnh using.

Để kiểm tra một định danh đã được định nghĩa thì ta dùng cú pháp `#if <định danh>`. Do đó ta có thể viết như sau:

```
#define DEBUG

//...Các đoạn mã nguồn bình thường, không bị tác động bởi trình tiền xử lý
...

#if DEBUG
// Các đoạn mã nguồn trong khối if debug được biên dịch
#else
// Các đoạn mã nguồn không định nghĩa debug và không được biên dịch
#endif

//...Các đoạn mã nguồn bình thường, không bị tác động bởi trình tiền xử lý.
```

1.9. Xử lý ngoại lệ

* Phát sinh và bắt giữ ngoại lệ

Trong ngôn ngữ C#, chúng ta chỉ có thể phát sinh (throw) những đối tượng có kiểu dữ liệu là `System.Exception`, hay những đối tượng được dẫn xuất từ kiểu dữ liệu này. Namespace `System` của CLR chứa một số các kiểu dữ liệu xử lý ngoại lệ mà chúng ta có thể sử dụng trong chương trình. Những kiểu dữ liệu ngoại lệ này bao gồm `ArgumentNullException`, `InvalidCastException`, và `OverflowException`, cũng như nhiều lớp khác nữa.

a/ Câu lệnh throw

Để phát tín hiệu không bình thường trong một lớp của ngôn ngữ C#, chúng ta phát sinh một ngoại lệ. Để làm được điều này, chúng ta sử dụng từ khóa `throw`. Dòng lệnh sau tạo ra một thể hiện mới của `System.Exception` và sau đó `throw` nó: `throw new System.Exception();`

Khi phát sinh ngoại lệ thì ngay tức khắc sẽ làm ngừng việc thực thi trong khi CLR sẽ tìm kiếm một trình xử lý ngoại lệ. Nếu một trình xử lý ngoại lệ không được tìm thấy trong phương thức hiện thời, thì CLR tiếp tục tìm trong phương thức gọi cho đến khi nào tìm thấy. Nếu CLR trả về lớp `Main()` mà không tìm thấy bất cứ trình xử lý ngoại lệ nào, thì nó sẽ kết thúc chương trình.

Ví dụ 1.10:

```
namespace Programming_CSharp
{
    using System;
```

```

public class Test
{
    public static void Main()
    {
        Console.WriteLine("Enter Main....");
        Test t = new Test();
        t.Func1();
        Console.WriteLine("Exit Main...");
    }
    public void Func1()
    {
        Console.WriteLine("Enter Func1...");
        Func2();
        Console.WriteLine("Exit Func1...");
    }
    public void Func2()
    {
        Console.WriteLine("Enter Func2...");
        throw new System.Exception();
        Console.WriteLine("Exit Func2...");
    }
}

```

Ví dụ 1.10 minh họa đơn giản viết ra màn hình console thông tin khi nó nhập vào trong một hàm và chuẩn bị đi ra từ một hàm. Hàm Main() tạo thể hiện mới của kiểu Test và sau đó gọi hàm Func1(). Sau khi in thông điệp "Enter Func1", hàm Func1() này gọi hàm Func2(). Hàm Func2() in ra thông điệp đầu tiên và phát sinh một ngoại lệ kiểu System.Exception. Việc thực thi sẽ ngưng ngay tức khắc, và CLR sẽ tìm kiếm trình xử lý ngoại lệ trong hàm Func2(). Do không tìm thấy ở đây, CLR tiếp tục vào stack lấy hàm đã gọi trước tức là Func1 và tìm kiếm trình xử lý ngoại lệ. Một lần nữa trong Func1 cũng không có đoạn xử lý ngoại lệ. Và CLR trả về hàm Main. Tại hàm Main cũng không có, nên CLR sẽ gọi trình mặc định xử lý ngoại lệ, việc này đơn giản là xuất ra một thông điệp lỗi.

b/ Câu lệnh catch

Trong C#, một trình xử lý ngoại lệ hay một đoạn chương trình xử lý các ngoại lệ được gọi là một khối catch và được tạo ra với từ khóa catch.

```

try
{
    // Câu lệnh thực hiện có thể phát sinh lỗi
}
catch(Type x)
{
    // Câu lệnh xử lý khi gặp lỗi
}

```

Trong ví dụ sau, câu lệnh throw được thực thi bên trong khối try, và một khối catch được sử dụng để công bố rằng một lỗi đã được xử lý.

Ví dụ 1.11:

```

namespace Programming_CSharp
{
    using System;
    public class Test
    {
        public static void Main()
        {
            Console.WriteLine("Enter Main...");
            Test t = new Test();
            t.Func1();
            Console.WriteLine("Exit Main...");
        }
        public void Func1()
        {
            Console.WriteLine("Enter Func1...");
            Func2();
            Console.WriteLine("Exit Func1...");
        }
        public void Func2()
        {
            Console.WriteLine("Enter Func2...");
        }
    }
}

```

```

{
    Console.WriteLine("Entering try block...");
    throw new System.Exception();
    Console.WriteLine("Exiting try block...");
}
catch
{
    Console.WriteLine("Exception caught and handled.");
}
Console.WriteLine("Exit Func2...");
}
}
}

```

Ví dụ 1.11 cũng tương tự như ví dụ minh họa trong ví dụ 1.10 ngoại trừ chương trình thêm vào trong một khối try/catch. Thông thường chúng ta cũng có thể đặt khối try bao quanh những đoạn chương trình tiềm ẩn gây ra sự nguy hiểm, như là việc truy cập file, cấp phát bộ nhớ...

Theo sau khối try là câu lệnh catch tổng quát. Câu lệnh catch trong ví dụ này là tổng quát bởi vì chúng ta không xác định loại ngoại lệ nào mà chúng ta bắt giữ. Trong trường hợp tổng quát này thì khối catch này sẽ bắt giữ bất cứ ngoại lệ nào được phát sinh. Sử dụng câu lệnh catch để bắt giữ ngoại lệ xác định sẽ được thảo luận trong phần sau của chương.

Trong ví dụ 1.11 này, khối catch đơn giản là thông báo ra một ngoại lệ được bắt giữ và được xử lý. Trong ví dụ của thế giới thực, chúng ta có thể đưa hành động đúng để sửa chữa vấn đề mà gây ra sự ngoại lệ. Ví dụ, nếu người sử dụng đang cố mở một tập tin có thuộc tính chỉ đọc, chúng ta có thể gọi một phương thức cho phép người dùng thay đổi thuộc tính của tập tin.

Nếu chương trình thực hiện thiếu bộ nhớ, chúng ta có thể phát sinh cho người dùng cơ hội để đóng bớt các ứng dụng khác lại. Thậm chí trong trường hợp xấu nhất ta không khắc phục được thì khối catch này có thể in ra thông điệp lỗi để người dùng biết.

Thử kiểm tra kỹ lại chương trình trên, chúng ta sẽ thấy xuất hiện đoạn mã đi vào từng hàm như Main(), Func1(), Func2(), và cả khối try. Chúng ta không bao giờ thấy nó thoát khỏi lệnh try (tức là in ra thông báo "Exiting try block...", hay thực hiện lệnh này), mặc dù nó vẫn thoát ra theo thứ tự Func2(), Func1(), và Main(). Chuyện gì xảy ra?

Khi một ngoại lệ được phát sinh, việc thi hành ngay lập tức sẽ bị tạm dừng và việc thi hành sẽ được chuyển qua khối lệnh catch. Nó không bao giờ trả về

luồng thực hiện ban đầu, tức là các lệnh sau khi phát ra ngoại lệ trong khối `try` không được thực hiện. Trong trường hợp này chúng ta sẽ không bao giờ nhận được thông báo “Exiting try block...”. Khối lệnh `catch` xử lý lỗi và sau đó chuyển việc thực thi chương trình đến các lệnh tiếp sau khối `catch`.

c/ Câu lệnh **finally**

Trong một số tình huống, việc phát sinh ngoại lệ và `unwind stack` có thể tạo ra một số vấn đề. Ví dụ như nếu chúng ta mở một tập tin hay trường hợp khác là xác nhận một tài nguyên, chúng ta có thể cần thiết một cơ hội để đóng một tập tin hay là giải phóng bộ nhớ đệm mà chương trình đã chiếm giữ trước đó.

Ghi chú: Trong ngôn ngữ C#, vấn đề này ít xảy ra hơn do cơ chế thu dọn tự động của C# ngăn ngừa những ngoại lệ phát sinh từ việc thiếu bộ nhớ.

Tuy nhiên, có một số hành động mà chúng ta cần phải quan tâm bất cứ khi nào một ngoại lệ được phát sinh ra, như việc đóng một tập tin, chúng ta có hai chiến lược để lựa chọn thực hiện. Một hướng tiếp cận là đưa hành động nguy hiểm vào trong khối `try` và sau đó thực hiện việc đóng tập tin trong cả hai khối `catch` và `try`. Tuy nhiên, điều này gây ra đoạn chương trình không được đẹp do sử dụng trùng lặp lệnh. Ngôn ngữ C# cung cấp một sự thay thế tốt hơn trong khối `finally`.

```
try
{
    // Câu lệnh thực hiện có thể phát sinh lỗi
}
catch(Type x)
{
    // Câu lệnh xử lý khi gặp lỗi
}
finally
{
    // Câu lệnh luôn được thực thi
}
```

Đoạn chương trình bên trong khối `catch` được đảm bảo thực thi mà không quan tâm đến việc khi nào thì một ngoại lệ được phát sinh. Phương thức `TestFunc()` trong ví dụ sau minh họa việc mở một tập tin như là hành động đầu tiên của nó, sau đó phương thức thực hiện một vài các phép toán toán học, và sau đó là tập tin được đóng. Có thể trong quá trình mở tập tin cho đến khi đóng tập tin chương trình phát sinh ra một ngoại lệ. Nếu xuất hiện ngoại lệ, và khi đó tập tin vẫn còn mở. Người phát triển biết rằng không có chuyện gì xảy ra, và cuối của phương thức này thì tập tin sẽ được đóng. Do chức năng đóng tập tin được di chuyển vào trong khối `finally`, ở đây nó sẽ được thực thi mà không cần quan tâm đến việc có phát sinh hay không một ngoại lệ trong chương trình.

Ví dụ 1.12

```

namespace Programming_CSharp
{
    using System;
    public class Test
    {
        public static void Main()
        {
            Test t=new Test();
            t.TestFunc();
        }
        //chia hai số và xử lý ngoại lệ nếu có
        public void TestFunc()
        {
            try{
                Console.WriteLine("Open file here");
            {
                double a=5;
                double b=0;
                Console.WriteLine("{0}/{1}={2}",a,b,DoDivide(a,b));
                Console.WriteLine("This line may or not print");
            }
            catch(System.DivideByZeroException)
            {
                Console.WriteLine("DivideByZeroException caught!");
            }
            catch
            {
                Console.WriteLine("Unknown exception caught");
            }
            finally
            {
                Console.WriteLine("Close file here");
            }
        }
    }
}

```

```

    }
}
//thực hiện chia nếu hợp lệ
public double DoDivide(double a,double b)
{
    if(b==0)
    {
        throw new System.DivideByZeroException();
    }
    if(a==0)
    {throw new System.ArithmeticException();
    }
    return a/b;
}
}
}

```

Trong ví dụ này một khối catch được loại bỏ và thêm vào khối finally. Bất cứ khi một ngoại lệ có được phát sinh ra hay không thì khối lệnh bên trong finally cũng được thực thi. Do vậy nên trong cả hai trường hợp ta cũng thấy xuất hiện thông điệp “Close file here”.

* Những đối tượng lớp ngoại lệ

Đối tượng System.Exception cung cấp một số các phương thức và thuộc tính hữu dụng. Thuộc tính Message cung cấp thông tin về ngoại lệ, như là lý do tại sao ngoại lệ được phát sinh. Thuộc tính Message là thuộc tính chỉ đọc, đoạn chương trình phát sinh ngoại lệ có thể thiết lập thuộc tính Message như là một đối mục cho bộ khởi dựng của ngoại lệ. Thuộc tính HelpLink cung cấp một liên kết để trợ giúp cho các tập tin liên quan đến các ngoại lệ. Đây là thuộc tính chỉ đọc. Thuộc tính StackTrace cũng là thuộc tính chỉ đọc và được thiết lập bởi **CLR**. Trong ví dụ 1.12 thuộc tính Exception.HelpLink được thiết lập và truy cập để cung cấp thông tin cho người sử dụng về ngoại lệ DivideBy-ZeroException. Thuộc tính StackTrace của ngoại lệ được sử dụng để cung cấp thông tin stack cho câu lệnh lỗi. Một thông tin stack cung cấp hàng loạt các cuộc gọi stack của phương thức gọi mà dẫn đến những ngoại lệ được phát sinh.

Ví dụ 1.13:

```

namespace Programming_CSharp
{

```

```

using System;
public class Test
{
    public static void Main()
    {
        Test t=new Test();
        t.TestFunc();
    }
    //chia hai số và xử lý ngoại lệ
    public void TestFunc()
    {
        try
        {
            Console.WriteLine("Open file here");
            Double a=12;
            Double b=0;
            Console.WriteLine("{0}/{1}={2}",a,b,DoDivide(a,b));
            Console.WriteLine("This line may or not print");
        }
        catch{
            Console.WriteLine("\n DivideByZeroException! Msg: {0}",e.Message);
            Console.WriteLine("\n HelpLink: {0}",e.HelpLink);
            Console.WriteLine("\n Here's a stack trace: {0}\n",e.StackTrace);
        }
        catch
        {
            Console.WriteLine("Unknown exception caught");
        }
    }
    //thực hiện phép chia hợp lệ
    public double DoDivide(double a,double b)
    {
        if(b==0)

```

```

{
    DivideByZeroException e=new DivideByZeroException();
    e.HelpLink="http://www.agvc.edu.vn");
    throw e;
}
if(a==0)
{
    throw new ArithmeticException();
}
Return a/b;
}

```

Bảng sau mô tả một số các lớp ngoại lệ chung được khai báo bên trong namespace System.

CÁC LỚP NGOẠI LỆ	
Tên ngoại lệ	Mô tả
MethodAccessException	Lỗi truy cập, do truy cập đến thành viên hay phương thức không được truy cập
ArgumentException	Lỗi tham số đối mục
ArgumentNullException	Đối mục Null, phương thức được truyền đối mục null không được chấp nhận
ArithmeticException	Lỗi liên quan đến các phép toán
ArrayTypeMismatchException	Kiểu mảng không hợp, khi cố lưu trữ kiểu không thích hợp vào mảng
DivideByZeroException	Lỗi chia zero
FormatException	Định dạng không chính xác một đối mục nào đó
IndexOutOfRangeException	Chỉ số truy cập mảng không hợp lệ, dùng nhỏ hơn chỉ số nhỏ nhất hay lớn hơn chỉ số lớn nhất của mảng
InvalidCastException	Phép gán không hợp lệ
MulticastNotSupportedException	Multicast không được hỗ trợ, do việc kết hợp hai delegate không đúng
NotFiniteNumberException	Không phải số hữu hạn số không hợp lệ
NotSupportedException	Phương thức không hỗ trợ, khi gọi một phương thức không tồn tại bên trong lớp
NullReferenceException	Tham chiếu null không hợp lệ
OutOfMemoryException	Out of memory
OverflowException	Lỗi tràn phép toán
StackOverflowException	Tràn stack
TypeInitializationException	Kiểu khởi tạo sai, khi bộ khởi dựng tĩnh có lỗi

* Phát sinh ngoại lệ

Giả sử chúng ta muốn khối `catch` thực hiện một vài hành động đúng nào đó rồi sau đó phát sinh lại ngoại lệ ra bên ngoài khối `catch` (trong một hàm gọi). Chúng ta được phép phát sinh lại cùng một ngoại lệ hay phát sinh lại các ngoại lệ khác. Nếu phát sinh ra ngoại lệ khác, chúng ta có thể phải nhúng ngoại lệ ban đầu vào bên trong ngoại lệ mới để phương thức gọi có thể hiểu được lai lịch và nguồn gốc của ngoại lệ. Thuộc tính `InnerException` của ngoại lệ mới cho phép truy cập ngoại lệ ban đầu.

Bởi vì `InnerException` cũng là một ngoại lệ, nên nó cũng có một ngoại lệ bên trong. Do vậy, toàn bộ dây chuyền ngoại lệ là một sự đóng tổ (nest) của một ngoại lệ này với một ngoại lệ khác.

Ví dụ 1.14:

```
namespace Programming_CSharp
{
    using System;
    //tạo ngoại lệ riêng
    public class MyCustomException: System.Exception
    {
        public MyCustomException(string message, Exception inner):
        base(message,inner){}
    }
    public class Test
    {
        public static void Main()
        {
            Test t=new Test();
            t.TestFunc();
        }
        //chia hai số và xử lý ngoại lệ
        public void TestFunc()
        {
            try
            {
                DangerousFunc1();
            }
        }
    }
}
```

```

catch(MyCustomException e)
{
    Console.WriteLine("\n{0}",e.Message);
    Console.WriteLine("Retrieving exception history....");
    Exception inner=e.InnerException;
    While(inner!=null)
    {
        Console.WriteLine"{0}",inner.Message);
        Inner=inner.InnerException;
    }
}

public void DangerousFunc1()
{
    try
    {
        DangerousFunc2();
    }
    catch(System.Exception e)
    {
        MyCustomException ex=new MyCustomException ("E3-Custom
Exception Situation",e);
        throw ex;
    }
}

public void DangerousFunc2()
{
    try
    {
        DangerousFunc3();
    }
    catch(System.DivideByZeroException e)
    {

```

```

Exception ex=new Exception("E2-Func2 caught divide by zero",e);
Throw ex;
}
}
public void DangerousFunc3()
{
try
{
DangerousFunc4();
}
catch(System.ArithmeticException)
{
throw;
}
catch(System.Exception)
{
Console.WriteLine("Exception handled here");
}
}
public void DangerousFunc4()
{
throw new DivideByZeroException ("E1- DivideByZero Exception");
}
}

```

2. Xây dựng lớp – Đối tượng

2.1. Lớp và đối tượng

Một lớp bao gồm có các thuộc tính và phương thức. Để khai báo một lớp ta sử dụng từ khóa ***class*** với cấu trúc sau đây:

```

[Thuộc tính truy cập] class <tên lớp>
{
    Khai báo các thuộc tính của lớp
    Khai báo các phương thức của lớp
}

```

Thuộc tính	Giới hạn truy cập
public	Không hạn chế. Những thành viên được đánh dấu public có thể được dùng bởi bất kì các phương thức của lớp bao gồm những lớp khác.
private	Thành viên trong một lớp A được đánh dấu là private thì chỉ được truy cập bởi các phương thức của lớp A.
protected	Thành viên trong lớp A được đánh dấu là protected thì chỉ được các phương thức bên trong lớp A và những phương thức dẫn xuất từ lớp A truy cập.
internal	Thành viên trong lớp A được đánh dấu là internal thì được truy cập bởi những phương thức của bất cứ lớp nào trong cùng khối hợp ngữ với A.
protected internal	Thành viên trong lớp A được đánh dấu là protected internal được truy cập bởi các phương thức của lớp A, các phương thức của lớp dẫn xuất của A, và bất cứ lớp nào trong cùng khối hợp ngữ của A.

Mong muốn chung là thiết kế các biến thành viên của lớp ở thuộc tính **private**. Khi đó chỉ có phương thức thành viên của lớp truy cập được giá trị của biến. C# xem thuộc tính **private** là mặc định nên trong ví dụ sau ta không khai báo thuộc tính truy cập cho 6 biến nên mặc định chúng là **private**:

// Các biến thành viên private

```
int Nam;
int Thang;
int Ngay;
int Gio;
int Phut;
int Giay;
```

Do lớp Tester và phương thức thành viên ThoiGianHienHanh của lớp ThoiGian được khai báo là public nên bất kỳ lớp nào cũng có thể truy cập được.

□ *Ghi chú*: Thói quen lập trình tốt là khai báo tường minh các thuộc tính truy cập của biến thành viên hay các phương thức trong một lớp. Mặc dù chúng ta biết chắc chắn rằng các thành viên của lớp là được khai báo **private** mặc định. Việc khai báo tường minh này sẽ làm cho chương trình dễ hiểu, rõ ràng và tự nhiên hơn.

Tham số của phương thức

Trong các ngôn ngữ lập trình thì tham số và đối mục được xem là như nhau, cũng tương tự khi đang nói về ngôn ngữ hướng đối tượng thì ta gọi một hàm là một phương thức hay hành vi. Tất cả các tên này điều tương đồng với nhau.

Một phương thức có thể lấy bất kỳ số lượng tham số nào, Các tham số này theo sau bởi tên của phương thức và được bao bọc bên trong dấu ngoặc tròn (). Mỗi tham số phải khai báo kèm với kiểu dữ liệu. ví dụ ta có một khai báo định nghĩa một phương thức có tên là Method, phương thức không trả về giá trị nào cả (khai báo giá trị trả về là void), và có hai tham số là một kiểu int và button:

```
void Method( int param1, button param2)
{
//...
}
```

Bên trong thân của phương thức, các tham số này được xem như những biến cục bộ, giống như là ta khai báo biến bên trong phương thức và khởi tạo giá trị bằng giá trị của tham số truyền vào. Ví dụ 2.1 minh họa việc truyền tham số vào một phương thức, trong trường hợp này thì hai tham số của kiểu là int và float.

□ *Ví dụ 2.1: Truyền tham số cho phương thức.*

```
using System;
public class Class1
{
public void SomeMethod(int p1, float p2)
{
Console.WriteLine("Ham nhan duoc hai tham so: {0} va {1}",
p1,p2);
}
}
public class Tester
{
static void Main()
{
int var1 = 5;
float var2 = 10.5f;
Class1 c = new Class1();
c.SomeMethod( var1, var2 );
}
```

}

□ *Kết quả:*

Ham nhan duoc hai tham so: 5 va 10.5

Phương thức SomeMethod sẽ lấy hai tham số int và float rồi hiển thị chúng ta màn hình bằng việc dùng hàm Console.WriteLine(). Những tham số này có tên là p1 và p2 được xem như là biến cục bộ bên trong của phương thức.

Trong phương thức gọi Main, có hai biến cục bộ được tạo ra là var1 và var2. Khi hai biến này được truyền cho phương thức SomeMethod thì chúng được ánh xạ thành hai tham số p1 và p2 theo thứ tự danh sách biến đưa vào.

* Sử dụng các đối tượng của lớp:

Để sử dụng lớp ta phải khai báo đối tượng của lớp đó. Khi một đối tượng của lớp được tạo ra thì nó có đầy đủ các thuộc tính, phương thức của lớp và sử dụng thuộc tính và phương thức của lớp. Các thành phần của một lớp chỉ được sử dụng khi có thể hiện của lớp, trừ trường hợp trong lớp có một hàm khởi dựng là static. Để khai báo một đối tượng của lớp ta dùng từ khóa **new** và khai báo nó theo cấu trúc sau:

<tên lớp> <tên đối tượng> = **new** <tên lớp> ([các giá trị khởi tạo nếu

Để truy nhập đến một phương thức ta thông qua tên biến đối tượng và toán tử chấm “.”:

<tên đối tượng>.<tên phương thức> ([danh sách các đối số nếu có])

Đối với các lớp lồng nhau, để truy cập đến những thuộc tính và phương thức của class lồng thì khi khai báo cần chỉ ra lớp chứa đựng nó. Muốn sử dụng biến của lớp chứa thì các biến của lớp dùng để chứa phải khai báo là static và phải khai báo đối tượng chứa nó.

Ví dụ 2.2: Bạn đã khai báo một lớp diem với đầy đủ các thuộc tính và các phương thức (giả sử là có phương thức hien()), bây giờ bạn muốn tạo một đối tượng tên là A của lớp này và sử dụng phương thức hiện điểm A thì phải khai báo A là một biến đối tượng như sau:

diem A = new diem ();

A.hien();

Ví dụ 2.3: Định nghĩa lớp pheptoa và trong lớp này có chứa lớp tinhhieu, muốn sử dụng đối tượng của lớp tinhhieu thì bắt buộc bạn phải khai báo chỉ rõ lớp chứa ở đây là lớp pheptoa như sau:

pheptoa.tinhhieu con = new pheptoa.tinhhieu();

Sau đó việc truy xuất đến các phương thức của lớp thì tiến hành bình thường nhưng lớp ở trong thì không sử dụng được phương thức của lớp chứa nó và chỉ sử dụng được thành phần dữ liệu tĩnh của lớp chứa mà thôi.

* Bộ khởi dựng(constructor/ Phương thức khởi tạo/ thiết lập)

Cú pháp:

```
public className([ds tham số]){
// Khởi tạo cho các thành phần dữ liệu của lớp
}
```

trong đó className: Tên lớp

Chú ý: Phương thức khởi tạo là phương thức có tên trùng với tên của lớp và không có kiểu trả về

□ Ví dụ 2.4: Minh họa sử dụng khởi tạo biến thành viên.

```
-----
public class ThoiGian
{
public void ThoiGianHienHanh()
{
System.DateTime now = System.DateTime.Now;
System.Console.WriteLine("\n Hien tai: \t {0}/{1}/{2} {3}:{4}:{5}",
now.Day, now.Month, now.Year, now.Hour, now.Minute, now.Second);
System.Console.WriteLine(" Thoi Gian:\t {0}/{1}/{2} {3}:{4}:{5}",
Ngay, Thang, Nam, Gio, Phut, Giay);
}
public ThoiGian( System.DateTime dt)
{
Nam = dt.Year;
Thang = dt.Month;
Ngay = dt.Day;
Gio = dt.Hour;
Phut = dt.Minute;
Giay = dt.Second;
// có gán cho biến thành viên Giay
}
public ThoiGian(int Year, int Month, int Date, int Hour, int Minute)
```

```

{
    Nam = Year;
    Thang = Month;
    Ngay = Date;
    Gio = Hour;
    Phut = Minute;
}
private int Nam;
private int Thang;
private int Ngay;
private int Gio;
private int Phut;
private int Giay = 30 ; // biến được khởi tạo.
}
public class Tester
{
    static void Main()
    {
        System.DateTime currentTime = System.DateTime.Now;
        ThoiGian t1 = new ThoiGian( currentTime );
        t1.ThoiGianHienHanh();
        ThoiGian t2 = new ThoiGian(2001,7,3,10,5);
        t2.ThoiGianHienHanh();
    }
}

```

□ Kết quả:

Hien tai:

Thoi Gian:

Hien tai:

Thoi Gian:

5/6/2002

5/6/2002

5/6/2002

3/7/2001

10:15:5

10:15:5

10:15:5

10:5:30

Nếu không khởi tạo giá trị của biến thành viên thì bộ khởi dựng mặc định sẽ khởi tạo giá trị là 0 mặc định cho biến thành viên có kiểu nguyên. Tuy nhiên, trong trường hợp này biến thành viên `Giay` được khởi tạo giá trị 30:

```
Giay = 30;
```

```
// Khởi tạo
```

Trong trường hợp bộ khởi tạo thứ hai không truyền giá trị cho biến `Giay` nên biến này vẫn lấy giá trị mà ta đã khởi tạo ban đầu là 30:

```
ThoiGian t2 = new ThoiGian(2001, 7, 3, 10, 5);
```

```
t2.ThoiGianHienHanh();
```

Ngược lại, nếu một giá trị được gán cho biến `Giay` như trong bộ khởi tạo thứ nhất thì giá trị mới này sẽ được chồng lên giá trị khởi tạo.

Trong ví dụ trên lần đầu tiên tạo đối tượng `ThoiGian` do ta truyền vào đối tượng `DateTime` nên hàm khởi dựng thứ nhất được thực hiện, hàm này sẽ gán giá trị 5 cho biến `Giay`. Còn khi tạo đối tượng `ThoiGian` thứ hai, hàm khởi dựng thứ hai được thực hiện, hàm này không gán giá trị cho biến `Giay` nên biến này vẫn còn lưu giữ lại giá trị 30 khi khởi tạo ban đầu.

Bộ khởi dựng sao chép

Bộ khởi dựng sao chép thực hiện việc tạo một đối tượng mới bằng cách sao chép tất cả các biến từ một đối tượng đã có và cùng một kiểu dữ liệu. Ví dụ chúng ta muốn đưa một đối tượng `ThoiGian` vào bộ khởi dựng lớp `ThoiGian` để tạo một đối tượng `ThoiGian` mới có cùng giá trị với đối tượng `ThoiGian` cũ. Hai đối tượng này hoàn toàn khác nhau và chỉ giống nhau ở giá trị biến thành viên sao khi khởi dựng.

Ngôn ngữ `C#` không cung cấp bộ khởi dựng sao chép, do đó chúng ta phải tự tạo ra. Việc sao chép các thành phần từ một đối tượng ban đầu cho một đối tượng mới như sau:

```
public ThoiGian( ThoiGian tg)
```

```
{
```

```
Nam = tg.Nam;
```

```
Thang = tg.Thang;
```

```

Ngay = tg.Ngay;
Gio = tg.Gio;
Phut = tg.Phut;
Giay = tg.Giay;
}

```

Khi đó ta có thể sao chép từ một đối tượng ThoiGian đã hiện hữu như sau:

```
ThoiGian t2 = new ThoiGian( t1 );
```

Trong đó t1 là đối tượng ThoiGian đã tồn tại, sau khi lệnh trên thực hiện xong thì đối tượng t2 được tạo ra như bản sao của đối tượng t1.

Từ khóa this

Từ khóa this được dùng để tham chiếu đến thể hiện hiện hành của một đối tượng. Tham chiếu this này được xem là con trỏ ẩn đến tất các phương thức không có thuộc tính tĩnh trong một lớp. Mỗi phương thức có thể tham chiếu đến những phương thức khác và các biến thành viên thông qua tham chiếu this này.

Tham chiếu this này được sử dụng thường xuyên theo ba cách:

□ Sử dụng khi các biến thành viên bị che lấp bởi tham số đưa vào, như trường hợp sau:

```

public void SetYear( int Nam)
{
    this.Nam = Nam;
}

```

Như trong đoạn mã trên phương thức SetYear sẽ thiết lập giá trị của biến thành viên Nam, tuy nhiên do tham số đưa vào có tên là Nam, trùng với biến thành viên, nên ta phải dùng tham chiếu this để xác định rõ các biến thành viên và tham số được truyền vào. Khi đó this.Nam chỉ đến biến thành viên của đối tượng, trong khi Nam chỉ đến tham số.

□ Sử dụng tham chiếu this để truyền đối tượng hiện hành vào một tham số của một phương thức của đối tượng khác:

```

public void Method1( OtherClass otherObject )
{
    // Sử dụng tham chiếu this để truyền tham số là bản
    // thân đối tượng đang thực hiện.
    otherObject.SetObject( this );
}

```

Như trên cho thấy khi cần truyền một tham số là chính bản thân của đối tượng đang thực hiện thì ta bắt buộc phải dùng tham chiếu this để truyền.

- Cách thứ ba sử dụng tham chiếu this là mảng chỉ mục (indexer)

2.2. Sử dụng các thành viên static

Những thuộc tính và phương thức trong một lớp có thể là những thành viên thể hiện (instance members) hay những thành viên tĩnh (static members). Những thành viên thể hiện hay thành viên của đối tượng liên quan đến thể hiện của một kiểu dữ liệu. Trong khi thành viên tĩnh được xem như một phần của lớp. Chúng ta có thể truy cập đến thành viên tĩnh của một lớp thông qua tên lớp đã được khai báo. Ví dụ chúng ta có một lớp tên là Button và có hai thể hiện của lớp tên là btnUpdate và btnDelete. Và giả sử lớp Button này có một phương thức tĩnh là Show(). Để truy cập phương thức tĩnh này ta viết :

```
Button.Show();
```

Đúng hơn là viết: btnUpdate.Show();

□ Ghi chú: Trong ngôn ngữ C# không cho phép truy cập đến các phương thức tĩnh và các biến thành viên tĩnh thông qua một thể hiện, nếu chúng ta cố làm điều đó thì trình biên dịch C# sẽ báo lỗi, điều này khác với ngôn ngữ C++.

Trong một số ngôn ngữ thì có sự phân chia giữa phương thức của lớp và các phương thức khác (toàn cục) tồn tại bên ngoài không phụ thuộc bất cứ một lớp nào. Tuy nhiên, điều này không cho phép trong C#, ngôn ngữ C# không cho phép tạo các phương thức bên ngoài của lớp, nhưng ta có thể tạo được các phương thức giống như vậy bằng cách tạo các phương thức tĩnh bên trong một lớp.

Phương thức tĩnh hoạt động ít nhiều giống như phương thức toàn cục, ta truy cập phương thức này mà không cần phải tạo bất cứ thể hiện hay đối tượng của lớp chứa phương thức toàn cục. Tuy nhiên, lợi ích của phương thức tĩnh vượt xa phương thức toàn cục vì phương thức tĩnh được bao bọc trong phạm vi của một lớp nơi nó được định nghĩa, do vậy ta sẽ không gặp tình trạng lộn xộn giữa các phương thức trùng tên do chúng được đặt trong namespace.

Gọi một phương thức tĩnh

Như chúng ta đã biết phương thức Main() là một phương thức tĩnh. Phương thức tĩnh được xem như là phần hoạt động của lớp hơn là của thể hiện một lớp. Chúng cũng không cần có một tham chiếu **this** hay bất cứ thể hiện nào tham chiếu tới.

Phương thức tĩnh không thể truy cập trực tiếp đến các thành viên không có tính chất tĩnh (nonstatic). Như vậy Main() không thể gọi một phương thức không tĩnh bên trong lớp. Ta xem lại đoạn chương trình minh họa trong ví dụ 2.1:

```
using System;
```

```
public class Class1
```

```
{
```

```
public void SomeMethod(int p1, float p2)
```

```
{
```

```
Console.WriteLine("Ham nhan duoc hai tham so: {0} va {1}", p1,p2);
```

```

    }
}
public class Tester
{
    static void Main()
    {
        int var1 = 5;
        float var2 = 10.5f;
        Class1 c = new Class1();
        c.SomeMethod( var1, var2 );
    }
}

```

Phương thức `SomeMethod()` là phương thức không tĩnh của lớp `Class1`, do đó để truy cập được phương thức của lớp này cần phải tạo một thể hiện là một đối tượng cho lớp `Class1`.

Sau khi tạo thì có thể thông qua đối tượng `c` ta có thể gọi được được phương thức `Some-Method()`.

Sử dụng bộ khởi dựng tĩnh

Nếu một lớp khai báo một bộ khởi tạo tĩnh (static constructor), thì được đảm bảo rằng phương thức khởi dựng tĩnh này sẽ được thực hiện trước bất cứ thể hiện nào của lớp được tạo ra.

□ Ghi chú: Chúng ta không thể điều khiển chính xác khi nào thì phương thức khởi dựng tĩnh này được thực hiện. Tuy nhiên ta biết chắc rằng nó sẽ được thực hiện sau khi chương trình chạy và trước bất kì biến đối tượng nào được tạo ra.

Theo ví dụ 2.4 ta có thể thêm một bộ khởi dựng tĩnh cho lớp `ThoiGian` như sau:

```

static ThoiGian()
{
    Ten = "Thoi gian";
}

```

Lưu ý rằng ở đây không có bất cứ thuộc tính truy cập nào như `public` trước bộ khởi dựng tĩnh. Thuộc tính truy cập không cho phép theo sau một phương thức khởi dựng tĩnh. Do phương thức tĩnh nên không thể truy cập bất cứ biến thành viên không thuộc loại tĩnh, vì vậy biến thành viên `Name` bên trên cũng phải được khai báo là tĩnh:

```
private static string Ten;
```

Cuối cùng ta thêm một dòng vào phương thức ThoiGianHienHanh() của lớp ThoiGian:

```
public void ThoiGianHienHanh()
{
    System.Console.WriteLine(" Ten: {0}", Ten);
    System.Console.WriteLine(" Thoi Gian:\t {0}/{1}/{2} {3}:{4}:{5}",
        Ngay, Thang, Nam, Gio, Phut, Giay);
}
```

Sau khi thay đổi ta biên dịch và chạy chương trình được kết quả sau:

```
Ten: Thoi Gian
```

```
Thoi Gian:
```

```
5/6/2002
```

```
18:35:20
```

Mặc dù chương trình thực hiện tốt, nhưng không cần thiết phải tạo ra bộ khởi dựng tĩnh để phục vụ cho mục đích này. Thay vào đó ta có thể dùng chức năng khởi tạo biến thành viên như sau:

```
private static string Ten = "Thoi Gian";
```

Tuy nhiên, bộ khởi tạo tĩnh có hữu dụng khi chúng ta cần cài đặt một số công việc mà không thể thực hiện được thông qua chức năng khởi dựng và công việc cài đặt này chỉ được thực hiện duy nhất một lần.

Sử dụng bộ khởi dựng private

Như đã nói ngôn ngữ C# không có phương thức toàn cục và hằng số toàn cục. Do vậy chúng ta có thể tạo ra những lớp tiện ích nhỏ chỉ để chứa các phương thức tĩnh. Cách thực hiện này luôn có hai mặt tốt và không tốt. Nếu chúng ta tạo một lớp tiện ích như vậy và không muốn bất cứ một thể hiện nào được tạo ra. Để ngăn ngừa việc tạo bất cứ thể hiện của lớp ta tạo ra bộ khởi dựng không có tham số và không làm gì cả, tức là bên trong thân của phương thức rỗng, và thêm vào đó phương thức này được đánh dấu là **private**. Do không có bộ khởi dựng **public**, nên không thể tạo ra bất cứ thể hiện nào của lớp.

Sử dụng các thuộc tính tĩnh

Một vấn đề đặt ra là làm sao kiểm soát được số thể hiện của một lớp được tạo ra khi thực hiện chương trình. Vì hoàn toàn ta không thể tạo được biến toàn cục để làm công việc đếm số thể hiện của một lớp.

Thông thường các biến thành viên tĩnh được dùng để đếm số thể hiện đã được tạo ra của một lớp. Cách sử dụng này được áp dụng trong minh họa sau:

□ Ví dụ 2.5: Sử dụng thuộc tính tĩnh để đếm số thể hiện.

```
using System;
public class Cat
{
    public Cat()
    {
        instance++;
    }
    public static void HowManyCats()
    {
        Console.WriteLine("{0} cats", instance);
    }
    private static int instance =0;
}
public class Tester
{
    static void Main()
    {
        Cat.HowManyCats();
        Cat mun = new Cat();
        Cat.HowManyCats();
        Cat muop = new Cat();
        Cat miu = new Cat();
        Cat.HowManyCats();
    }
}
```

□ Kết quả:

0 cats
1 cats
3 cats

Bên trong lớp Cat ta khai báo một biến thành viên tĩnh tên là instance biến này dùng để đếm số thể hiện của lớp Cat, biến này được khởi tạo giá trị 0. Lưu ý rằng biến thành viên tĩnh được xem là thành phần của lớp, không phải là thành viên của thể hiện, do vậy nó sẽ không được khởi tạo bởi trình biên dịch khi tạo các thể hiện. Khởi tạo tường minh là yêu cầu bắt buộc với các biến thành viên tĩnh. Khi một thể hiện được tạo ra thì bộ dựng của lớp Cat sẽ thực hiện tăng biến instance lên một đơn vị.

2.3. Hủy đối tượng

Bộ hủy của C#

Cú pháp phương thức hủy trong ngôn ngữ C# cũng giống như trong ngôn ngữ C++. Nhưng về hành động cụ thể chúng có nhiều điểm khác nhau. Ta khảo báo một phương thức hủy trong C# như sau:

```
~Class1() {}
```

Tuy nhiên, trong ngôn ngữ C# thì cú pháp khai báo trên là một shortcut liên kết đến một phương thức kết thúc Finalize được kết với lớp cơ sở, do vậy khi viết

```
~Class1()
{
    // Thực hiện một số công việc
}
```

Cũng tương tự như viết :

```
Class1.Finalize()
{
    // Thực hiện một số công việc
    base.Finalize();
}
```

Do sự tương tự như trên nên khả năng dẫn đến sự lộn xộn nhầm lẫn là không tránh khỏi, nên chúng ta phải tránh viết các phương thức hủy và viết các phương thức Finalize tường minh nếu có thể được.

Phương thức Dispose

Như chúng ta đã biết thì việc gọi một phương thức kết thúc Finalize trong C# là không hợp lệ, vì phương thức này dành cho bộ thu dọn thực hiện. Nếu chúng ta xử lý các tài nguyên không kiểm soát như xử lý các handle của tập tin và ta muốn được đóng hay giải phóng nhanh chóng bất cứ lúc nào, ta có thể thi giao diện IDisposable, phần chi tiết IDisposable sẽ được trình bày chi tiết trong Bài 8. Giao diện IDisposable yêu cầu những thành phần thực thi của nó định nghĩa một phương thức tên là Dispose() để thực hiện công việc dọn dẹp mà ta yêu cầu. Ý nghĩa của phương thức Dispose là cho phép chương trình thực hiện các công việc dọn dẹp hay giải phóng tài nguyên mong muốn mà không phải chờ cho đến khi

phương thức `Finalize()` được gọi.

Khi chúng ta cung cấp một phương thức `Dispose` thì phải ngưng bộ thu dọn gọi phương thức `Finalize()` trong đối tượng của chúng ta. Để ngưng bộ thu dọn, chúng ta gọi một phương thức tĩnh của lớp GC (garbage collector) là `GC.SuppressFinalize()` và truyền tham số là tham chiếu `this` của đối tượng. Và sau đó phương thức `Finalize()` sử dụng để gọi phương thức `Dispose()` như đoạn mã sau:

```
public void Dispose()
{
    // Thực hiện công việc dọn dẹp
    // Yêu cầu bộ thu dọn GC trong thực hiện kết thúc
    GC.SuppressFinalize( this );
}

public override void Finalize()
{
    Dispose();
    base.Finalize();
}
```

Phương thức Close

Khi xây dựng các đối tượng, chúng ta có muốn cung cấp cho người sử dụng phương thức `Close()`, vì phương thức `Close` có vẻ tự nhiên hơn phương thức `Dispose` trong các đối tượng có liên quan đến xử lý tập tin. Ta có thể xây dựng phương thức `Dispose()` với thuộc tính là `private` và phương thức `Close()` với thuộc tính `public`. Trong phương thức `Close()` đơn giản là gọi thực hiện phương thức `Dispose()`.

2.4. Truyền tham số và nạp chồng phương thức

Truyền tham số

Như đã thảo luận trong chương trước, tham số có kiểu dữ liệu là giá trị thì sẽ được truyền giá trị vào cho phương thức. Điều này có nghĩa rằng khi một đối tượng có kiểu là giá trị được truyền vào cho một phương thức, thì có một bản sao chép đối tượng đó được tạo ra bên trong phương thức. Một khi phương thức được thực hiện xong thì đối tượng sao chép này sẽ được hủy. Tuy nhiên, đây chỉ là trường hợp bình thường, ngôn ngữ C# còn cung cấp khả năng cho phép ta truyền các đối tượng có kiểu giá trị dưới hình thức là tham chiếu. Ngôn ngữ C# đưa ra một bổ sung tham số là `ref` cho phép truyền các đối tượng giá trị vào trong phương thức theo kiểu tham chiếu. Và tham số bổ sung `out` trong trường hợp muốn truyền dưới dạng tham chiếu mà không cần phải khởi tạo giá trị ban đầu cho tham số truyền. Ngoài ra ngôn ngữ C# còn hỗ trợ bổ sung `params` cho phép phương thức chấp nhận nhiều số lượng các tham số.

Truyền tham chiếu

Những phương thức chỉ có thể trả về duy nhất một giá trị, mặc dù giá trị này có thể là một tập hợp các giá trị. Nếu chúng ta muốn phương thức trả về nhiều hơn một giá trị thì cách thực hiện là tạo các tham số dưới hình thức tham chiếu. Khi đó trong phương thức ta sẽ xử lý và gán các giá trị mới cho các tham số tham chiếu này, kết quả là sau khi phương thức thực hiện xong ta dùng các tham số truyền vào như là các kết quả trả về. Ví dụ 2.6 sau minh họa việc truyền tham số tham chiếu cho phương thức.

□ Ví dụ 2.6: Trả giá trị trả về thông qua tham số.

```
using System;

public class Time
{
    public void DisplayCurrentTime()
    {
        Console.WriteLine("{0}/{1}/{2}/ {3}:{4}:{5}", Date,
            Month, Year, Hour, Minute, Second);
    }

    public int GetHour()
    {
        return Hour;
    }

    public void GetTime(int h, int m, int s)
    {
        h = Hour;
        m = Minute;
        s = Second;
    }

    public Time( System.DateTime dt)
    {
        Year = dt.Year;
        Month = dt.Month;
        Date = dt.Day;
        Hour = dt.Hour;
        Minute = dt.Minute;
```

```

Second = dt.Second;
}
private int Year;
private int Month;
private int Date;
private int Hour;
private int Minute;
private int Second;
}
public class Tester
{
static void Main()
{
System.DateTime currentTime = System.DateTime.Now;
Time t = new Time( currentTime);
t.DisplayCurrentTime();
int theHour = 0;
int theMinute = 0;
int theSecond = 0;
t.GetTime( theHour, theMinute, theSecond);
System.Console.WriteLine("Current time: {0}:{1}:{2}",
theHour, theMinute, theSecond);
}
}

```

□ Kết quả:

8/6/2002

14:15:20

Current time: 0:0:0

Như ta thấy, kết quả xuất ra ở dòng cuối cùng là ba giá trị 0:0:0, rõ ràng phương thức GetTime() không thực hiện như mong muốn là gán giá trị Hour,

Minute, Second cho các tham số truyền vào. Tức là ba tham số này được truyền vào dưới dạng giá trị. Do đó để thực hiện như mục đích của chúng ta là lấy các giá trị của Hour, Minute, Second thì phương thức GetTime() có ba tham số được truyền dưới dạng tham chiếu. Ta thực hiện như sau, đầu tiên, thêm là thêm khai báo ref vào trước các tham số trong phương thức GetTime():

```
public void GetTime( ref int h, ref int m, ref int s)
{
    h = Hour;
    m = Minute;
    s = Second;
}
```

Điều thay đổi thứ hai là bổ sung cách gọi hàm GetTime để truyền các tham số dưới dạng tham chiếu như sau:

```
t.GetTime( ref theHour, ref theMinute, ref theSecond);
```

Nếu chúng ta không thực hiện bước thứ hai, tức là không đưa từ khóa ref khi gọi hàm thì trình biên dịch C# sẽ báo một lỗi rằng không thể chuyển tham số từ kiểu int sang kiểu ref int. Cuối cùng khi biên dịch lại chương trình ta được kết quả đúng như yêu cầu. Bằng việc khai báo tham số tham chiếu, trình biên dịch sẽ truyền các tham số dưới dạng các tham chiếu, thay cho việc tạo ra một bản sao chép các tham số này. Khi đó các tham số bên trong GetTime() sẽ tham chiếu đến cùng biến đã được khai báo trong hàm Main(). Như vậy mọi sự thay đổi với các biến này điều có hiệu lực tương tự như là thay đổi trong hàm Main().

Tóm lại cơ chế truyền tham số dạng tham chiếu sẽ thực hiện trên chính đối tượng được đưa vào. Còn cơ chế truyền tham số giá trị thì sẽ tạo ra các bản sao các đối tượng được truyền vào, do đó mọi thay đổi bên trong phương thức không làm ảnh hưởng đến các đối tượng được truyền vào dưới dạng giá trị.

Truyền tham chiếu với biến chưa khởi tạo

Ngôn ngữ C# bắt buộc phải thực hiện một phép gán cho biến trước khi sử dụng, do đó khi khai báo một biến như kiểu cơ bản thì trước khi có lệnh nào sử dụng các biến này thì phải có lệnh thực hiện việc gán giá trị xác định cho biến. Như trong ví dụ 2.6 trên, nếu chúng ta không khởi tạo biến theHour, theMinute, và biến theSecond trước khi truyền như tham số vào phương thức GetTime() thì trình biên dịch sẽ báo lỗi. Nếu chúng ta sửa lại đoạn mã của ví dụ 2.6 như sau:

```
int theHour;
int theMinute;
int theSecond;

t.GetTime( ref int theHour, ref int theMinute, ref int theSecond);
```

Việc sử dụng các đoạn lệnh trên không phải hoàn toàn vô lý vì mục đích của chúng ta là nhận các giá trị của đối tượng Time, việc khởi tạo giá trị của các

biến đưa vào là không cần thiết.

Tuy nhiên khi biên dịch với đoạn mã lệnh như trên sẽ được báo các lỗi sau:

Use of unassigned local variable 'theHour'

Use of unassigned local variable 'theMinute'

Use of unassigned local variable 'theSecond'

Để mở rộng cho yêu cầu trong trường hợp này ngôn ngữ C# cung cấp thêm một bổ sung tham chiếu là out. Khi sử dụng tham chiếu out thì yêu cầu bắt buộc phải khởi tạo các tham số tham chiếu được bỏ qua. Như các tham số trong phương thức GetTime(), các tham số này không cung cấp bất cứ thông tin nào cho phương thức mà chỉ đơn giản là cơ chế nhận thông tin và đưa ra bên ngoài. Do vậy ta có thể đánh dấu tất cả các tham số tham chiếu này là out, khi đó ta sẽ giảm được công việc phải khởi tạo các biến này trước khi đưa vào phương thức.

Lưu ý là bên trong phương thức có các tham số tham chiếu out thì các tham số này phải được gán giá trị trước khi trả về. Ta có một số thay đổi cho phương thức GetTime() như sau:

```
public void GetTime( out int h, out int m, out int s)
{
    h = Hour;
    m = Minute;
    s = Second;
}
```

và cách gọi mới phương thức GetTime() trong Main():

```
t.GetTime( out theHour, out theMinute, out theSecond);
```

Tóm lại ta có các cách khai báo các tham số trong một phương thức như sau: kiểu dữ liệu giá trị được truyền vào phương thức bằng giá trị. Sử dụng tham chiếu ref để truyền kiểu dữ liệu giá trị vào phương thức dưới dạng tham chiếu, cách này cho phép vừa sử dụng và có khả năng thay đổi các tham số bên trong phương thức được gọi. Tham chiếu out được sử dụng chỉ để trả về giá trị từ một phương thức. Ví dụ 2.7 sau sử dụng ba kiểu tham số trên.

□ Ví dụ 2.7: Sử dụng tham số.

```
-----
using System;

public class Time
{
    public void DisplayCurrentTime()
    {
```

```

Console.WriteLine("{0}/{1}/{2} {3}:{4}:{5}",
Date, Month, Year, Hour, Minute, Second);
}
public int GetHour()
{
return Hour;
}
public void SetTime(int hr, out int min, ref int sec)
{
// Nếu số giây truyền vào >30 thì tăng số Minute và Second = 0
if ( sec >=30 )
{
Minute++;
Second = 0;
}
Hour = hr; // thiết lập giá trị hr được truyền vào
// Trả về giá trị mới cho min và sec
min = Minute;
sec = Second;
}
public Time( System.DateTime dt)
{
Year = dt.Year;
Month = dt.Month;
Date = dt.Day;
Hour = dt.Hour;
Minute = dt.Minute;
Second = dt.Second;
}
// biến thành viên private
private int Year;
private int Month;
private int Date;

```

```

private int Hour;
private int Minute;
private int Second;
}
public class Tester
{
    static void Main()
    {
        System.DateTime currentTime = System.DateTime.Now;
        Time t = new Time(currentTime);
        t.DisplayCurrentTime();
        int theHour = 3;
        int theMinute;
        int theSecond = 20;
        t.SetTime( theHour, out theMinute, ref theSecond);
        Console.WriteLine("The Minute is now: {0} and {1} seconds ",
            theMinute, theSecond);
        theSecond = 45;
        t.SetTime( theHour, out theMinute, ref theSecond);
        Console.WriteLine("The Minute is now: {0} and {1} seconds",
            theMinute, theSecond);
    }
}

```

□ Kết quả:

8/6/2002

15:35:24

The Minute is now: 35 and 24 seconds

The Minute is now: 36 and 0 seconds

Phương thức SetTime trên đã minh họa việc sử dụng ba kiểu truyền tham số vào một phương thức. Tham số thứ nhất theHour được truyền vào dạng giá trị, mục đích của tham số này là để thiết lập giá trị cho biến thành viên Hour và tham số này không được sử dụng để về bất cứ giá trị nào.

Tham số thứ hai là `theMinute` được truyền vào phương thức chỉ để nhận giá trị trả về của biến thành viên `Minute`, do đó tham số này được khai báo với từ khóa `out`.

Cuối cùng tham số `theSecond` được truyền vào với khai báo `ref`, biến tham số này vừa dùng để thiết lập giá trị trong phương thức. Nếu `theSecond` lớn hơn 30 thì giá trị của biến thành viên `Minute` tăng thêm một đơn vị và biến thành viên `Second` được thiết lập về 0. Sau cùng thì `theSecond` được gán giá trị của biến thành viên `Second` và được trả về.

Do hai biến `theHour` và `theSecond` được sử dụng trong phương thức `SetTime` nên phải được khởi tạo trước khi truyền vào phương thức. Còn với biến `theMinute` thì không cần thiết vì nó không được sử dụng trong phương thức mà chỉ nhận giá trị trả về.

Nạp chồng phương thức

Thông thường khi xây dựng các lớp, ta có mong muốn là tạo ra nhiều hàm có cùng tên. Cũng như hầu hết trong các ví dụ trước thì các lớp điều có nhiều hơn một phương thức khởi dựng.

Như trong lớp `Time` có các phương thức khởi dựng nhận các tham số khác nhau, như tham số là đối tượng `DateTime`, hay tham số có thể được tùy chọn để thiết lập các giá trị của các biến thành viên thông qua các tham số nguyên. Tóm lại ta có thể xây dựng nhiều các phương thức cùng tên nhưng nhận các tham số khác nhau. Chức năng này được gọi là nạp chồng phương thức.

Một ký hiệu (signature) của một phương thức được định nghĩa như tên của phương thức cùng với danh sách tham số của phương thức. Hai phương thức khác nhau khi ký hiệu của chúng khác là khác nhau tức là khác nhau khi tên phương thức khác nhau hay danh sách tham số khác nhau. Danh sách tham số được xem là khác nhau bởi số lượng các tham số hoặc là kiểu dữ liệu của tham số. Ví dụ đoạn mã sau, phương thức thứ nhất khác phương thức thứ hai do số lượng tham số khác nhau. Phương thức thứ hai khác phương thức thứ ba do kiểu dữ liệu tham số khác nhau:

```
void myMethod( int p1 );
void myMethod( int p1, int p2 );
void myMethod( int p1, string p2 );
```

Một lớp có thể có bất cứ số lượng phương thức nào, nhưng mỗi phương thức trong lớp phải có ký hiệu khác với tất cả các phương thức thành viên còn lại của lớp.

Ví dụ 2.8 minh họa lớp `Time` có hai phương thức khởi dựng, một phương thức nhận tham số là một đối tượng `DateTime` còn phương thức thứ hai thì nhận sáu tham số nguyên.

□ Ví dụ 2.8: Minh họa nạp chồng phương thức khởi dựng.

```

using System;
public class Time
{
    public void DisplayCurrentTime()
    {
        Console.WriteLine("{0}/{1}/{2} {3}:{4}:{5}",
            Date, Month, Year, Hour, Minute, Second);
    }
    public Time( System.DateTime dt)
    {
        Year = dt.Year;
        Month = dt.Month;
        Date = dt.Day;
        Hour = dt.Hour;
        Minute = dt.Minute;
        Second = dt.Second;
    }
    public Time(int Year, int Month, int Date, int Hour, int Minute, int Second)
    {
        this.Year = Year;
        this.Month = Month;
        this.Date = Date;
        this.Hour = Hour;
        this.Minute = Minute;
        this.Second = Second;
    }
    // Biến thành viên private
    private int Year;
    private int Month;
    private int Date;
    private int Hour;
    private int Minute;
    private int Second;

```

```

}
public class Tester
{
    static void Main()
    {
        System.DateTime currentTime = System.DateTime.Now;
        Time t1 = new Time( currentTime);
        t1.DisplayCurrentTime();
        Time t2 = new Time(2002,6,8,18,15,20);
        t2.DisplayCurrentTime();
    }
}

```

□ Kết quả:

2/1/2002

8/6/2002

17:50:17

18:15:20

Như chúng ta thấy, lớp Time trong ví dụ minh họa 4.9 có hai phương thức khởi dựng. Nếu hai phương thức có cùng ký hiệu thì trình biên dịch sẽ không thể biết được gọi phương thức nào khi khởi tạo hai đối tượng là t1 và t2. Tuy nhiên, ký hiệu của hai phương thức này khác nhau vì ham số truyền vào khác nhau, do đó trình biên dịch sẽ xác định được phương thức nào được gọi dựa vào các tham số.

2.5. Đóng gói dữ liệu với thuộc tính

Thuộc tính là khái niệm cho phép truy cập trạng thái của lớp thay vì thông qua truy cập trực tiếp các biến thành viên, nó sẽ được thay thế bằng việc thực thi truy cập thông qua phương thức của lớp.

Thuộc tính là một đặc tính mới được giới thiệu trong ngôn ngữ C#. Đặc tính này cung cấp khả năng bảo vệ các trường dữ liệu bên trong một lớp bằng việc đọc và viết chúng thông qua thuộc tính. Trong ngôn ngữ khác, điều này có thể được thực hiện thông qua việc tạo các phương thức lấy dữ liệu (getter method) và phương thức thiết lập dữ liệu (setter method).

Thuộc tính được thiết kế nhằm vào hai mục đích: cung cấp một giao diện đơn cho phép truy cập các biến thành viên, Tuy nhiên cách thức thực thi truy cập

giống như phương thức trong đó các dữ liệu được che dấu, đảm bảo cho yêu cầu thiết kế hướng đối tượng. Để hiểu rõ đặc tính này ta sẽ xem ví dụ 2.9 bên dưới:

□ Ví dụ 2.9: Sử dụng thuộc tính.

```

using System;
public class Time
{
    public void DisplayCurrentTime()
    {
        Console.WriteLine("Time\t: {0}/{1}/{2} {3}:{4}:{5}",
            date, month, year, hour, minute, second);
    }
    public Time( System.DateTime dt)
    {
        year = dt.Year;
        month = dt.Month;
        date = dt.Day;
        hour = dt.Hour;
        minute = dt.Minute;
        second = dt.Second;
    }
    public int Hour
    {
        get
        {
            return hour;
        }
        set
        {
            hour = value;
        }
    }
}
// Biến thành viên private

```

```

private int year;
private int month;
private int date;
private int hour;
private int minute;
private int second;
}
public class Tester
{
static void Main()
{
System.DateTime currentTime = System.DateTime.Now;
Time t = new Time( currentTime );
t.DisplayCurrentTime();
// Lấy dữ liệu từ thuộc tính Hour
int theHour = t.Hour;
Console.WriteLine(" Retrieved the hour: {0}", theHour);
theHour++;
t.Hour = theHour;
Console.WriteLine("Updated the hour: {0}", theHour);
}
}

```

□ Kết quả:

Time: 2/1/2003 17:55:1

Retrieved the hour: 17

Updated the hour: 18

Để khai báo thuộc tính, đầu tiên là khai báo tên thuộc tính để truy cập, tiếp theo là phần thân định nghĩa thuộc tính nằm trong cặp dấu ({}). Bên trong thân của thuộc tính là khai báo hai bộ truy cập lấy và thiết lập dữ liệu:

```

public int Hour
{

```

```

get
{
    return hour;
}

set
{
    hour = value;
}
}

```

Mỗi bộ truy cập được khai báo riêng biệt để làm hai công việc khác nhau là lấy hay thiết lập giá trị cho thuộc tính. Giá trị thuộc tính có thể được lưu trong cơ sở dữ liệu, khi đó trong phần thân của bộ truy cập sẽ thực hiện các công việc tương tác với cơ sở dữ liệu. Hoặc là giá trị thuộc tính được lưu trữ trong các biến thành viên của lớp như trong ví dụ:

```
private int hour;
```

Truy cập lấy dữ liệu (get accessor)

Phần khai báo tương tự như một phương thức của lớp dùng để trả về một đối tượng có kiểu dữ liệu của thuộc tính. Trong ví dụ trên, bộ truy cập lấy dữ liệu get của thuộc tính Hour cũng tương tự như một phương thức trả về một giá trị int. Nó trả về giá trị của biến thành viên hour nơi mà giá trị của thuộc tính Hour lưu trữ:

```

get
{
    return hour;
}

```

Trong ví dụ này, một biến thành viên cục bộ được trả về, nhưng nó cũng có thể truy cập dễ dàng một giá trị nguyên từ cơ sở dữ liệu, hay thực hiện việc tính toán tùy ý. Bất cứ khi nào chúng ta tham chiếu đến một thuộc tính hay là gán giá trị thuộc tính cho một biến thì bộ truy cập lấy dữ liệu get sẽ được thực hiện để đọc giá trị của thuộc tính:

```
Time t = new Time( currentTime );
```

```
int theHour = t.Hour;
```

Khi lệnh thứ hai được thực hiện thì giá trị của thuộc tính sẽ được trả về, tức là bộ truy cập lấy dữ liệu get sẽ được thực hiện và kết quả là giá trị của thuộc tính được gán cho biến cục bộ theHour.

Bộ truy cập thiết lập dữ liệu (set accessor)

Bộ truy cập này sẽ thiết lập một giá trị mới cho thuộc tính và tương tự như

một phương thức trả về một giá trị void. Khi định nghĩa bộ truy cập thiết lập dữ liệu chúng ta phải sử dụng từ khóa value để đại diện cho tham số được truyền vào và được lưu trữ bởi thuộc tính:

```
set
{
    hour = value;
}
```

Như đã nói trước, do ta đang khai báo thuộc tính lưu trữ dưới dạng biến thành viên nên trong phần thân của bộ truy cập ta chỉ sử dụng biến thành viên mà thôi. Bộ truy cập thiết lập hoàn toàn cho phép chúng ta có thể viết giá trị vào trong cơ sở dữ liệu hay cập nhật bất cứ biến thành viên nào khác của lớp nếu cần thiết.

Khi chúng ta gán một giá trị cho thuộc tính thì bộ truy cập thiết lập dữ liệu set sẽ được tự động thực hiện và một tham số ngầm định sẽ được tạo ra để lưu giá trị mà ta muốn gán:

```
theHour++;
t.Hour = theHour;
```

Lợi ích của hướng tiếp cận này cho phép các thành phần bên ngoài (client) có thể tương tác với thuộc tính một cách trực tiếp, mà không phải hy sinh việc che dấu dữ liệu cũng như đặc tính đóng gói dữ liệu trong thiết kế hướng đối tượng.

3. Kế thừa – Đa hình

3.1. Sự kế thừa

Trong ngôn ngữ C#, quan hệ đặc biệt hóa được thực thi bằng cách sử dụng *sự kế thừa*. Đây không phải là cách duy nhất để thực thi đặc biệt hóa, nhưng nó là cách chung nhất và tự nhiên nhất để thực thi quan hệ này.

Trong mô hình trước, ta có thể nói ListBox kế thừa hay được dẫn xuất từ Window. Window được xem như là lớp cơ sở, và ListBox được xem như là lớp dẫn xuất. Như vậy, ListBox dẫn xuất tất cả các thuộc tính và hành vi từ lớp Window và thêm những phần đặc biệt riêng để xác nhận ListBox.

3.2. Đa hình

Có hai cách thức khá mạnh để thực hiện việc kế thừa. Một là sử dụng lại mã nguồn, khi chúng ta tạo ra lớp ListBox, chúng ta có thể sử dụng lại một vài các thành phần trong lớp cơ sở như Window.

Tuy nhiên, cách sử dụng thứ hai chứng tỏ được sức mạnh to lớn của việc kế thừa đó là *tính đa hình* (polymorphism). Theo tiếng Anh từ này được kết hợp từ poly là nhiều và morph có nghĩa là form (hình thức). Do vậy, đa hình được hiểu như là khả năng sử dụng nhiều hình thức của một kiểu mà không cần phải quan tâm đến từng chi tiết.

Khi một tổng đài điện thoại gọi cho máy điện thoại của chúng ta một tín hiệu có cuộc gọi. Tổng đài không quan tâm đến điện thoại của ta là loại nào. Có

thể ta đang dùng một điện thoại cũ dùng motor để rung chuông, hay là một điện thoại điện tử phát ra tiếng nhạc số.

Hoàn toàn các thông tin về điện thoại của ta không có ý nghĩa gì với tổng đài, tổng đài chỉ biết một kiểu cơ bản là điện thoại mà thôi và điện thoại này sẽ biết cách báo chuông. Còn việc báo chuông như thế nào thì tổng đài không quan tâm. Tóm lại, tổng đài chỉ cần báo điện thoại hãy làm điều gì đó để rung. Còn phần còn lại tức là cách thức rung là tùy thuộc vào từng loại điện thoại. Đây chính là tính đa hình.

Kiểu đa hình

Do một ListBox là một Window và một Button cũng là một Window, chúng ta mong muốn sử dụng cả hai kiểu dữ liệu này trong tình huống cả hai được gọi là Window. Ví dụ như trong một form giao diện trên MS Windows, form này chứa một tập các thể hiện của Window. Khi form được hiển thị, nó yêu cầu tất cả các thể hiện của Window tự thực hiện việc tô vẽ. Trong trường hợp này, form không muốn biết thành phần thể hiện là loại nào như Button, CheckBox,... Điều quan trọng là form kích hoạt toàn bộ tập hợp này tự thực hiện việc vẽ. Hay nói ngắn gọn là form muốn đối xử với những đối tượng Window này một cách đa hình.

Phương thức đa hình

Để tạo một phương thức hỗ tính đa hình, chúng ta cần phải khai báo khóa **virtual** trong phương thức của lớp cơ sở. Ví dụ, để chỉ định rằng phương thức DrawWindow() của lớp Window là đa hình, đơn giản là ta thêm từ khóa **virtual** vào khai báo như sau:

```
public virtual void DrawWindow()
```

Lúc này thì các lớp dẫn xuất được tự do thực thi các cách xử riêng của mình trong phiên bản mới của phương thức DrawWindow(). Để làm được điều này chỉ cần thêm từ khóa override để chồng lên phương thức ảo DrawWindow() của lớp cơ sở. Sau đó thêm các đoạn mã nguồn mới vào phương thức viết chồng này.

3.3. Lớp trừu tượng

Mỗi lớp con của lớp Window nên thực thi một phương thức DrawWindow() cho riêng mình. Tuy nhiên điều này không thực sự đòi hỏi phải thực hiện một cách bắt buộc. Để yêu cầu các lớp con (lớp dẫn xuất) phải thực thi một phương thức của lớp cơ sở, chúng ta phải thiết kế một phương thức một cách trừu tượng.

Một phương thức trừu tượng không có sự thực thi. Phương thức này chỉ đơn giản tạo ra một tên phương thức và ký hiệu của phương thức, phương thức này sẽ được thực thi ở các lớp dẫn xuất.

Những lớp trừu tượng được thiết lập như là cơ sở cho những lớp dẫn xuất, nhưng việc tạo các thể hiện hay các đối tượng cho các lớp trừu tượng được xem là không hợp lệ. Một khi chúng ta khai báo một phương thức là trừu tượng, thì chúng ta phải ngăn cấm bất cứ việc tạo thể hiện cho lớp này.

Do vậy, nếu chúng ta thiết kế phương thức DrawWindow() như là trừu

tượng trong lớp Window, chúng ta có thể dẫn xuất từ lớp này, nhưng ta không thể tạo bất cứ đối tượng cho lớp này. Khi đó mỗi lớp dẫn xuất phải thực thi phương thức DrawWindow(). Nếu lớp dẫn xuất không thực thi phương thức trừa tượng của lớp cơ sở thì lớp dẫn xuất đó cũng là lớp trừa tượng, và ta cũng không thể tạo các thể hiện của lớp này được.

Phương thức trừa tượng được thiết lập bằng cách thêm từ khóa **abstract** vào đầu của phần định nghĩa phương thức, cú pháp thực hiện như sau:

```
abstract public void DrawWindow( );
```

Do phương thức không cần phần thực thi, nên không có dấu ({}) mà chỉ có dấu chấm phẩy (;) sau phương thức. Như thế với phương thức DrawWindow() được thiết kế là trừa tượng thì chỉ cần câu lệnh trên là đủ.

Nếu một hay nhiều phương thức được khai báo là trừa tượng, thì phần định nghĩa lớp phải được khai báo là **abstract**, với lớp Window ta có thể khai báo là lớp trừa tượng như sau:

```
abstract public void Window
```

3.4. Các lớp lồng nhau

Các lớp chứa những thành viên, và những thành viên này có thể là một lớp khác có kiểu do người dùng định nghĩa (user-defined type). Do vậy, một lớp Button có thể có một thành viên của kiểu Location, và kiểu Location này chứa thành viên của kiểu dữ liệu Point. Cuối cùng, Point có thể chứa thành viên của kiểu int.

Cho đến lúc này, các lớp được tạo ra chỉ để dùng cho các lớp bên ngoài, và chức năng của các lớp đó như là lớp trợ giúp (helper class). Chúng ta có thể định nghĩa một lớp trợ giúp bên trong các lớp ngoài (outer class). Các lớp được định nghĩa bên trong gọi là các lớp lồng (nested class), và lớp chứa được gọi đơn giản là lớp ngoài.

Những lớp lồng bên trong có lợi là có khả năng truy cập đến tất cả các thành viên của lớp ngoài. Một phương thức của lớp lồng có thể truy cập đến biến thành viên private của lớp ngoài.

Hơn nữa, lớp lồng bên trong có thể ẩn đối với tất cả các lớp khác, lớp lồng có thể là private cho lớp ngoài.

Cuối cùng, một lớp làm lồng bên trong là public và được truy cập bên trong phạm vi của lớp ngoài. Nếu một lớp Outer là lớp ngoài, và lớp Nested là lớp public lồng bên trong lớp Outer, chúng ta có thể tham chiếu đến lớp Nested như Outer.Nested, khi đó lớp bên ngoài hành động ít nhiều giống như một namespace hay một phạm vi.

□ Ghi chú: Đối với người lập trình Java, lớp lồng nhau trong C# thì giống như những lớp nội static (static inner) trong Java. Không có sự tương ứng trong C# với những lớp nội nonstatic (nonstatic inner) trong Java.

4. Nạp chồng toán tử

4.1. Sử dụng từ khóa operator

Trong ngôn ngữ C#, các toán tử là các phương thức tĩnh, giá trị trả về của nó thể hiện kết quả của một toán tử và những tham số là các toán hạng. Khi chúng ta tạo một toán tử cho một lớp là chúng ta đã thực việc nạp chồng (overloaded) những toán tử đó, cũng giống như là chúng ta có thể nạp chồng bất cứ phương thức thành viên nào. Do đó, để nạp chồng toán tử cộng (+) chúng ta có thể viết như sau:

```
public static Fraction operator + ( Fraction lhs, Fraction rhs)
```

Trong toán tử trên ta có sự qui ước đặt tên của tham số là lhs và rhs. Tham số tên lhs thay thế cho “*left hand side*” tức là toán hạng bên trái, tương tự tham số tên rhs thay thế cho “*right hand side*” tức là toán hạng bên phải.

Cú pháp ngôn ngữ C# cho phép nạp chồng một toán tử bằng cách viết từ khóa **operator** và theo sau là toán tử được nạp chồng. Từ khóa **operator** là một bổ sung phương thức (method operator). Như vậy, để nạp chồng toán tử cộng (+) chúng ta có thể viết **operator +**.

Khi chúng ta viết:

```
Fraction theSum = firstFraction + secondFraction;
```

Thì toán tử nạp chồng + được thực hiện, với firstFraction được truyền vào như là tham số đầu tiên, và secondFraction được truyền vào như là tham số thứ hai. Khi trình biên dịch gặp biểu thức: firstFraction + secondFraction thì trình biên dịch sẽ chuyển biểu thức vào:

```
Fraction.operator+(firstFraction, secondFraction)
```

Kết quả sau khi thực hiện là một đối tượng Fraction mới được trả về, trong trường hợp này phép gán sẽ được thực hiện để gán một đối tượng Fraction cho theSum.

□ *Ghi chú:* Đối với người lập trình C++, trong ngôn ngữ C# không thể tạo được toán tử nonstatic, và do vậy nên toán tử nhị phân phải lấy hai toán hạng.

4.2. Toán tử so sánh bằng

Nếu chúng ta nạp chồng toán tử bằng (==), thì chúng ta cũng nên phủ quyết phương thức ảo Equals() được cung cấp bởi lớp object và chuyển lại cho toán tử bằng thực hiện. Điều này cho phép lớp của chúng ta thể tương thích với các ngôn ngữ .NET khác không hỗ trợ tính nạp chồng toán tử nhưng hỗ trợ nạp chồng phương thức. Những lớp FCL không sử dụng nạp chồng toán tử, nhưng vẫn mong đợi lớp của chúng ta thực hiện những phương thức cơ bản này. Do đó ví dụ lớp ArrayList mong muốn chúng ta thực thi phương thức Equals().

Lớp object thực thi phương thức Equals() với khai báo sau:

```
public override bool Equals( object o )
```

Bằng cách phủ quyết phương thức này, chúng ta cho phép lớp Fraction hành động một cáchmđa hình với tất cả những lớp khác. Bên trong thân của phương

thức Equals() chúng ta cần phải đảm bảo rằng chúng ta đang so sánh với một Fraction khác, và nếu như chúng ta đã thực thi một toán tử so sánh bằng thì có thể định nghĩa phương thức Equals() như sau:

```
public override bool Equals( object o)
{
    if ( !(o is Fraction) )
    {
        return false;
    }
    return this == (Fraction) o;
}
```

Toán tử **is** được sử dụng để kiểm tra kiểu của đối tượng lúc chạy chương trình có tương thích với toán hạng trong trường hợp này là Fraction. Do o là Fraction nên toán tử **is** sẽ trả về true.

4.3. Toán tử chuyển đổi

C# cho phép chuyển đổi từ kiểu int sang kiểu long một cách ngầm định, và cũng cho phép chúng ta chuyển từ kiểu long sang kiểu int một cách tường minh. Việc chuyển từ kiểu int sang kiểu long được thực hiện ngầm định bởi vì hiển nhiên bất kỳ giá trị nào của int cũng được thích hợp với kích thước của kiểu long. Tuy nhiên, điều ngược lại, tức là chuyển từ kiểu long sang kiểu int phải được thực hiện một cách tường minh (sử dụng ép kiểu) bởi vì ta có thể mất thông tin khi giá trị của biến kiểu long vượt quá kích thước của int lưu trong bộ nhớ:

```
int myInt = 5;
long myLong;
myLong = myInt; // ngầm định
myInt = (int) myLong; // tường minh
```

Chúng ta muốn thực hiện việc chuyển đổi này với lớp Fraction. Khi đưa ra một số nguyên, chúng ta có thể hỗ trợ ngầm định để chuyển đổi thành một phân số bởi vì bất kỳ giá trị nguyên nào ta cũng có thể chuyển thành giá trị phân số với mẫu số là 1 như $24 = 24/1$.

Khi đưa ra một phân số, chúng ta muốn cung cấp một sự chuyển đổi tường minh trở lại một số nguyên, điều này có thể hiểu là một số thông tin sẽ bị mất. Do đó, khi chúng ta chuyển phân số $9/4$ thành giá trị nguyên là 2.

Từ ngữ ngầm định (implicit) được sử dụng khi một chuyển đổi đảm bảo thành công mà không mất bất cứ thông tin nào của dữ liệu nguyên thủy. Trường hợp ngược lại, tường minh (explicit) không đảm bảo bảo toàn dữ liệu sau khi chuyển đổi do đó việc này sẽ được thực hiện một cách công khai.

5. Cấu trúc

Cú pháp để khai báo một cấu trúc cũng tương tự như cách khai báo một lớp:

```
[thuộc tính] [bổ sung truy cập] struct <tên cấu trúc> [: danh sách giao diện]
{
    [thành viên của cấu trúc]
}
```

Không giống như những lớp, cấu trúc không hỗ trợ việc thừa kế. Chúng được thừa kế ngầm định từ lớp object (tương tự như tất cả các kiểu dữ liệu trong C#, bao gồm các kiểu dữ liệu xây dựng sẵn) nhưng không thể kế thừa từ các lớp khác hay cấu trúc khác. Cấu trúc cũng được ngầm định là **sealed**, điều này có ý nghĩa là không có lớp nào hay bất cứ cấu trúc nào có thể dẫn xuất từ nó. Tuy nhiên, cũng giống như các lớp, cấu trúc có thể thực thi nhiều giao diện. Sau đây là một số sự khác nhau nữa là:

Không có bộ hủy và bộ khởi tạo mặc định tùy chọn: Những cấu trúc không có bộ hủy và cũng không có bộ khởi tạo mặc định không tham số tùy chọn. Nếu chúng ta không cung cấp bất cứ bộ khởi tạo nào thì cấu trúc sẽ được cung cấp một bộ khởi tạo mặc định, khi đó giá trị 0 sẽ được thiết lập cho tất cả các dữ liệu thành viên hay những giá trị mặc định tương ứng cho từng kiểu dữ liệu. Nếu chúng ta cung cấp bất cứ bộ khởi tạo nào thì chúng ta phải khởi tạo tất cả các trường trong cấu trúc.

Không cho phép khởi tạo: chúng ta không thể khởi tạo các trường thể hiện (instance fields) trong cấu trúc, do đó đoạn mã nguồn sau sẽ không hợp lệ:

```
private int xVal = 20;
private int yVal = 50;
```

Chúng ta tạo một thể hiện của cấu trúc bằng cách sử dụng từ khóa new trong câu lệnh gán, như khi chúng ta tạo một đối tượng của lớp. Lớp Tester tạo một thể hiện của Location như sau:

```
Location loc1 = new Location( 200, 300);
```

Ở đây một thể hiện mới tên là loc1 và nó được truyền hai giá trị là 200 và 300.

Cấu trúc là một kiểu giá trị

Phần định nghĩa của lớp Tester trên bao gồm một đối tượng Location là loc1 được tạo với giá trị là 200 và 300. Dòng lệnh sau sẽ gọi thực hiện bộ khởi tạo của cấu trúc Location:

```
Location loc1 = new Location( 200, 300);
```

Sau đó phương thức WriteLine() được gọi:

```
Console.WriteLine("Loc1 location: {0}", loc1);
```

Dĩ nhiên là WriteLine chờ đợi một đối tượng, nhưng Location là một cấu

trúc (một kiểu giá trị). Trình biên dịch sẽ tự động boxing cấu trúc (cũng giống như trình biên dịch đã làm với các kiểu dữ liệu giá trị khác). Một đối tượng sau khi boxing được truyền vào cho phương thức WriteLine(). Tiếp sau đó là phương thức ToString() được gọi trên đối tượng boxing này, do cấu trúc ngầm định kế thừa từ lớp object, và nó cũng có thể đáp ứng sự đa hình, bằng cách phủ quyết các phương thức như bất cứ đối tượng nào khác.

Loc1 location 200, 300

Tuy nhiên do cấu trúc là kiểu giá trị, nên khi truyền vào trong một hàm, thì chúng chỉ truyền giá trị vào hàm. Cũng như ta thấy ở dòng lệnh kế tiếp, khi đó một đối tượng Location được truyền vào phương thức myFunc():

t.myFunc(loc1);

Trong phương thức myFunc() hai giá trị mới được gán cho x và y, sau đó giá trị mới sẽ được xuất ra màn hình:

Loc1 location: 50, 100

Khi phương thức myFunc() trả về cho hàm gọi (Main()) và chúng ta gọi tiếp phương thức WriteLine() một lần nữa thì giá trị không thay đổi:

Loc1 location: 200, 300

Như vậy cấu trúc được truyền vào hàm như một đối tượng giá trị, và một bản sao sẽ được tạo bên trong phương thức myFunc(). Nếu chúng ta thử khai báo của Location là **class** như sau:

```
public class Location
```

Sau đó chạy lại chương trình thì có kết quả:

Loc1 location: 200, 3000

In myFunc loc: 50, 100

Loc1 location: 50, 100

Lúc này Location là một đối tượng tham chiếu nên khi truyền vào phương thức myFunc() thì việc gán giá trị mới cho x và y điều làm thay đổi đối tượng Location.

Gọi bộ khởi dựng mặc định

Như đề cập ở phần trước, nếu chúng ta không tạo bộ khởi dựng thì một bộ khởi dựng mặc định ngầm định sẽ được trình biên dịch tạo ra. Chúng ta có thể nhìn thấy điều này nếu bỏ bộ khởi dựng tạo ra:

```
/*public Location( int xCoordinate , int yCoordinate)
{
xVal = xCoordinate;
yVal = yCoordinate;
}
```

*/

và ta thay dòng lệnh đầu tiên trong hàm Main() tạo Location có hai tham số bằng câu lệnh tạo không tham số như sau:

```
//Location loc1 = new Location( 200, 300)
```

```
Location
```

```
loc1 = new Location();
```

Bởi vì lúc này không có phương thức khởi dựng nào khai báo, một phương thức khởi dựng ngầm định sẽ được gọi. Kết quả khi thực hiện giống như sau:

```
Loc1 location 0, 0
```

```
In myFunc loc: 50, 100
```

```
Loc1 location: 0, 0
```

Bộ khởi tạo mặc định đã thiết lập tất cả các biến thành viên với giá trị 0.

Tạo cấu trúc không gọi new

Bởi vì Location là một cấu trúc không phải là lớp, do đó các thể hiện của nó sẽ được tạo trong stack. Trong ví dụ trên khi toán tử **new** được gọi:

```
Location loc1 = new Location( 200, 300);
```

kết quả một đối tượng Location được tạo trên stack.

Tuy nhiên, toán tử **new** gọi bộ khởi dựng của lớp Location, không giống như với một lớp, cấu trúc có thể được tạo ra mà không cần phải gọi toán tử **new**. Điều này giống như các biến của các kiểu dữ liệu được xây dựng sẵn (như int, long, char,..) được tạo ra.

□ *Ghi chú:* Đây là một sự khuyến cáo, trong ví dụ sau chúng ta minh họa cách tạo một cấu trúc mà không phải sử dụng toán tử **new** bởi vì có sự khác nhau giữa C# và ngôn ngữ C++ và sự khác nhau này chính là cách ngôn ngữ C# đối xử với những lớp khác những cấu trúc. Tuy nhiên, việc tạo một cấu trúc mà không dùng từ khóa **new** sẽ không có lợi và có thể tạo một chương trình khó hiểu, tiềm ẩn nhiều lỗi, và khó duy trì.

6. Thực thi giao diện

Cú pháp để định nghĩa một giao diện như sau:

```
[thuộc tính] [bổ sung truy cập] interface <tên giao diện> [: danh sách cơ sở]
{
    <phần thân giao diện>
}
```

Phần thuộc tính chúng ta sẽ đề cập sau. Thành phần bổ sung truy cập bao gồm: **public**, **private**, **protected**, **internal**, và **protected internal**, ý nghĩa tương tự như các bổ sung truy cập của lớp.

Theo sau từ khóa **interface** là tên của giao diện. Thông thường tên của giao diện được bắt đầu với từ I hoa (điều này không bắt buộc nhưng việc đặt tên như vậy rất rõ ràng và dễ hiểu, tránh nhầm lẫn với các thành phần khác). Ví dụ một số giao diện có tên như sau:

IStorable, ICloneable,...

Danh sách cơ sở là danh sách các giao diện mà giao diện này mở rộng, phần này sẽ được trình bày trong phần thực thi nhiều giao diện của chương. Phần thân của giao diện chính là phần thực thi giao diện sẽ được trình bày bên dưới.

Giả sử chúng ta muốn tạo một giao diện nhằm mô tả những phương thức và thuộc tính của một lớp cần thiết để lưu trữ và truy cập từ một cơ sở dữ liệu hay các thành phần lưu trữ dữ liệu khác như là một tập tin. Chúng ta quyết định gọi giao diện này là IStorage.

Trong giao diện này chúng ta xác nhận hai phương thức: Read() và Write(), khai báo này sẽ được xuất hiện trong phần thân của giao diện như sau:

```
interface IStorable
{
    void Read();
    void Write(object);
}
```

Mục đích của một giao diện là để định nghĩa những khả năng mà chúng ta muốn có trong một lớp. Ví dụ, chúng ta có thể tạo một lớp tên là Document, lớp này lưu trữ các dữ liệu trong cơ sở dữ liệu, do đó chúng ta quyết định lớp này thực thi giao diện IStorable. Để làm được điều này, chúng ta sử dụng cú pháp giống như việc tạo một lớp mới Document được thừa kế từ IStorable bằng dấu hai chấm (:) và theo sau là tên giao diện:

```
public class Document : IStorable
{
    public void Read()
    {
        ....
    }
    public void Write()
    {
        ....
    }
}
```

Thực thi nhiều giao diện

Trong ngôn ngữ C# cho phép chúng ta thực thi nhiều hơn một giao diện. Ví dụ, nếu lớp Document có thể được lưu trữ và dữ liệu cũng được nén. Chúng ta có thể chọn thực thi cả hai giao diện IStorable và ICompressible. Như vậy chúng ta phải thay đổi phần khai báo trong danh sách cơ sở để chỉ ra rằng cả hai giao diện đều được thực thi, sử dụng dấu phẩy (,) để phân cách giữa hai giao diện:

```
public class Document : IStorable, ICompressible
```

Mở rộng giao diện

C# cung cấp chức năng cho chúng ta mở rộng một giao diện đã có bằng cách thêm các phương thức và các thành viên hay bổ sung cách làm việc cho các thành viên. Ví dụ, chúng ta có thể mở rộng giao diện ICompressible với một giao diện mới là ILoggedCompressible. Giao diện mới này mở rộng giao diện cũ bằng cách thêm phương thức ghi log các dữ liệu đã lưu:

```
interface ILoggedCompressible : ICompressible
{
    void LogSavedBytes();
}
```

Kết hợp các giao diện

Một cách tương tự, chúng ta có thể tạo giao diện mới bằng cách kết hợp các giao diện cũ và ta có thể thêm các phương thức hay các thuộc tính cho giao diện mới. Ví dụ, chúng ta quyết định tạo một giao diện IStorableCompressible. Giao diện mới này sẽ kết hợp những phương thức của cả hai giao diện và cũng thêm vào một phương thức mới để lưu trữ kích thước nguyên thủy của các dữ liệu trước khi nén:

```
interface IStorableCompressible : IStorable, ILoggedCompressible
{
    void LogOriginalSize();
}
```

Chúng ta có thể truy cập những thành viên của giao diện IStorable như thể là các thành viên của lớp Document:

```
Document doc = new Document("Test Document");
doc.status = -1;
doc.Read();
```

hay là ta có thể tạo thể hiện của giao diện bằng cách gán đối tượng Document cho một kiểu dữ liệu giao diện, và sau đó sử dụng giao diện này để truy cập các phương thức:

```
IStorable isDoc = (IStorable) doc;
isDoc.status = 0;
```

```
isDoc.Read();
```

□ *Ghi chú:* Cũng như đã nói trước đây, chúng ta không thể tạo thể hiện của giao diện một cách trực tiếp. Do đó chúng ta không thể thực hiện như sau:

```
IStorable isDoc = new IStorable();
```

Tuy nhiên chúng ta có thể tạo thể hiện của lớp thực thi như sau:

```
Document doc = new Document("Test Document");
```

Sau đó chúng ta có thể tạo thể hiện của giao diện bằng cách gán đối tượng thực thi đến kiểu dữ liệu giao diện, trong trường hợp này là IStorable:

```
IStorable isDoc = (IStorable) doc;
```

Chúng ta có thể kết hợp những bước trên như sau:

```
IStorable isDoc = (IStorable) new Document("Test Document");
```

Nói chung, cách thiết kế tốt nhất là quyết định truy cập những phương thức của giao diện thông qua tham chiếu của giao diện. Do vậy cách tốt nhất là sử dụng `isDoc.Read()`, hơn là sử dụng `doc.Read()` trong ví dụ trước. Truy cập thông qua giao diện cho phép chúng ta đối xử giao diện một cách đa hình. Nói cách khác, chúng ta tạo hai hay nhiều hơn những lớp thực thi giao diện, và sau đó bằng cách truy cập lớp này chỉ thông qua giao diện.

Khi thực thi một lớp chúng ta có thể tự do đánh dấu bất kỳ hay tất cả các phương thức thực thi giao diện như là một phương thức ảo. Ví dụ, lớp `Document` thực thi giao diện `IStorable` và có thể đánh dấu các phương thức `Read()` và `Write()` như là phương thức ảo. Lớp `Document` có thể đọc và viết nội dung của nó vào một kiểu dữ liệu `File`. Những người phát triển sau có thể dẫn xuất một kiểu dữ liệu mới từ lớp `Document`, có thể là lớp `Note` hay lớp `EmailMessage`, và những người này mong muốn lớp `Note` đọc và viết vào cơ sở dữ liệu hơn là vào một tập tin.

Trong việc thực thi giao diện cho tới giờ, những lớp thực thi (trong trường hợp này là `Document`) tạo ra các phương thức thành viên cùng ký hiệu và kiểu trả về như là phương thức được mô tả trong giao diện. Chúng ta không cần thiết khai báo tường minh rằng đây là một thực thi của một giao diện, việc này được hiểu ngầm bởi trình biên dịch.

Tuy nhiên, có vấn đề xảy ra khi một lớp thực thi hai giao diện và cả hai giao diện này có các phương thức cùng một ký hiệu. Ví dụ tạo ra hai giao diện: `IStorable` và `ITalk`. Sau đó thực thi phương thức `Read()` trong giao diện `ITalk` để đọc ra tiếng nội dung của một cuốn sách. Không may là phương thức này sẽ tranh chấp với phương thức `Read()` của `IStorable` mà `Document` phải thực thi.

Bởi vì cả hai phương thức `IStorable` và `ITalk` có cùng phương thức `Read()`, việc thực thi lớp `Document` phải sử dụng thực thi tường minh cho mỗi phương thức. Với việc thực thi tường minh, lớp thực thi `Document` sẽ khai báo tường minh cho mỗi phương thức:

```
void ITalk.Read();
```

Điều này sẽ giải quyết việc tranh chấp, nhưng nó sẽ tạo ra hàng loạt các hiệu ứng thú vị. Đầu tiên, không cần thiết sử dụng thực thi tường minh với những phương thức khác của Talk:

```
public void Talk();
```

vì không có sự tranh chấp cho nên ta khai báo như thông thường. Điều quan trọng là các phương thức thực thi tường minh không có bổ sung truy cập:

```
void ITalk.Read();
```

Phương thức này được hiểu ngầm là **public**.

Thật vậy, một phương thức được khai báo tường minh thì sẽ không được khai báo với các từ khóa bổ sung truy cập: **abstract**, **virtual**, **override**, và **new**.

Một điều quan trọng khác là chúng ta không thể truy cập phương thức thực thi tường minh thông qua chính đối tượng. Khi chúng ta viết:

```
theDoc.Read();
```

Trình biên dịch chỉ hiểu rằng chúng ta thực thi phương thức giao diện ngầm định cho IStorable. Chỉ một cách duy nhất truy cập các phương thức thực thi tường minh là thông qua việc gán cho giao diện để thực thi:

```
ITalk itDoc = theDoc as ITalk;
```

```
if ( itDoc != null )
```

```
{
```

```
itDoc.Read();
```

```
}
```

7. Mảng

7.1. Mảng

Mảng là một tập hợp có thứ tự của những đối tượng, tất cả các đối tượng này cùng một kiểu. Mảng trong ngôn ngữ C# có một vài sự khác biệt so với mảng trong ngôn ngữ C++ và một số ngôn ngữ khác, bởi vì chúng là những đối tượng. Điều này sẽ cung cấp cho mảng sử dụng các phương thức và những thuộc tính.

Ngôn ngữ C# cung cấp cú pháp chuẩn cho việc khai báo những đối tượng Array. Tuy nhiên, cái thật sự được tạo ra là đối tượng của kiểu System.Array. Mảng trong ngôn ngữ C# kết hợp cú pháp khai báo mảng theo kiểu ngôn ngữ C và kết hợp với định nghĩa lớp do đó thể hiện của mảng có thể truy cập những phương thức và thuộc tính của System.Array.

Khai báo mảng

Chúng ta có thể khai báo một mảng trong C# với cú pháp theo sau:

```
<kiểu dữ liệu>[] <tên mảng>
```

Ví dụ ta có khai báo như sau:

```
int[] myIntArray;
```

Cặp dấu ngoặc vuông ([]) báo cho trình biên dịch biết rằng chúng ta đang khai báo một mảng.

Kiểu dữ liệu là kiểu của các thành phần chứa bên trong mảng. Trong ví dụ bên trên, myIntArray được khai báo là mảng số nguyên.

Chúng ta tạo thể hiện của mảng bằng cách sử dụng từ khóa new như sau:

```
myIntArray = new int[6];
```

Khai báo này sẽ thiết lập bên trong bộ nhớ một mảng chứa sáu số nguyên

Giá trị mặc định

Khi chúng ta tạo một mảng có kiểu dữ liệu giá trị, mỗi thành phần sẽ chứa giá trị mặc định của kiểu dữ liệu. Với khai báo:

```
myIntArray = new int[5];
```

sẽ tạo ra một mảng năm số nguyên, và mỗi thành phần được thiết lập giá trị mặc định là 0, đây cũng là giá trị mặc định của số nguyên.

Không giống với mảng kiểu dữ liệu giá trị, những kiểu tham chiếu trong một mảng không được khởi tạo giá trị mặc định. Thay vào đó, chúng sẽ được khởi tạo giá trị null. Nếu chúng ta cố truy cập đến một thành phần trong mảng kiểu dữ liệu tham chiếu trước khi chúng được khởi tạo giá trị xác định, chúng ta sẽ tạo ra một ngoại lệ.

Truy cập các thành phần trong mảng

Để truy cập vào thành phần trong mảng ta có thể sử dụng toán tử chỉ mục ([]). Mảng dùng cơ sở 0, do đó chỉ mục của thành phần đầu tiên trong mảng luôn luôn là 0. Như ví dụ trước thành phần đầu tiên là myArray[0].

7.2. Danh sách mảng

Khởi tạo thành phần của mảng

Chúng ta có thể khởi tạo nội dung của một mảng ngay lúc tạo thể hiện của mảng bằng cách đặt những giá trị bên trong dấu ngoặc {}. C# cung cấp hai cú pháp để khởi tạo các thành phần của mảng, một cú pháp dài và một cú pháp ngắn:

```
int[] myIntArray = new int[5] { 2, 4, 6, 8, 10};
```

```
int[] myIntArray = { 2, 4, 6, 8, 10};
```

7.3. Câu lệnh foreach

Câu lệnh lặp **foreach** khá mới với những người đã học ngôn ngữ C, từ khóa này được sử dụng trong ngôn ngữ Visual Basic. Câu lệnh **foreach** cho phép chúng ta lặp qua tất cả các mục trong một mảng hay trong một tập hợp.

Cú pháp sử dụng lệnh lặp **foreach** như sau:

```
foreach (<kiểu dữ liệu thành phần> <tên truy cập> in <mảng/tập hợp> )
{
```

```
// thực hiện thông qua <tên truy cập> tương ứng với
// từng mục trong mảng hay tập hợp
}
```

8. Xử lý chuỗi

C# xem những chuỗi như là những kiểu dữ liệu cơ bản tức là các lớp này rất linh hoạt, mạnh mẽ, và nhất là dễ sử dụng. Mỗi đối tượng chuỗi là một dãy cố định các ký tự Unicode.

Nói cách khác, các phương thức được dùng để làm thay đổi một chuỗi thực sự trả về một bản sao đã thay đổi, chuỗi nguyên thủy không thay đổi. Khi chúng ta khai báo một chuỗi C# bằng cách dùng từ khóa string, là chúng ta đã khai báo một đối tượng của lớp System.String, đây là một trong những kiểu dữ liệu được xây dựng sẵn được cung cấp bởi thư viện lớp .NET (.NET Framework Class Library). Do đó một kiểu dữ liệu chuỗi C# là kiểu dữ liệu System.String, và trong suốt bài này dùng hai tên hoán đổi lẫn nhau.

Khai báo của lớp System.String như sau:

```
public sealed class String : IComparable, ICloneable, IConvertible
```

Khai báo này cho thấy lớp String đã được đóng dấu là không cho phép kế thừa, do đó chúng ta không thể dẫn xuất từ lớp này được. Lớp này cũng thực thi ba giao diện hệ thống là IComparable, ICloneable, và IConvertible – giao diện này cho phép lớp System.String chuyển đổi với những lớp khác trong hệ thống .NET.

Như chúng ta đã xem trong bài 6, giao diện IComparable được thực thi bởi các kiểu dữ liệu đã được sắp xếp. Ví dụ như chuỗi thì theo cách sắp xếp Alphabe. Bất cứ chuỗi nào đưa ra cũng có thể được so sánh với chuỗi khác để chỉ ra rằng chuỗi nào có thứ tự trước. Những lớp IComparable thực thi phương thức CompareTo().

Những đối tượng ICloneable có thể tạo ra những thể hiện khác với cùng giá trị như là thể hiện nguyên thủy. Do đó ta có thể tạo ra một chuỗi mới từ chuỗi ban đầu và giá trị của chuỗi mới bằng với chuỗi ban đầu. Những lớp ICloneable thực thi phương thức Clone().

Những lớp IConvertible cung cấp phương thức để dễ dàng chuyển đổi qua các kiểu dữ liệu cơ bản khác như là.ToInt32(), ToDouble(), ToDecimal(),...

Tạo một chuỗi

Cách phổ biến nhất để tạo ra một chuỗi là gán cho một chuỗi trích dẫn tức là chuỗi nằm trong dấu ngoặc kép, kiểu chuỗi này cũng được biết như là một chuỗi hằng, khai báo như sau:

```
string newString = "Day la chuoi hang";
```

Những chuỗi trích dẫn có thể được thêm các ký tự escape, như là “\n” hay “\t”, ký tự này bắt đầu với dấu chéo ngược (“\”), các ký tự này được dùng để chỉ

ra rằng tại vị trí đó xuống dòng hay tab được xuất hiện. Bởi vì dấu gạch chéo ngược này cũng được dùng trong vài cú pháp dòng lệnh, như là địa chỉ URLs hay đường dẫn thư mục, do đó trong chuỗi trích dẫn dấu chéo ngược này phải được đặt trước dấu chéo ngược khác, tức là dùng hai dấu chéo ngược trong trường hợp này.

Tạo chuỗi dùng phương thức ToString của đối tượng

Một cách rất phổ biến khác để tạo một chuỗi là gọi phương thức ToString() của một đối tượng và gán kết quả đến một biến chuỗi. Tất cả các kiểu dữ liệu cơ bản phủ quyết phương thức này rất đơn giản là chuyển đổi giá trị (thông thường là giá trị số) đến một chuỗi thể hiện của giá trị. Trong ví dụ theo sau, phương thức ToString() của kiểu dữ liệu int được gọi để lưu trữ giá trị của nó trong một chuỗi:

```
int myInt = "9";
string intString = myInt.ToString();
```

Phương thức myInt.ToString() trả về một đối tượng String và đối tượng này được gán cho intString.

Thao tác trên chuỗi

Lớp string cung cấp rất nhiều số lượng các phương thức để so sánh, tìm kiếm và thao tác trên chuỗi, các phương thức này được trình bày trong bảng sau:

System.String	
Phương thức/ Trường	Ý nghĩa
Empty	Trường public static thể hiện một chuỗi rỗng.
Compare()	Phương thức public static để so sánh hai chuỗi.
CompareOrdinal()	Phương thức public static để so sánh hai chuỗi không quan tâm đến thứ tự.
Concat()	Phương thức public static để tạo chuỗi mới từ một hay nhiều chuỗi.
Copy()	Phương thức public static tạo ra một chuỗi mới bằng sao từ chuỗi khác.
Equal()	Phương thức public static kiểm tra xem hai chuỗi có cùng giá trị hay không.
Format()	Phương thức public static định dạng một chuỗi dùng ký tự lệnh định dạng xác định.
Intern()	Phương thức public static trả về tham chiếu đến thể hiện của chuỗi.
IsInterned()	Phương thức public static trả về tham chiếu của chuỗi
Join()	Phương thức public static kết nối các chuỗi xác định giữa mỗi thành phần của mảng chuỗi.
Chars()	Indexer của chuỗi.
Length()	Chiều dài của chuỗi.

Biểu thức qui tắc là một ngôn ngữ mạnh dùng để mô tả và thao tác văn bản. Một biểu thức qui tắc thường được áp dụng cho một chuỗi, hay một tập hợp các ký tự. Thông thường một chuỗi là toàn bộ văn bản hay tài liệu.

Kết quả của việc áp dụng một biểu thức qui tắc đến một chuỗi là trả về một chuỗi con hoặc là trả về một chuỗi mới có thể được bổ sung từ một vài phần của chuỗi nguyên thủy ban đầu. Chúng ta nên nhớ rằng string là không thể thay đổi được và do đó cũng không thể thay đổi bởi biểu thức qui tắc.

Bằng cách áp dụng chính xác biểu thức qui tắc cho chuỗi sau:

Mot, hai, ba, Trung Tam Dao Tao CNTT

chúng ta có thể trả về bất cứ hay tất cả danh sách các chuỗi con (Mot, hai,...) và có thể tạo ra các phiên bản chuỗi được bổ sung của những chuỗi con (như : TrUNG TAM,...).

Biểu thức qui tắc này được quyết định bởi cú pháp các ký tự qui tắc của chính bản thân nó.

Một biểu thức qui tắc bao gồm hai kiểu ký tự:

Ký tự bình thường (literal): những ký tự này mà chúng ta sử dụng để so khớp với chuỗi ký tự đích.

Metacharacter: là các biểu tượng đặc biệt, có hành động như là các lệnh trong bộ phân tích (parser) của biểu thức.

Sử dụng biểu thức quy tắc qua lớp Regex

MS.NET cung cấp một hướng tiếp cận hướng đối tượng (object-oriented approach) cho biểu thức quy tắc để so khớp, tìm kiếm và thay thế chuỗi. Biểu thức quy tắc của ngôn ngữ C# là được xây dựng từ lớp regex của ngôn ngữ Perl5.

Namespace System.Text.RegularExpressions của thư viện BCL (Base Class Library) chứa đựng tất cả các đối tượng liên quan đến biểu thức quy tắc trong môi trường .NET. Và lớp quan trọng nhất mà biểu thức quy tắc hỗ trợ là Regex.

Sử dụng Regex để tìm kiếm tập hợp

Hai lớp được thêm vào trong namespace .NET cho phép chúng ta thực hiện việc tìm kiếm một chuỗi một cách lặp đi lặp lại cho đến hết chuỗi, và kết quả trả về là một tập hợp. Tập hợp được trả về có kiểu là MatchCollection, bao gồm không có hay nhiều đối tượng Match.

Sử dụng Regex để gom nhóm

Đôi khi lập trình chúng ta cần gom nhóm một số các biểu thức tương tự với nhau theo một quy định nào đó. Ví dụ như chúng ta cần tìm kiếm địa chỉ IP và nhóm chúng lại vào trong nhóm IPAddresses được tìm thấy bất cứ đâu trong một chuỗi.

Lớp Group cho phép chúng ta tạo những nhóm và tìm kiếm dựa trên biểu thức quy tắc, và thể hiện kết quả từ một nhóm biểu thức đơn.

Một biểu thức nhóm định rõ một nhóm và cung cấp một biểu thức quy tắc,

bất cứ chuỗi con nào được so khớp bởi biểu thức quy tắc thì sẽ được thêm vào trong nhóm. Ví dụ, để tạo một nhóm chúng ta có thể viết như sau:

```
@”(?<ip>(\d|\.)+)\s”
```

Bài tập:

Bài tập 1: Nhập vào, biên dịch và chạy chương trình. Hãy cho biết chương trình làm điều gì?

```
-----
class BaiTap1_1
{
public static void Main()
{
int x = 0;
for(x = 1; x < 10; x++)
{
System.Console.Write(“{0:03}”, x);
}
}
}
```

Bài tập 2: Tìm lỗi của chương trình sau? sửa lỗi và biên dịch chương trình.

```
-----
class BaiTap1_2
{
public static void Main()
{
for(int i=0; i < 10 ; i++)
System.Console.WriteLine(“so :{1}”, i);
}
}
```

Bài tập 3: Tìm lỗi của chương trình sau. Sửa lỗi và biên dịch lại chương trình.

```
-----
using System;
```

```

class BaiTap1_3
{
public static void Main()
{
double myDouble;
decimal myDecimal;
myDouble = 3.14;
myDecimal = 3.14;
Console.WriteLine("My Double: {0}", myDouble);
Console.WriteLine("My Decimal: {0}", myDecimal);
}
}

```

Bài tập 4: Tìm lỗi của chương trình sau. Sửa lỗi và biên dịch lại chương trình.

```

class BaiTap1_4
{
static void Main()
{
int value;
if (value > 100);
System.Console.WriteLine("Number is greater than 100");
}
}

```

Bài tập 5: Xây dựng một lớp đường tròn lưu giữ bán kính và tâm của đường tròn. Tạo các phương thức để tính chu vi, diện tích của đường tròn.

Bài tập 6: Thêm thuộc tính *BanKinh* vào lớp được tạo ra từ bài tập 1.

Bài tập 7: Tạo ra một lớp lưu trữ giá trị nguyên tên *myNumber*. Tạo thuộc tính cho thành viên này. Khi số được lưu trữ thì nhân cho 100. Và khi số được truy cập thì chia cho 100.

Bài tập 8: Chương trình sau có lỗi. Hãy sửa lỗi của chương trình và biên dịch chương trình. Dòng lệnh nào gây ra lỗi?

```

using System;
using System.Console;
class VD1
{
    public string first;
}
class Tester
{
    public static void Main()
    {
        VD1 vd = new VD1();
        Write("Nhap vao mot chuoi: ");
        vd.first = ReadLine();
        Write("Chuoi nhap vao: {0}", vd.first);
    }
}

```

Bài tập 9: Hãy xây dựng các lớp đối tượng trong câu hỏi 3, thiết lập các quan hệ kế thừa dựa trên cây kế thừa mà bạn xây dựng. Mỗi đối tượng chỉ cần một thuộc tính là myName để cho biết tên của nó (như Xe_Toyota thì myName là "Toi la Toyota"...). Các đối tượng có phương thức Who() cho biết giá trị myName của nó. Hãy thực thi sự đa hình trên các lớp đó. Cuối cùng tạo một lớp Tester với hàm Main() để tạo một mảng các đối tượng Xe, đưa từng đối tượng cụ thể vào mảng đối tượng Xe, sau đó cho lặp từng đối tượng trong mảng để nó tự giới thiệu tên (bằng cách gọi hàm Who() của từng đối tượng).

Bài tập 10: Xây dựng lớp điểm trong không gian hai chiều, với các toán tử cộng, trừ, nhân, chia.

Bài tập 11: Tương tự như bài tập 2 nhưng điểm nằm trong không gian 3 chiều.

Bài tập 12: Xây dựng lớp số phức (số ảo) với các phép toán cộng, trừ, nhân, chia.

Bài tập 13: Xây dựng cấu trúc phân số với dữ liệu thành viên là tử số và mẫu số, xây dựng các hàm cộng, trừ, nhân, chia để thực hiện các phép toán trên hai phân số.

Bài tập 14: Xây dựng cấu trúc sinh viên.

Bài tập 15: Hãy viết một giao diện khai báo một thuộc tính ID chứa chuỗi giá trị. Viết một lớp Employee thực thi giao diện đó.

Bài tập 16: Đoạn mã nguồn sau đây có lỗi hãy sửa lỗi và hãy cho biết tại sao có lỗi này. Sau khi sửa lỗi hãy viết một lớp Circle thực thi giao diện này?

```
public interface IDimensions
{
    long width;
    long height;
    double Area();
    double Circumference();
    int Side();
}
```

Bài tập 17: Chương trình sau đây có lỗi hãy sửa lỗi, biên dịch và chạy lại chương trình? Giải thích tại sao chương trình có lỗi.

```
using System;
interface IPoint
{
    // Property signatures:
    int x
    {
        get;
        set;
    }
    int y
    {
        get;
        set;
    }
}
class MyPoint : IPoint
{
    // Fields:
```

```
private int myX;
private int myY;
// Constructor:
public MyPoint(int x, int y)
{
    myX = x;
    myY = y;
}
// Property implementation:
public int x
{
    get
    {
        return myX;
    }
    set
    {
        myX = value;
    }
}
public int y
{
    get
    {
        return myY;
    }
    set
    {
        myY = value;
    }
}
}
class MainClass
```

```
{  
private static void PrintPoint(IPoint p)  
{  
    Console.WriteLine("x={0}, y={1}", p.x, p.y);  
}  
public static void Main()  
{  
    MyPoint p = new MyPoint(2,3);  
    Console.Write("My Point: ");  
    PrintPoint(p);  
    IPoint p2 = new IPoint();  
    PrintPoint(p2);  
}  
}
```

Bài tập 18: Xây dựng một giao diện IDisplay có khai báo thuộc tính Name kiểu chuỗi. Hãy viết hai lớp Dog và Cat thực thi giao diện IDisplay, cho biết thuộc tính Name là tên của đối tượng.

BÀI 2: XÂY DỰNG GIAO DIỆN NGƯỜI DÙNG

Giới thiệu:

Giao diện người dùng (User Interface – UI) là thành phần giúp người sử dụng tương tác với chương trình hoặc phần mềm thông qua các đối tượng trực quan như cửa sổ, nút nhấn, hộp thoại, menu, biểu mẫu và các điều khiển khác. Trong lập trình ứng dụng Windows bằng C#, giao diện người dùng đóng vai trò quan trọng trong việc tạo ra môi trường làm việc thân thiện, dễ sử dụng và nâng cao trải nghiệm người dùng.

Thông qua việc thiết kế giao diện, lập trình viên có thể xây dựng các ứng dụng trực quan, hỗ trợ nhập liệu, xử lý dữ liệu và hiển thị kết quả một cách hiệu quả. Các công cụ như Windows Forms cho phép tạo giao diện bằng phương pháp kéo thả kết hợp với lập trình xử lý sự kiện, giúp việc phát triển ứng dụng trở nên nhanh chóng và thuận tiện hơn.

Mục tiêu:

Học xong bài này, sinh viên có khả năng:

- Trình bày các phương thức về đường dẫn và tập tin.
- Trình bày được hai loại thực đơn của .NET.
- Vận dụng được các sự kiện sử dụng cho phù hợp.
- Vận dụng được các loại hộp thoại tập tin.
- Tạo được hộp thoại lưu/mở tập tin.
- Tạo được thực đơn ngữ cảnh.
- Nghiêm túc trong học tập và thực hiện tốt các yêu cầu được giao.

Nội dung chính:

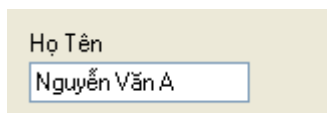
1. Các đối tượng

1.1. Các nhãn và hộp nhập văn bản, lớp nút nhấn

a. Label

Công dụng: Control được sử dụng để hiển thị chữ trên form và không cho phép người dùng thay đổi. Label được sử dụng để mô tả thông tin cho những control khác trên Form.

Ví dụ:



Dòng chữ xuất hiện bên trên TextBox đó là Label, có mục đích giải thích cho TextBox.

Tạo Label:



- Chọn công cụ
- rê chuột và vẽ Label trên form.

Thuộc tính cơ bản:

Thuộc tính	Ý nghĩa
AutoSize	Điều chỉnh kích thước đối tượng cho vừa với chiều dài chuỗi ký tự
Font	
Name	Quy định font chữ cho văn bản
Bold	True:Đậm/False:bỏ đậm
Italic	True:Nghiêng/False:bỏ nghiêng
Size	Quy định cỡ chữ cho văn bản
TextAlign	Canh lề(Left/Center/Right)

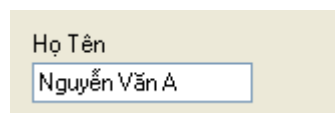
b. Textbox

Công dụng:

- Dùng trình bày văn bản và cho phép người dùng được thay đổi nội dung văn bản.

- Công dụng chính là cho người dùng nhập văn bản. Mặc định giá trị lớn nhất mà TextBox nhận là 2048 ký tự.

Ví dụ:



Tạo TextBox:

- Chọn công cụ  TextBox

- rê chuột và vẽ TextBox trên form.

Thuộc tính cơ bản:

Thuộc tính	Ý nghĩa
Text	Xác định giá trị hiển thị bên trong TextBox
Multiline	Cho phép TextBox hiển thị nhiều dòng chữ
PassWordChar	Thuộc tính này cho phép lấy một ký tự làm đại diện cho tất cả các ký tự khác được nhập vào từ người dùng

c. Button

Công dụng:

- Dùng để thực thi lệnh.

- Khi nhấp chuột lên button, chương trình nhận được tín hiệu Click và lệnh được thi hành.

- Sự kiện click được sử dụng khi người dùng nhấn vào nó:

```
private void button1_Click(object sender, EventArgs e)
```

```
{
    this.button1.Text = this.Text;
}
```

Tạo Button:

- Chọn công cụ
- rê chuột và vẽ Button trên form.

Thuộc tính cơ bản:

Thuộc tính	Ý nghĩa
Text	Nhập nội dung vào Button

1.2. Các sự kiện bàn phím và chuột

a. Sự kiện bàn phím

Phát sinh khi một phím được nhấn hoặc thả

Có 3 sự kiện: KeyPress, KeyUp, KeyDown

KeyPress phát sinh kèm theo với mã ASCII của phím được nhấn

KeyPress không cho biết trạng thái các phím bổ sung {Shift, Alt, Ctrl...}

Sử dụng KeyUp & KeyDown để xác định trạng thái các phím bổ sung.

Sự kiện với tham số kiểu EventArgs	
KeyDown	Phát sinh khi phím được nhấn
KeyUp	Phát sinh khi phím được thả
Sự kiện với tham số kiểu KeyEventArgs	
KeyPress	Khởi tạo khi phím được nhấn
Thuộc tính của lớp KeyEventArgs	
KeyChar	Chứa ký tự ASCII của phím được nhấn
Handled	Cho biết sự kiện KeyPress có được xử lý chưa
Thuộc tính của lớp KeyEventArgs	
Alt, Control, Shift	Trạng thái các phím bổ sung
Handled	Cho biết sự kiện đã xử lý
KeyCode	Trả về mã ký tự được định nghĩa trong Keys enumeration
KeyData	Chứa mã ký tự với thông tin phím bổ sung
KeyValue	Trả về số int, đây chính là mã Windows Virtual Key Code
Modifier	Trả về giá trị của phím bổ sung

Ví dụ minh họa các sự kiện: KeyPress, KeyDown, KeyUp

Khi user nhấn một phím

Bắt sự kiện KeyPress: xuất ra phím được nhấn

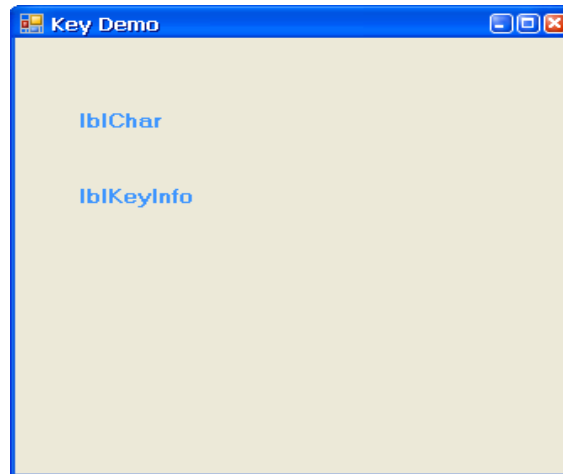
Bắt sự kiện KeyDown: xuất ra các tham số trong KeyEventArgs

Khi user thả phím

Xóa các thông tin mô tả phím được nhấn trong các label

Cách thực hiện

Tạo một form minh họa

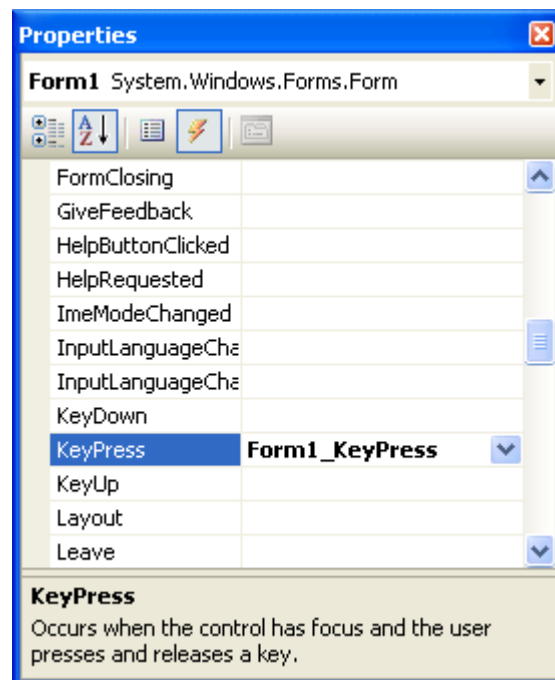


Thiết kế trên form có 2 Label:

lblChar: hiển thị ký tự được nhấn trong KeyPress

lblKeyInfo: hiển thị các thông tin của KeyEventArgs khi KeyDown

Tạo KeyPress Event Handling cho form



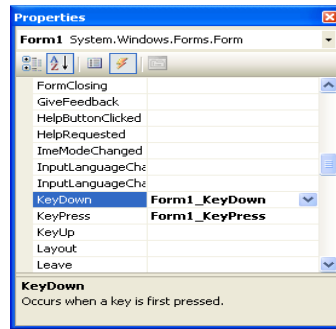
```
private void Form1_KeyPress(Object sender, KeyPressEventArgs e0
```

```
{//Hiển thị ký tự trên label
```

```
lblChar.Text="Key pressed:"+e.KeyChar;
```

}

Tạo KeyDown Event Handling cho form



```
private void Form1_KeyDown(object sender, KeyEventArgs e)
{
    lblKeyInfo.Text=
    "Alt:"+(“e.Alt?”Yes”.”No”)+’\n\’+
    “Shift.”+(“e.Shift?”Yes”.”No”)+’\n\’+
    “Ctrl.”+(“e.Control?”Yes”.”No”)+’\n\’+
    “KeyCode: “+e.KeyCode+’\n\’+
    “KeyData.”+ e.KeyData+’\n\’+
    “KeyValue.”+e.KeyValue+’\n\’;
}
```

b. Sự kiện chuột

Mouse là thiết bị tương tác thông dụng trên GUI

Một số các thao tác phát sinh từ mouse.

Di chuyển

Kích chuột

Ứng dụng cần xử lý sự kiện chuột nào sẽ khai báo trình xử lý tương ứng.

Lớp MouseEventArgs được sử dụng để chứa thông tin truyền vào cho trình xử lý sự kiện mouse.

Mỗi trình xử lý sự kiện sẽ có tham số là đối tượng object và đối tượng MouseEventArgs (hoặc EventArgs)

Sự kiện chuột với tham số kiểu EventArgs

MouseEnter	Xuất hiện khi con trỏ chuột đi vào vùng biên của control
MouseLeave	Xuất hiện khi con trỏ chuột rời khỏi biên của control

Sự kiện chuột với tham số kiểu MouseEventArgs

MouseDown/ MouseUp	Xuất hiện khi button được nhấn/thả và con trỏ chuột đang ở trong vùng biên của control
-----------------------	--

MouseMove	Xuất hiện khi chuột di chuyển và con trỏ chuột ở trong vùng biên của control
Button	Button được nhấn {Left, Right, Middle, none} có kiểu là MouseButton
Clicks	Số lần button được nhấn
X	Tọa độ x của con trỏ chuột trong control
Y	Tọa độ y của con trỏ chuột trong control

Ví dụ minh họa kích chuột trái tại một điểm A, giữ chuột trái và di chuyển chuột, chương trình sẽ vẽ đường thẳng từ điểm A đến vị trí hiện tại chuột.

Các sự kiện cần xử lý

MouseDown:

Xác định điểm A ban đầu

MouseMove

Kiểm tra nếu Left button của chuột đang giữ

Sử dụng Graphics để vẽ đường thẳng từ A đến vị trí hiện tại

Cách thực hiện:

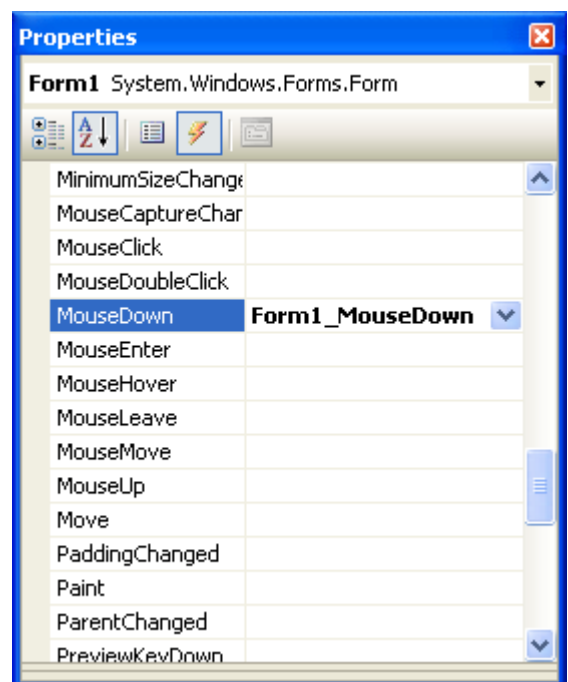
Tạo biến lưu trữ điểm A khi user kích chuột trái

Biến pA có kiểu Point là biến thành viên của Form1

```
public partial class Form1:Form
{
    private Point Pa;
    public Form1(){
        InitializeComponent();}
}
```

Khai báo xử lý sự kiện MouseDown trong Form1

Trong cửa sổ event của Form1, kích đúp vào sự kiện MouseDown



```
private void Form1_MouseDown(object sender, MouseEventArgs e)
{
    //lưu lại điểm thứ 1
    pA=e.Location;
}
```

Cài đặt xử lý sự kiện MouseMove
 Kiểm tra nếu LeftButton được nhấn
 Vẽ đường thẳng từ pA đến vị trí hiện tại
 private void Form1_MouseMove(object sender, MouseEventArgs e)

```
{
    if(e.Button==MouseButtons.Left)//LeftButton
    được nhấn
```

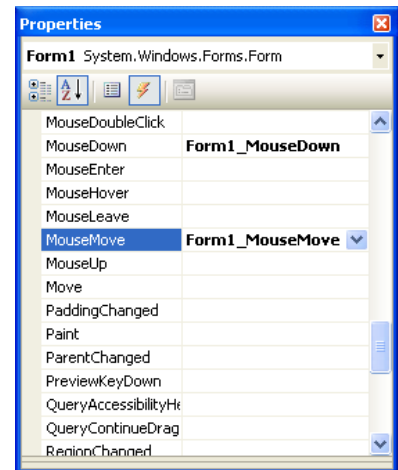
```
{
    Graphics g=this.CreateGraphics(); // lấy thiết bị đồ họa
```

```
    Pen pen=new Pen(Color.Red,2f); //tạo bút vẽ
```

```
    g.DrawLine(pen,pA,e.Location);// vẽ đường thẳng
```

```
}
```

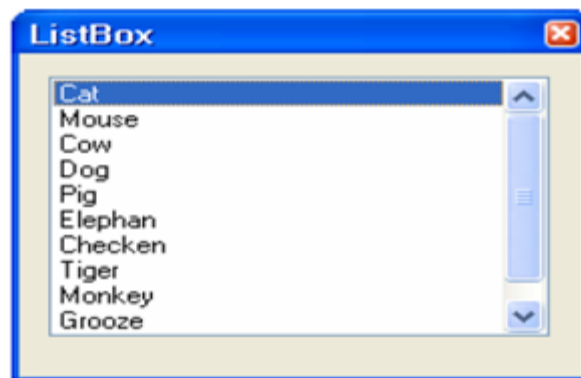
```
}
```



1.3. ListBox, ComboBox, Calendar

a. ListBox

Công dụng: Dùng để hiển thị một danh sách các phần tử lựa chọn. Người dùng có thể chọn một trong những phần tử này.



Ta có thể thêm danh mục những phần tử vào trong một ListBox bằng cách, chọn ListBox, tại cửa sổ Properties Window, chọn thuộc tính Items, nhấn vào nút  và cửa sổ String Collection Editor xuất hiện.

Tạo ListBox:

Chọn công cụ



Rê chuột và vẽ ListBox trên form.

Thuộc tính cơ bản:

Tên thuộc tính	Ý nghĩa
SelectionMode	Xác định cách thức mà người dùng lựa chọn những phần tử trong ListBox. Có 4 giá trị None: Người dùng không thể chọn bất cứ phần tử nào trong ListBox One: Người sử dụng chỉ chọn một giá trị từ ListBox MultiSimple: Cho phép người sử dụng chọn nhiều phần tử từ ListBox MultiExtended: Người dùng có thể chọn nhiều phần tử và sử dụng phím SHIFT, CTRL và phím mũi tên để chọn những phần tử từ ListBox
Sorted	Thuộc tính xác định khi nào những phần tử trong ListBox có thể được sắp xếp
SelectedIndex	Thuộc tính cho phép gán hoặc nhận vị trí được chọn hiện tại trong ListBox
SelectedItem	Thuộc tính này được dùng để gán hoặc nhận phần tử đang được chọn

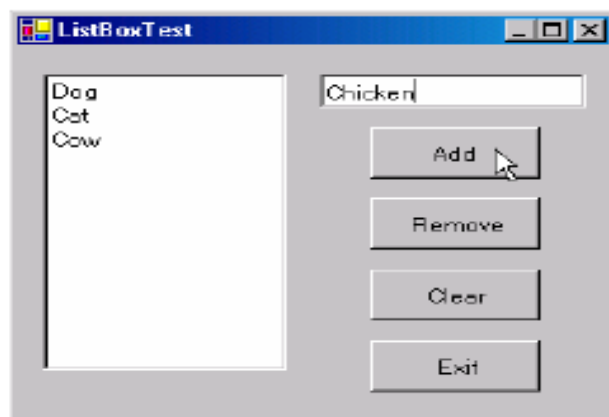
Phương thức:

Phương thức	Công dụng
Add("Chuỗi")	Thêm một mục giá trị là chuỗi
ToString()	Trả về chuỗi ký tự được chọn

Nhập giá trị vào ListBox: <Ten_ListBox>.Items.Add ("Chuỗi") ;

Lấy giá trị trong ListBox: <Ten_ListBox>.SelectedItem.ToString() ;

Sự kiện thường dùng: SelectedIndexChanged: Xảy ra khi chọn một dòng.
Ngầm định khi nháy đúp ở chế độ thiết kế.



Ví dụ 1.1: Xây dựng Form

```
private void btnAdd_Click(object sender, EventArgs e)
{
```

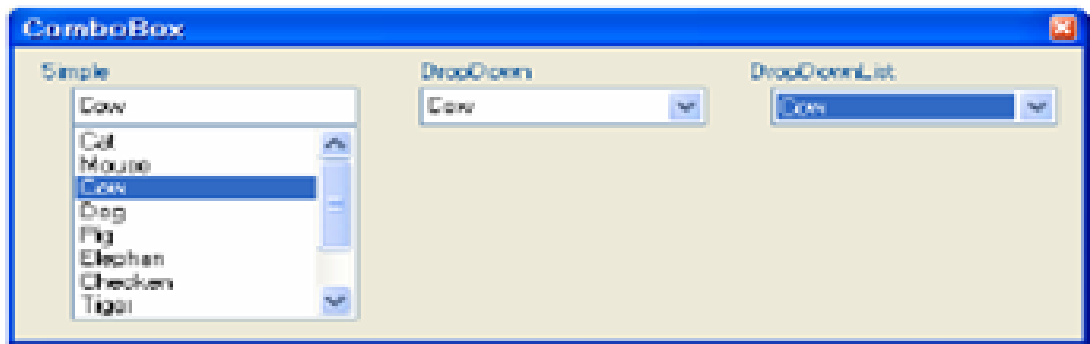
```

listBox1.Items.Add(txtInput.Text);
}
private void btnRemove_Click(object sender,EventArgs e)
{
int row=ListBox1.SelectedIndex;
if(row!=-1) listBox1.Items.RemoveAt(row);
}
private void btnClear_Click(object sender,EventArgs e)
{
listBox1.Items.Clear();
}

```

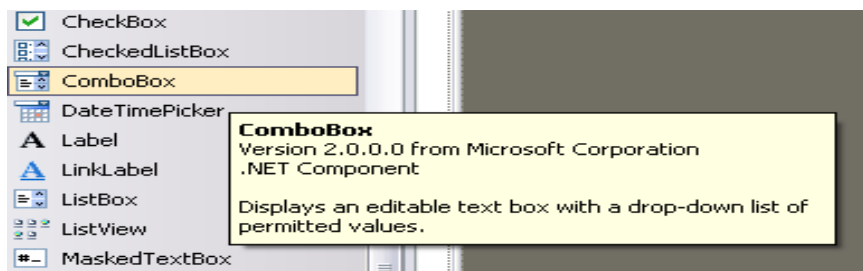
2/ ComboBox

Công dụng: Dùng để hiển thị một danh sách các lựa chọn / hoặc nhập vào một giá trị. Là sự kết hợp giữa ListBox và TextBox.



Tạo ComboBox:

- Chọn công cụ



- Rê chuột và vẽ ComboBox trên form.

Thuộc tính:

Thuộc tính	Ý nghĩa
DisplayMember	Gán nội dung thể hiện trên ComboBox
Items	Liệt kê các mục giá trị trong ComboBox
SelectedItem	Lấy Item được chọn
SelectedText	Lấy nội dung thể hiện trên ComboBox từ DisplayMember

SelectedValue	Lấy giá trị từ ValueMember
ValueMember	Gán giá trị cho ComboBox
DropDownStyle	Xác định kiểu của ComboBox Simple: Chọn hoặc gõ giá trị DropDown(mặc định): Chọn hoặc gõ giá trị DropDownList: Chỉ cho phép chọn giá trị
SelectedIndex	Chỉ số dòng được chọn. Nếu không chọn có giá trị là -1
Sorted	Có/Không sắp xếp dữ liệu trong ComboBox. Mặc định là False

Sự kiện thường dùng: SelectedIndexChanged: Xảy ra khi chọn 1 dòng.

Các phương thức thường dùng:

Add: Thêm một dòng vào ComboBox

RemoveAt(row): Xóa dòng ở vị trí row

Clear: Xóa tất cả các dòng trong ComboBox

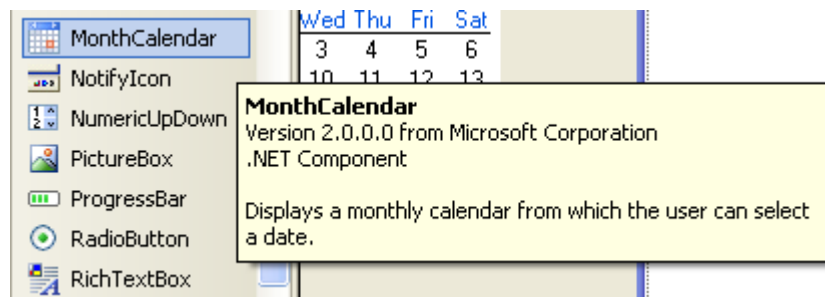
c. Calendar

Calendar được sử dụng để hiển thị ngày tháng. Người sử dụng có thể xem ngày tháng theo tuần hoặc tháng.



Tạo Calendar

- Chọn công cụ



- Kéo và thả vào form.

Những thuộc tính thường sử dụng bao gồm:

Thuộc tính	Ý nghĩa
DayNameFormat	Thuộc tính này được sử dụng để xác định định dạng tên

	của những ngày trong tuần
VisibleDay	Được sử dụng để xác định tháng được xuất hiện hay không trong Calendar. Thuộc tính này được cập nhật sau khi sự kiện VisibleMonthChanged được thực hiện
FirstdayOfWeek	Thuộc tính này được sử dụng để xác định ngày của tuần được xuất hiện tại cột đầu tiên của điều khiển Calendar
SelectedDate	Thuộc tính này được dùng để biểu diễn cho ngày được chọn trong điều khiển Calendar
SelectionMode	Thuộc tính này được sử dụng để xác định khi nào một người dùng có thể chọn một ngày, một tuần, hoặc một tháng. Mặc định, giá trị của thuộc tính này là Day

1.4. Image, ImageList

a. PictureBox

Công dụng: Là khung chứa hình ảnh.

Tạo PictureBox:

- Chọn công cụ
- rê chuột và vẽ PictureBox.

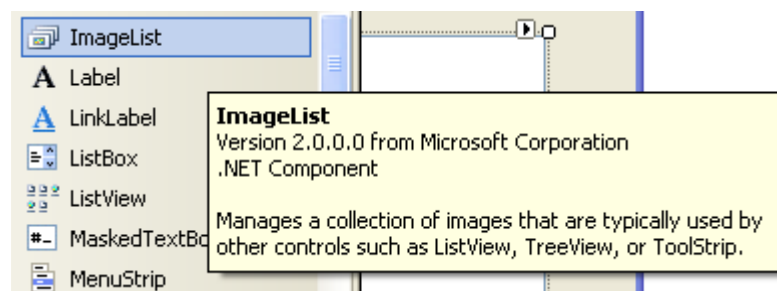
Thuộc tính cơ bản:

Thuộc tính	Ý nghĩa
Image	Hình chứa trong <i>PictureBox</i>

b. ImageList

Công dụng: Quản lý các hình ảnh được chứa trong các điều khiển khác như: ListView, TreeView, ToolStrip,...

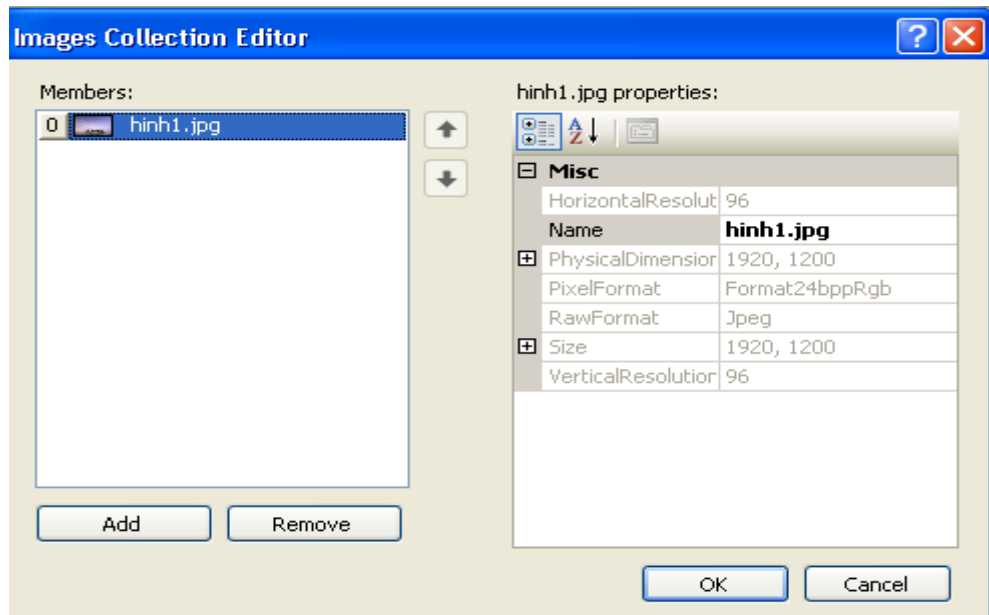
Tạo ImageList: Chọn công cụ và kéo thả vào form.



Các thuộc tính thường dùng:

Thuộc tính	Ý nghĩa
Image	Các hình được quản lý bởi ImageList Images (Collection)

- Khi chọn  sẽ xuất hiện công cụ Images Collection Editor, cho phép thêm, xóa, sắp xếp các ảnh.



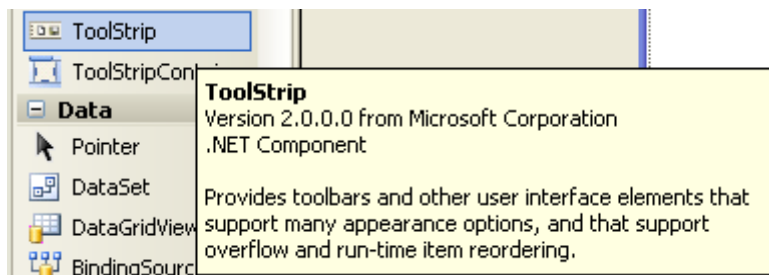
1.5. Toolbar, Tab, Tabpage, Tooltip

a. Toolbar

Công dụng: Tạo ra Toolbar.

Tạo ToolStrip:

- Chọn công cụ



- Kéo control ToolStrip vào trong form.

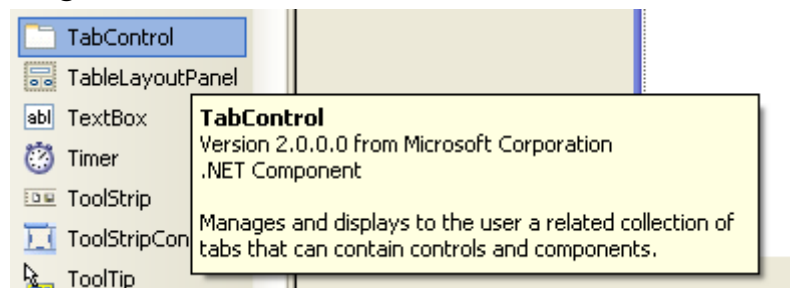
- Bấm vào ToolStrip vừa tạo, ta có thể tạo ra các tool (công cụ) như: Label, TextBox, Button, ComboBox, DropDownButton....

b. Tab và Tabpage

Công dụng: Tab Control được sử dụng để tạo nhiều vùng làm việc cho người sử dụng.

Tạo Tab:

- Chọn công cụ

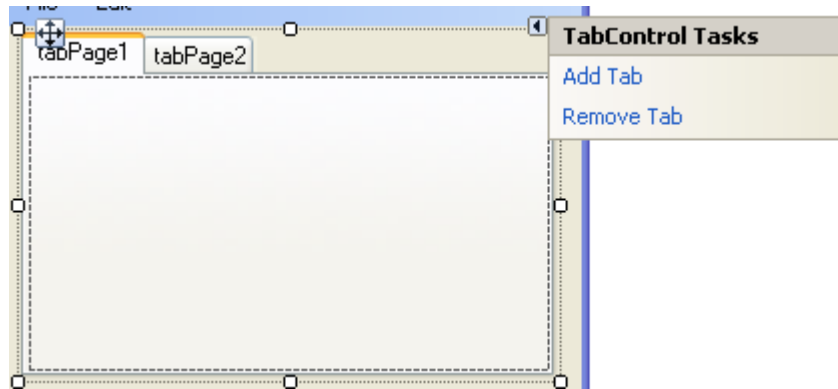


- Kéo Tab Control vào form.

Thuộc tính cơ bản:

Thuộc tính	Ý nghĩa
Name	Tên Tab
Tabpages	Cho phép chỉnh sửa, sắp xếp lại các TabPage

- Mặc định tab có 2 TabPage, để thêm TabPage ta nhấn Add Tab, để xóa TabPage ta chọn remove Tab



- Ta cũng có thể thêm hoặc hủy tabPage bằng các phương thức:

Thêm TabPage: TabPage.Add

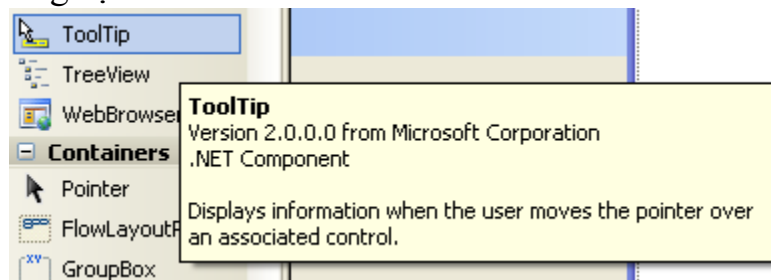
Xoá TabPage: TabPage.Remove

c. Tooltip

Công dụng: Hiển thị thông tin khi người dùng di chuyển chuột đến một điều khiển có sử dụng kết hợp công cụ Tooltip.

Tạo ToolStrip:

- Chọn công cụ



- Kéo control Tooltip vào trong form

1.6. ListView, TreeView

a. ListView

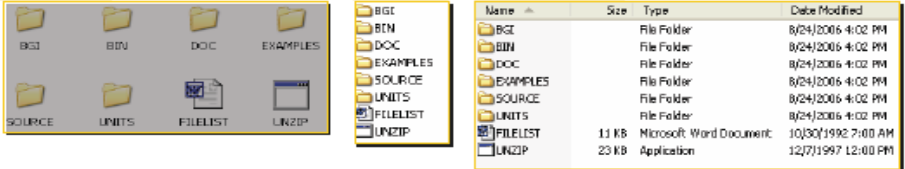
- Dùng để hiển thị dữ liệu theo các dòng và các cột

+ Có thể chọn một hoặc nhiều dòng

+ Có thể hiển thị các biểu tượng theo các dòng

- Thuộc tính cơ bản:

Thuộc tính	Ý nghĩa
CheckBoxes	Có/Không xuất hiện các checkbox trên các dòng dữ liệu. Mặc định

	là False
Columns	Các cột hiển thị trong chế độ Details
FullRowSelect	Chọn được nhiều phân tử trong ListView
GridLines	Hiển thị lưới (Chỉ hiển thị trong chế độ Details)
Items	Mảng các dòng trong ListView
LargeImageList	Danh sách ảnh dạng lớn hiển thị trên ListView
SmallImageList	Danh sách ảnh dạng nhỏ hiển thị trên ListView
MultiSelect	Có/Không cho phép chọn nhiều dòng. Mặc định là True
SelectedItems	Mảng các dòng được chọn
View	

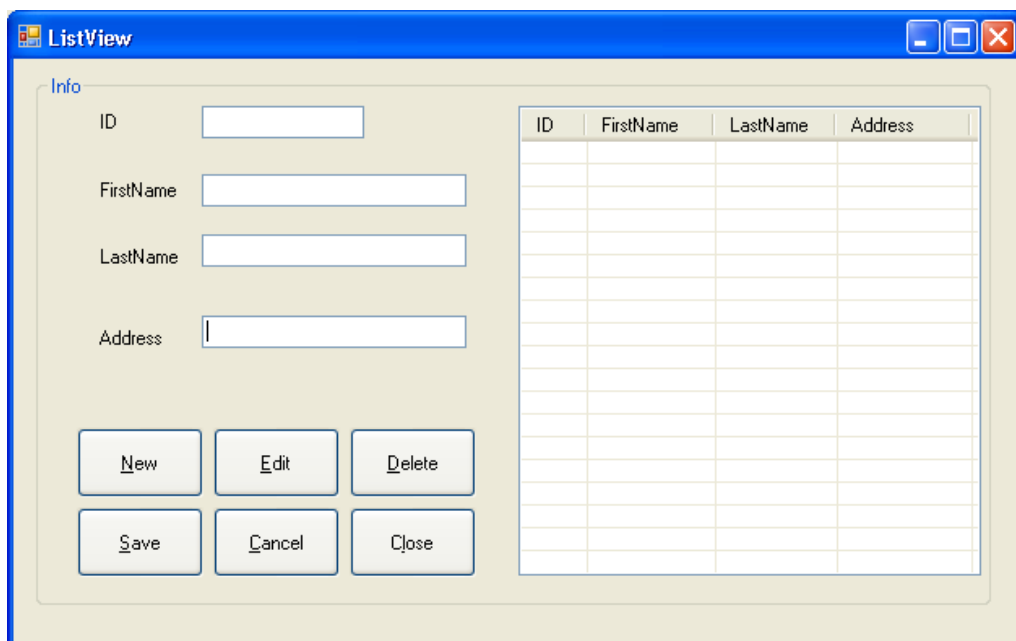
Phương thức:

Phương thức	Công dụng
Add	Thêm một dòng vào ListView
Clear	Xóa tất cả các dòng của ListView
Remove	Xóa một dòng trong ListView
RemoveAt(index)	Xóa một dòng ở vị trí index

Sự kiện thường dùng: ItemSelectionChanged: Xảy ra khi chọn một dòng.

Ví dụ 1.2:

Thiết kế form nhập, sửa, xóa dữ liệu như hình minh họa.



Cách thực hiện:

Thiết kế Form và đặt tên các đối tượng

- txtId, txtFirstName,txtLastName,txtAddress.
- btnNew, btnEdit,btnDelete,btnSave,btnCancel.

Đặt thuộc tính cho ListView1

Columns: Thêm 4 cột

FullRowSelect: True

GridLine: True

MulitiSelect: False

View: Details

Xử lý sự kiện các button

```
private void btnNew_Click(object sender, EventArgs e)
{
    txtId.Clear();
    txtFirstName.Clear();
    txtLastName.Clear();
    txtAddress.Clear();
}

private void btnEdit_Click(object sender, EventArgs e)
{
    txtId.Focus();
}

private void btnCancel_Click(object sender, EventArgs e)
{
    txtId.Clear();
    txtFirstName.Clear();
    txtLastName.Clear();
    txtAddress.Clear();
}

private void btnDelete_Click(object sender, EventArgs e)
{
    try
    {
```

```

listView1.Items.RemoveAt(listView1.SelectedIndex);
}

catch(Exception)
{
}

public void listView1_ItemSelectionChanged(object sender,
ListViewItemSelectionChangedEventArgs e)
{
txtId.Text=listView1.Items[e.ItemIndex].SubItems[0].Text;
txtFirstName.Text=listView1.Items[e.ItemIndex].SubItems[1].Text;
txtLastName.Text=listView1.Items[e.ItemIndex].SubItems[2].Text;
txtAddress.Text=listView1.Items[e.ItemIndex].SubItems[3].Text;

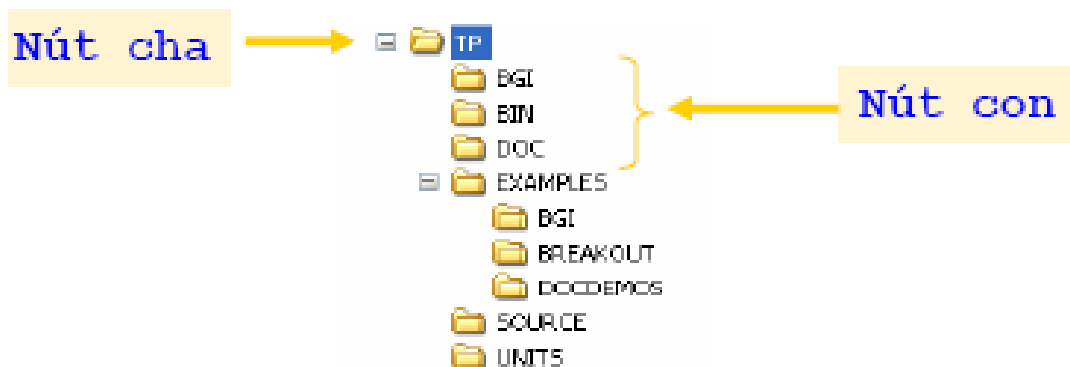
}

private void btnClose_Click(object sender, EventArgs e)
{
this.Close();
}

```

2/ TreeView

- Hiển thị thông tin theo các nút(node)
- Các nút cha có các nút con
- Nút đầu tiên gọi là nút gốc
- + Ấn dấu [+] để mở nút
- + Ấn dấu [-] để thu gọn nút
- Mỗi nút có hình ảnh kèm theo



- Thuộc tính cơ bản:

Thuộc tính	Ý nghĩa
------------	---------

CheckBoxes	Có/Không xuất hiện các checkbox trên các node. Mặc định là False
Checked	Có/Không một Node được check(Thuộc tính CheckBoxes phải được đặt là True)
ImageList	Chỉ ra danh sách ảnh hiển thị trên các node
Nodes	Mảng các TreeNodes trong TreeView Nodes.Add: Bổ sung một node vào cây Nodes.Clear: Xóa toàn bộ các node trên cây Nodes.Remove: Xóa một node trên cây và các node con của nó
SelectedNode	Node hiện thời được chọn
FullPath	Chỉ ra đường dẫn đến node bắt đầu từ node gốc
SelectedImageIndex	Chỉ ra chỉ số ảnh được hiển thị trên node khi node được chọn
Text	Text hiển thị của Node
ImageIndex	Chỉ ra chỉ số ảnh được hiển thị trên node khi node không được chọn
FirstNode	Node con đầu tiên của node
LastNode	Node con cuối cùng của node
PrevNode	Node con trước của node con hiện thời
NextNode	Node con tiếp theo node hiện thời

Phương thức:

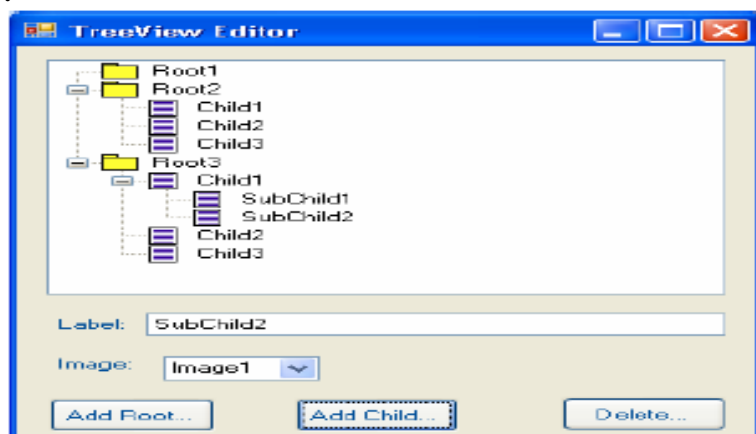
Phương thức	Công dụng
Collapse	Thu nhỏ các node con của node
Expand	Mở rộng các node con của node
ExpandAll	Mở rộng tất cả các node con
GetNodeCount	Trả về số lượng node con

Sự kiện:

AfterSelect: Xảy ra khi một node được chọn.

BeforeExpand: Xảy ra khi mở rộng một node.

Ví dụ 1.3:



```

private void AddTreeView_Load(object sender, EventArgs e)
{
    treeView1.ImageList=imageList1;
    comboBox1.Items.Add("Image1");
    comboBox1.Items.Add("Image2");
    comboBox1.SelectedIndex=0;
}
private void btnAddRoot_Click(object sender, EventArgs e)
{
    if(txtInput.Text.Trim()=="") return ;
    TreeNode childNode=new TreeNode();
    childNode.Text=txtInput.Text;
    childNode.ImageIndex=comboBox1.SelectedIndex;
    treeView1.Nodes.Add(childNode);
}
private void treeView1_AfterSelect(object sender, TreeViewEventArgs e)
{
    currentNode=e.Node;
}
private void btnAddchild_Click(object sender, EventArgs e)
{
    if(txtInput.Text.Trim()=="") return;
    TreeNode childNode=new TreeNode();
    childNode.Text=txtInput.Text;
    childNode.ImageIndex=comboBox1.SelectedIndex;
    currentNode.Nodes.Add(childNode);
    currentNode.ExpandAll();
}
private void btnAddDelete_Click(object sender, EventArgs e)
{
    currentNode.Remove();
}

```

2. Các hộp thoại

2.1. Hộp thoại thông điệp

Lớp MessageBox dùng để hiển thị hộp thoại với tiêu đề, các loại nút và nhiều biểu tượng khác nhau.

Sử dụng phương thức show để trình bày hộp thoại. Không thể tạo đối tượng MessageBox.

Các loại MessageBox:

a. MessageBoxButtons

Giá trị	Ý nghĩa
AbortRetryIgnore	Hộp thoại chứa đựng ba nút Abort, Retry, và Ignore.
OK	Hộp thoại chứa đựng một nút OK
OKCancel	Hộp thoại chứa đựng hai nút OK và Cancel
RetryCancel	Hộp thoại chứa đựng hai nút Retry, Cancel
YesNo	Hộp thoại chứa đựng hai nút Yes và No
YesNoCancel	Hộp thoại chứa đựng ba nút Yes, No và Cancel

b. MessageBoxIcons

Giá trị	Ý nghĩa
Error	Hộp thoại chứa đựng biểu tượng vòng tròn có nền màu đỏ với ký tự X màu trắng bên trong
Exclamation	Hộp thoại chứa đựng biểu tượng hình tam giác có nền màu đỏ với ký tự ! màu vàng bên trong
Hand	Hộp thoại chứa đựng biểu tượng vòng tròn có nền màu đỏ với ký tự X hoa màu trắng bên trong
Information	Hộp thoại chứa đựng biểu tượng vòng tròn có ký tự I thường bên trong
None	Hộp thoại chứa đựng biểu tượng, đây là giá trị mặc định.
Question	Hộp thoại chứa đựng biểu tượng vòng tròn có nền màu xanh với dấu ? màu trắng bên trong
Stop	Hộp thoại chứa đựng biểu tượng vòng tròn có nền màu đỏ với chữ X hoa màu trắng bên trong
Warning	Hộp thoại chứa đựng biểu tượng hình tam giác có nền màu đỏ với ký tự ! màu vàng bên trong

c. MessageBoxDefaultButton

Giá trị	Ý nghĩa
Button1	Hộp thoại sẽ được chấp nhận ứng với nút thứ nhất được chọn mặc định nếu người sử dụng nhấn phím Enter
Button2	Hộp thoại sẽ được chấp nhận ứng với nút thứ hai được chọn mặc định nếu người sử dụng nhấn phím Enter
Button3	Hộp thoại sẽ được chấp nhận ứng với nút thứ ba được chọn mặc định nếu người sử dụng nhấn phím Enter

d. DialogResult

Giá trị	Ý nghĩa
OK	Nếu nút Ok của hộp thoại xuất hiện trên màn hình được chọn
Cancel	Nếu nút Cancel của hộp thoại xuất hiện trên màn hình được chọn
Yes	Nếu nút Yes của hộp thoại xuất hiện trên màn hình được chọn
No	Nếu nút No của hộp thoại xuất hiện trên màn hình được chọn
None	Nếu nút None của hộp thoại xuất hiện trên màn hình được chọn
Ignore	Nếu nút Ignore của hộp thoại xuất hiện trên màn hình được chọn
Retry	Nếu nút Yes của hộp thoại xuất hiện trên màn hình được chọn
Abort	Nếu nút Yes của hộp thoại xuất hiện trên màn hình được chọn

Các phương thức show của MessageBox

Show(String): Trình bày hộp thoại với nội dung là chuỗi và tiêu đề rỗng.

Ví dụ: `MessageBox.Show("Bạn chưa nhập đủ dữ liệu");`

Show(IW,String): Nhận hai tham số là `Iwin32Window` và chuỗi trình bày hộp thoại với nội dung là chuỗi và tiêu đề rỗng.

Ví dụ: `Form frm=new Form();`

`frm.Show();`

`MessageBox.Show(frm, msg);` //msg là nội dung của chuỗi trình bày

Show(String, String): Trình bày hộp thoại với nội dung là chuỗi và tiêu đề

Ví dụ: `MessageBox.Show("Tieu de", "Noi Dung");`

Show(IW,String,String): Trình bày hộp thoại với nội dung là chuỗi và tiêu đề

Ví dụ:

```
Form frm=new Form();
```

```
frm.Show();
```

```
MessageBox.Show(frm,msg,title);
```

Show(String,String, MBB): Trình bày hộp thoại với nội dung là chuỗi và tiêu đề và hằng `MessageBoxButtons`.

Ví dụ:

```
MessageBox.Show(msg,title, MessageBoxButtons.RetryCancel);
```

Show(IW,String,String,MBB): Trình bày hộp thoại với nội dung là chuỗi và tiêu đề và hằng số `MessageBoxButtons` với `Iwin32Window`

Ví dụ:

```
Form frm=new Form();
```

```
frm.Show();
```

```
MessageBox.Show(frm,msg,title,MessageBoxButtons.YesNoCancel);
```

Show(String,String,MBB,MBI): Trình bày hộp thoại với nội dung là chuỗi và tiêu đề, hằng số `MessageBoxButtons` và `MessageBoxIcon`.

Ví dụ:

```
MessageBox.Show(msg,title,MessageBoxButtons.AbortRertyIgnore,MessageBoxIcon.Warning);
```

Show(IW,String,String,MBB,MBI):

Ví dụ:

```
Form frm=new Form();
```

```
frm.Show();
```

```
MessageBox.Show(frm,msg,title,
```

```
MessageBoxButtons.AbortRertyIgnore,MessageBoxIcon.Warning);
```

2.2. Các hộp thoại tập tin

Trên thực tế có rất nhiều ứng dụng của người dùng điều hướng hệ thống tập tin để mở và dùng các tập tin dữ liệu hệ thống.

Có hai thao tác chính mà người dùng thường sử dụng trên tập tin đó là mở tập tin và lưu tập tin. Cả hai thao tác này đều yêu cầu người dùng phải chỉ ra đường dẫn tập tin và loại tập tin.

`System.IO` cung cấp cho chúng ta 4 lớp (`Directory`, `File`, `DirectoryInfo`,

FileInfo) cho phép thao tác với tập tin riêng lẻ cũng như tương tác với cấu trúc thư mục của máy. Hai lớp đầu Directory và File cho phép những thao tác tạo, gỡ bỏ và thao tác khác nhau trên các tập tin và thư mục. Hai lớp có liên hệ mật thiết DirectoryInfo và FileInfo cũng có những chức năng tương tự nhưng các thành viên không phải là static, nên muốn triệu gọi các thành viên trước tiên bạn phải tạo một thể hiện đối tượng lớp. Hai lớp đầu Directory và File dẫn xuất từ lớp System.Object trong khi DirectoryInfo và FileInfo dẫn xuất từ lớp FileSystemInfo. Lớp FileSystemInfo là lớp cơ bản abstract có một số thuộc tính và phương thức cung cấp những thông tin liên quan đến một tập tin và thư mục.

Các thuộc tính của lớp cơ bản FileSystemInfo:

Thuộc tính	Ý Nghĩa
CreationTime	Thời gian file, folder được tạo
DirectoryName (FileInfo), Parent (DirectoryInfo)	Tên đường dẫn của folder chức dụng
Exists	Xác định file ,folder hiện hữu
Extension	Tên mở rộng của file, trả về khoảng trắng nếu là folder
FullName	Tên đường dẫn của file ,folder
LastAccessTime	Thời gian file, folder truy xuất lần cuối
LastWriteTime	Time file or folder was last modified
Name	Name of the file or folder
Root	Đường dẫn gốc
Length	Kích thước file tính bằng bytes (chỉ FileInfo)

Các phương thức chúng ta có thể thực hiện như sau:

Name	Purpose
Create()	Tạo một folder hoặc một file rỗng
Delete()	Hủy file, folder
MoveTo()	Di chuyển hoặc sửa tên file, folder.
CopyTo()	(FileInfo only) Sao chép file, không sao chép phương thức cho folders.
GetDirectories()	(DirectoryInfo only) Trả về một mảng các đối tượng của DirectoryInfo đại diện tất cả folders được chứa trong folder này.
GetFiles()	(DirectoryInfo only) Trả về một mảng các đối tượng của FileInfo đại diện tất cả folders được chứa trong folder này
GetFileSystemObjects()	(DirectoryInfo only) Trả về đối tượng FileInfo và DirectoryInfo như mảng của tham khảo FileSystemInfo .

* Hộp thoại lưu tập tin

Công dụng: Tạo ra hộp thoại Save File.

Tạo SaveFileDialog: Nhấp đúp vào control  SaveFileDialog

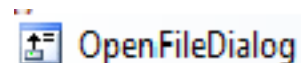
Thuộc tính:

Thuộc tính	Ý nghĩa
AddExtension	True cho phép thêm vào tên mở rộng
CheckFileExsits	True sẽ xuất hiện hộp thoại với chuỗi cảnh báo nếu người sử dụng chọn tập tin không tồn tại
CheckPathExits	True sẽ kiểm tra đường dẫn có hợp lệ hay không trước khi trả về
CreatePrompt	Cửa sổ cảnh báo xuất hiện nếu tập tin chuẩn bị ghi ra chưa tồn tại
DefaultExt	Chọn tên mở rộng mặc định nếu người sử dụng không cung cấp tên mở rộng cho tập tin.
FileName	Gán hay trả về tên tập tin chọn
Filter	Khai báo chuỗi lọc các loại tập tin
InitialDirectory	Thư mục sẽ chọn khi hộp thoại kích hoạt. Nếu không cung cấp giá trị cho thuộc tính này thì ổ đĩa mặc định sẽ được chọn
Overwriteprompt	True thì cửa sổ cảnh báo khi tập tin sẽ ghi đang tồn tại
RestoreDirectory	Phục hồi lại thư mục hiện hành trước khi đóng hộp thoại
Title	Tiêu đề của hộp thoại

* Hộp thoại mở tập tin:

Công dụng: Tạo ra hộp thoại Open File.

Tạo OpenFileDialog: Nhấp đúp vào control



Thuộc tính:

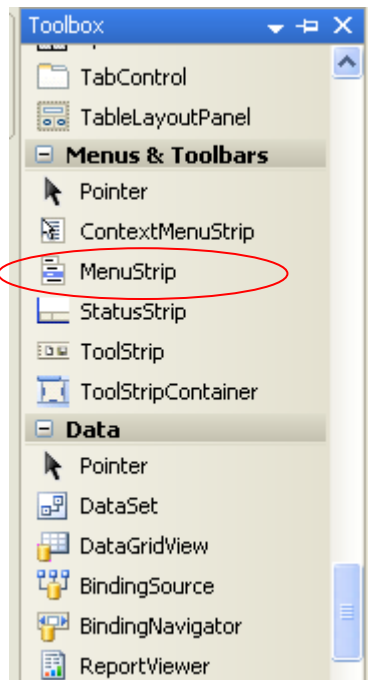
Thuộc tính	Ý nghĩa
AddExtension	True cho phép thêm vào tên mở rộng
CheckFileExsits	True sẽ xuất hiện hộp thoại với chuỗi cảnh báo nếu người sử dụng chọn tập tin không tồn tại
CheckPathExits	True sẽ kiểm tra đường dẫn có hợp lệ hay không trước khi trả về
DefaultExt	Chọn tên mở rộng mặc định nếu người sử dụng không cung cấp tên mở rộng cho tập tin.
FileName	Gán hay trả về tên tập tin chọn
FileNames	Gán hay trả về mảng chứa tên tập tin chọn
Filter	Khai báo chuỗi lọc các loại tập tin.
InitialDirectory	Thư mục sẽ chọn khi hộp thoại kích hoạt. Nếu không cung cấp thuộc tính này thì ổ đĩa mặc định sẽ được chọn

MultiSelect	True cho phép chúng ta chọn nhiều tập tin
ReadOnlyChecked	True là Check vào nút Check có chuỗi open as read-only
RestoreDirectory	Phục hồi lại thư mục hiện hành trước khi đóng hộp thoại
ShowReadOnly	True trình bày nút Check có chuỗi Open as read-only
Title	Tiêu đề hộp thoại

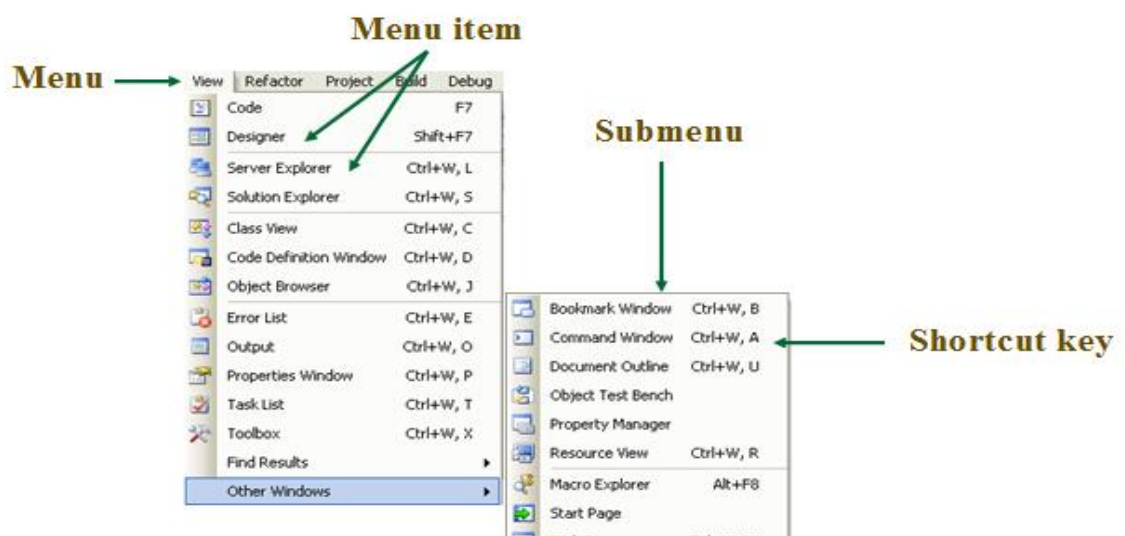
3. Thực đơn

a. Thực đơn chính

Menu xuất hiện tại Menu Bar được khởi tạo bằng cách kéo thả control MenuStrip từ ToolBox vào Form.



Khi ấy Menu xuất hiện cho phép chúng ta khởi tạo danh mục menu như hình



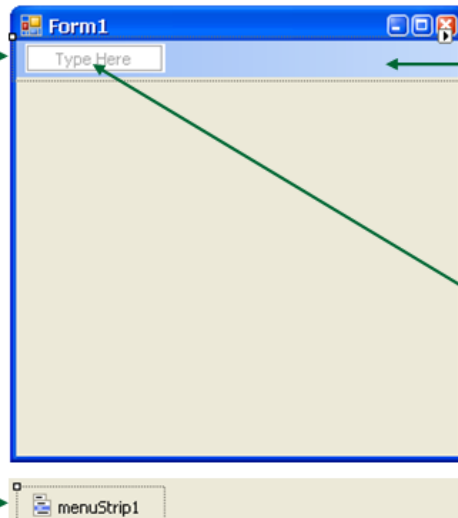
Cách tạo menu

Trong ToolBox kéo control MenuStrip thả vào form

Thanh menuBar xuất hiện trên cùng của form

Trong màn hình design dễ dàng tạo lập các menu item

Nhập tên menu

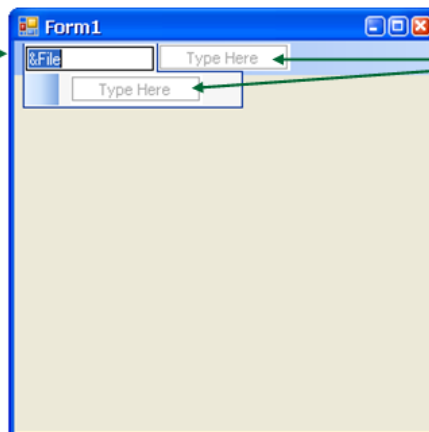


MenuBar

ToolStripMenuItem

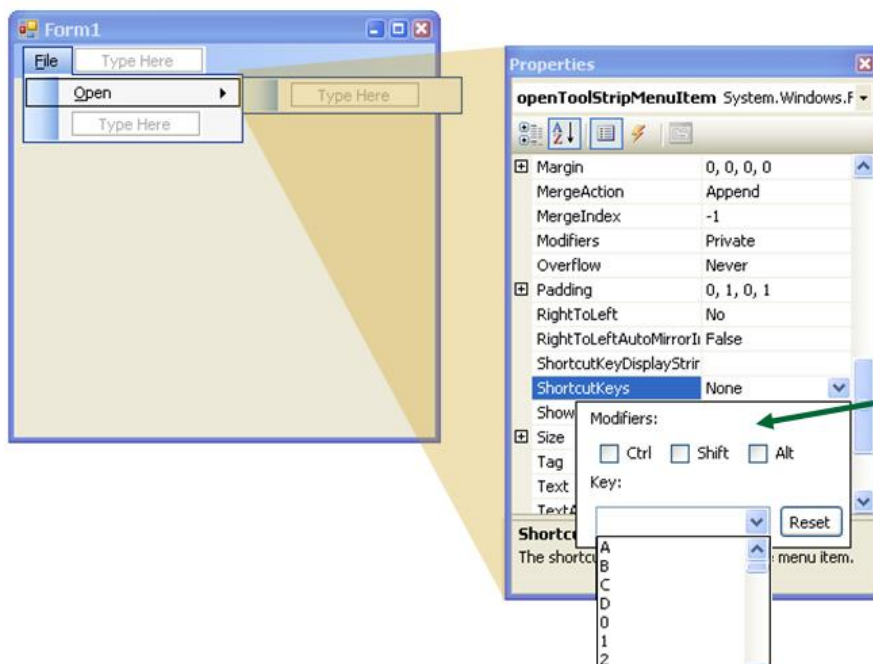
MenuStrip

Đặt ký tự & trước ký tự làm phím tắt



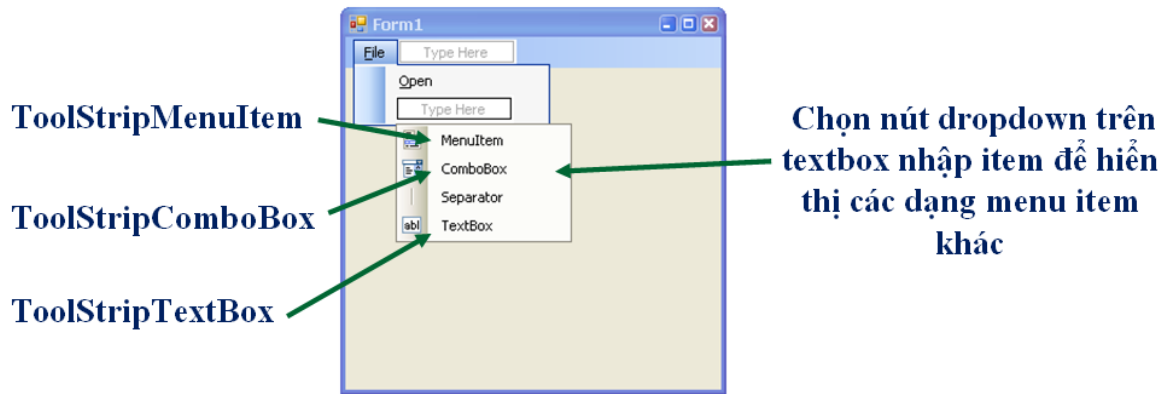
Tạo menu item mới bằng cách nhập vào textbox

Thiết lập shortcut key cho menu item



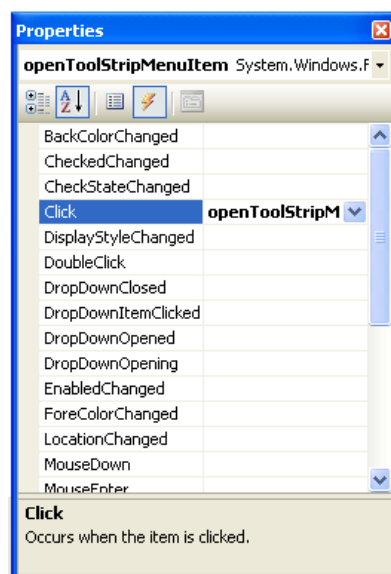
Khai báo shortcut key

Menu item có thể là TextBox hoặc ComboBox



Thuộc tính của MenuStrip, ToolStripMenuItem	
MenuStrip	
Items	Chứa những top menu item
MdiWindowListItem	Chọn top menu item hiển thị tên các cửa sổ con
ToolStripMenuItem	
Checked	Xác định trạng thái check của menu item
Index	Chỉ mục menu item trong menu cha
DropDownItems	Chứa những menu item con
ShortcutKeys	Phím tắt
Text	Tiêu đề menu item
ShowShortcutKeys	Xác định trạng thái hiển thị phím tắt bên cạnh menu item

Sự kiện cho Menu Item thường dùng là Click



b. Các sự kiện popup và chia sẻ

Sự kiện Popup: Xảy ra khi một nhãn menu được mở ra.

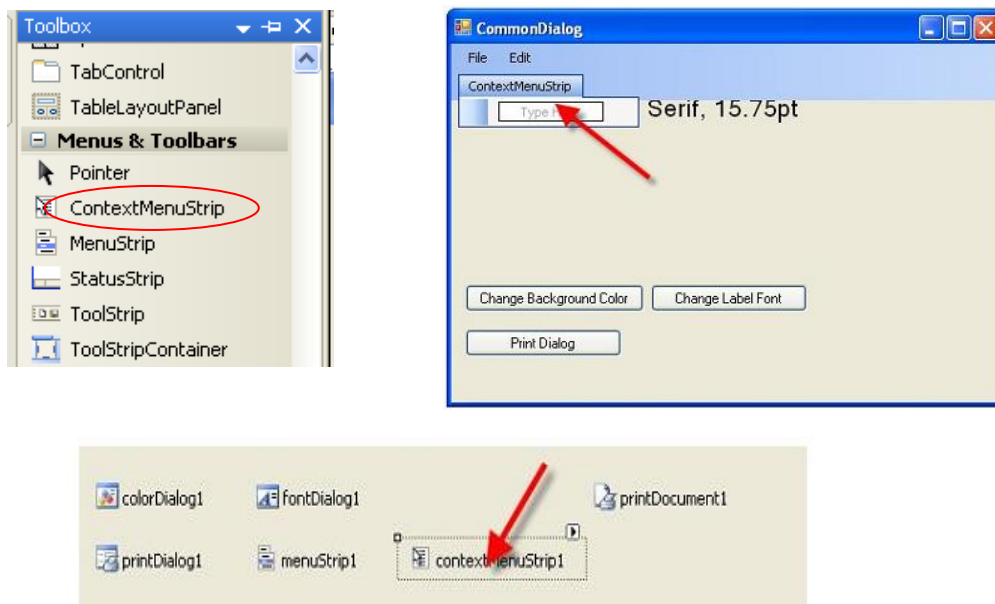
```
private void cmnuContext_Popup(object sender, EventArgs e)
{
    MessageBox.Show(cmnuContext.GetContextMenu().ToString());
}
```

Sự kiện Click: Xảy ra khi một nhãn được click chuột trái.

```
private void openToolStripMenuItem_Click(object sender, EventArgs e)
{
    MessageBox.Show("Ban chon Menu File");
}
```

c. Thực đơn ngữ cảnh

Thực đơn ngữ cảnh là loại thực đơn gắn với một điều khiển cụ thể nào đó. Khi người dùng nhấn chuột phải vào điều khiển có gắn thực đơn ngữ cảnh thì thực đơn ngữ cảnh của điều khiển đó sẽ hiện ra và cho phép người dùng chọn công việc mong muốn từ thực đơn.



Tạo đối tượng ContextMenuStrip:

- Cách 1: Di chuyển điều khiển ContextMenuStrip từ Toolbox vào nơi chứa điều khiển trên Form
- Cách 2: Chọn điều khiển ContextMenuStrip trên Toolbox và vẽ điều khiển trên Form
- Cách 3: Chọn điều khiển ContextMenuStrip trên Toolbox và double-click trên Form
- Cách 4: Double-click điều khiển ContextMenuStrip trên Toolbox.

⚠ Lưu ý: Đối tượng ContextMenuStrip xuất hiện trên cùng của Form. Nhưng khi thực thi thì không xuất hiện. Chỉ xuất hiện khi được gọi.

Thuộc tính Name: Thiết lập tên điều khiển và tiếp đầu ngữ tên là cmn.

Sự kiện Opening: Xảy ra khi mở điều khiển ContextMenuStrip.

Tạo các phần tử ToolStripMenuItem

Tạo ToolStripMenuItem: Kích vào ContextMenuStrip để soạn thảo các ToolStripMenuItem.

Thuộc tính Name: Thiết lập tên điều khiển và tiếp đầu ngữ là cmnitem.

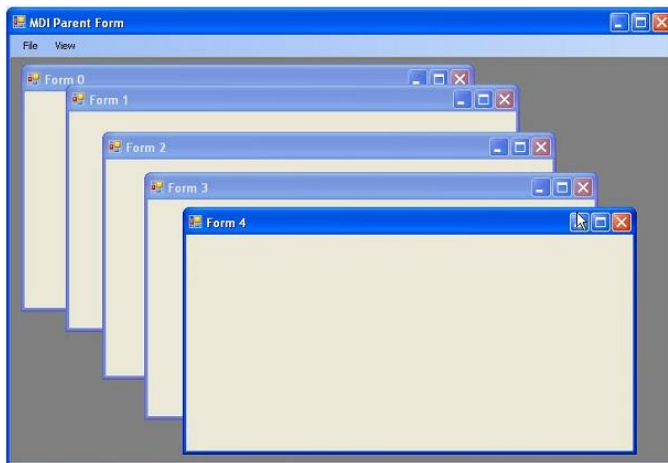
Tạo phân cách các ToolStripMenuItem: Chọn vị trí cần tạo, Click Phải chọn Insert->Chọn Separator

4. Các ứng dụng MDI

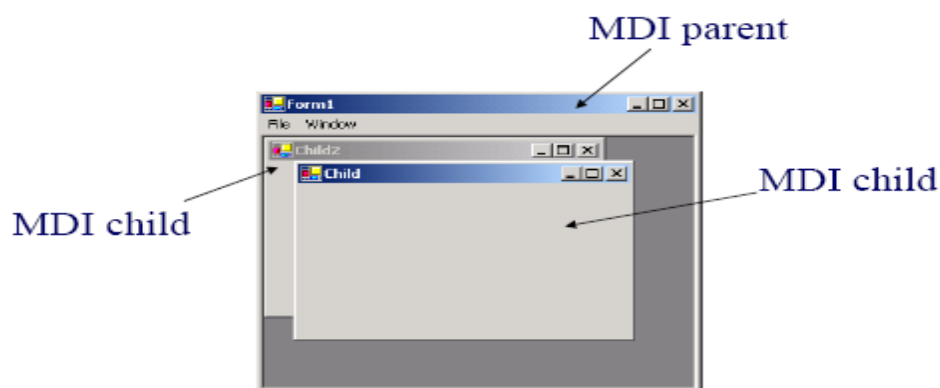
Ứng dụng Multiple Document Interface (MDI) là ứng dụng cho phép hiển thị nhiều cửa sổ ứng dụng tại cùng thời điểm. Một ví dụ về ứng dụng MDI đó là chương trình Microsoft Word, chúng ta có thể tạo cùng lúc nhiều cửa sổ soạn thảo trên cùng một cửa sổ chung của ứng dụng.

Một ứng dụng MDI có thể có một hoặc nhiều form MDI cha, và mỗi form MDI cha này có thể chứa nhiều form MDI con. Một MDI form cha có thể chứa nhiều MDI form con, nhưng một MDI form con không thể thuộc về nhiều MDI form cha.

MDI cha có thể được khởi tạo tại thời điểm thiết kế ứng dụng hoặc khi chương trình chạy bằng cách gán thuộc tính **IsMdiContainer** của Form là **true**.



Ví dụ 6.1:



Thuộc tính MDI Parent của Form:

IsMdiContainer: Có /Không một Form là Form MDI Parent. Mặc định là False.

ActiveMdiChild: Trả về Form Child đang được kích hoạt

Thuộc tính MDI Child:

IsMdiChild: Có/Không Form là một MDI child

MdiParent: Chỉ ra một MDI parent của Form

<tên form>.MdiParent=this

Bài tập:

Bài 1:

- Tạo một Project Windows Forms. Mở Form tên Form1 mặc định ở dạng Designer
- Mở Properties của Form
- Thiết lập lại một số giá trị Properties của Form như sau:

Property	Value
Text	Trey Seseach
FormBorderStyle	Fixed3D
StartPosition	Manual
Location	100,200
Opacity	75%

Từ ToolBox. Tạo 3 button với Text lần lượt là Border Sytle, Resize, Opacity



Bài 2:

- Tạo một Project Windows Forms. Mở Form tên Form1 mặc định ở dạng Designer.
- Mở Properties của Form
- Thiết lập lại một số giá trị Properties của Form như sau:

Property	Value
----------	-------

FormBorderStyle	None
Back-Color	Red

- Từ Toolbox tạo Button ở bên trái phía dưới form. Đặt Text là Close Form
- Double – click vào Close Form button và cài đặt code cho Button1_Click như sau: `this.Close();`

Bài 3:

Tạo Form đặt tên frmBai3 và thiết lập Form với các thuộc tính sau:

- Màu nền: Màu xanh dương
- Độ mờ của Form là 85%
- Thể hiện của Form khi xuất hiện là đầy màn hình.
- Nội dung Text là Bai 3
- Tạo icon cho Form tùy ý

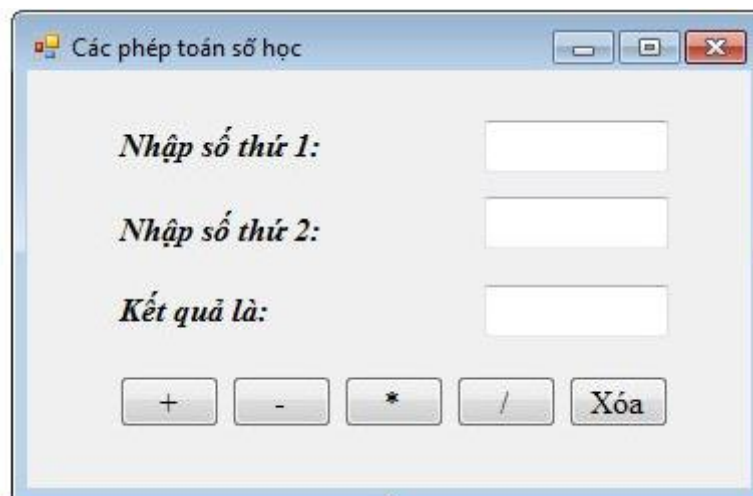
Bài 4:

Tạo Form đặt tên là frmBai4 và thiết lập Form với các thuộc tính sau:

- Vị trí bắt đầu form là giữa màn hình
- Nội dung Text là Bài 4
- Thiết lập con trỏ chuột là Hand
- Loại Form là FixedSingle
- Độ mờ Form là 30 %

Bài 5 :

- Thiết kế form như mẫu.



(txtsSo1, txtSo2, txtKQ, btnCong, btnTru, btnNhan, btnChia, btnXoa)

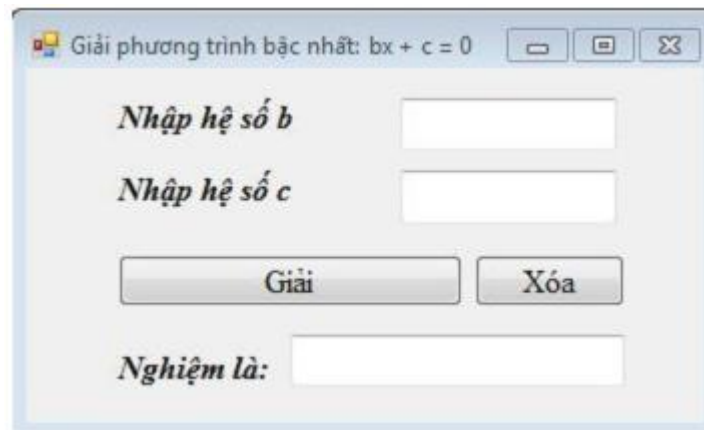
- Viết chương trình làm các phép toán: cộng, trừ, nhân, chia cho các

button btnCong, btnTru, btnNhan, btnChia.

- Xóa: Xóa trắng các TextBox.
- Thêm vào button Dừng (btnDung), khi nhấp vào btnDung thì dừng chương trình.
- Cải tiến:

Khi nhấp button btnChia, nếu txtSo2 là 0 thì xuất hiện hộp thông báo lỗi rồi xóa trống txtSo2 và di chuyển con trỏ đến TextBox này.

Bài 6: Viết chương trình giải phương trình bậc nhất: $bx + c = 0$



(txtB, txtC, btnGiai, txtKQ)

- Thêm vào button Dừng, khi nhấp vào button này thì dừng chương trình.

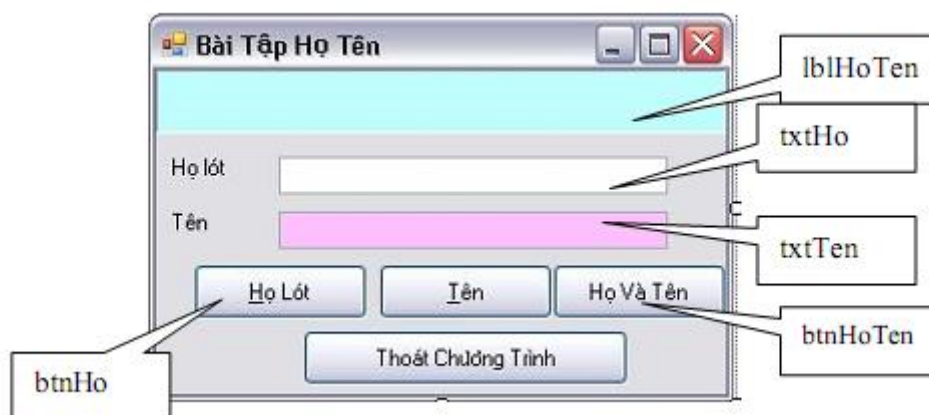
Bài 7: Thiết kế form như sau:

Yêu cầu: Viết lệnh để chương trình hoạt động như sau:

Nhấn vào Button btnHo thì gán nội dung của txtHo cho lblHoTen

Hướng dẫn: lblHoTen.Text = txtHo.Text

Nhấn vào Button cmdTen thì gán nội dung của txtTen cho lblHoTen



Hướng dẫn: lblHoTen.Text = txtTen.Text

Nhấn vào Button btnHoTen thì gán nội dung của txtHo cộng với txtTen cho lblHoTen

Hướng dẫn: `lblHoTen.Text = txtHo.Text + " " + txtTen.Text`

Nhấn Double click vào `lblHoTen` thì nội dung của `lblHoTen` bị xóa

Hướng dẫn: Vào sự kiện Double Click gõ:

`lblHoTen.Text = ""` // gán bằng rỗng => Xóa

Nhấn vào `btnKetThuc` thì đóng chương trình

Hướng dẫn: Vào sự kiện Click gõ:

`This.Close();`

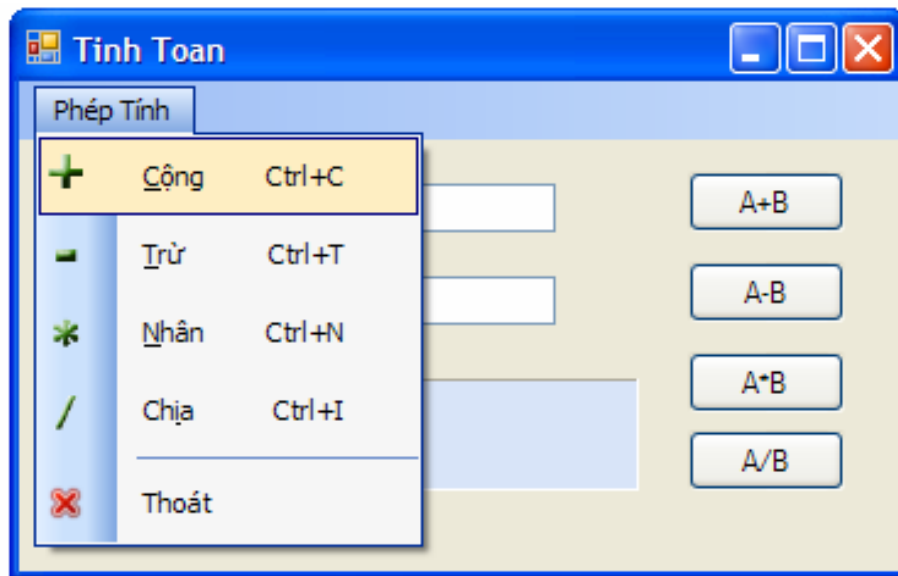
Bài 8: Thiết kế form theo mẫu sau

Yêu cầu:

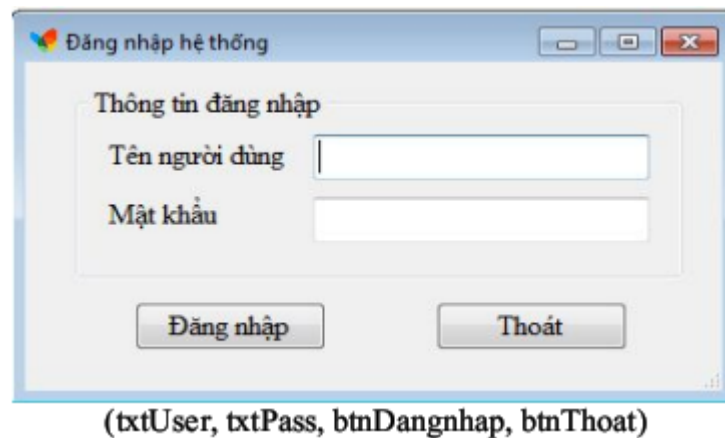
- Khi click vào nút tính tiền sẽ tính tiền theo các dịch vụ đã chọn. Nếu chưa nhập họ tên mà tính tiền sẽ xuất bản hộp thoại thông báo sau:

Bài 9: Thiết kế chương trình như hình minh họa sau:

- (1) ToolStripMenuItem Cộng hoặc Ctrl+C thực hiện phép cộng
- (2) ToolStripMenuItem Trừ hoặc Ctrl+T thực hiện phép trừ
- (3) ToolStripMenuItem Nhân hoặc Ctrl+N thực hiện phép nhân
- (4) ToolStripMenuItem Chia hoặc Ctrl+I thực hiện phép chia
- (5) ToolStripMenuItem Thoát khỏi chương trình



Bài 10: Xây dựng một ứng dụng như sau:



(txtUser, txtPass, btnDangnhap, btnThoat)

Yêu cầu:

- Khi chạy chương trình thì Form2 (Đăng nhập hệ thống) được mở lên trước.

- Nhập:

+ Tên người dùng vào TextBox txtUser

+ Mật khẩu vào TextBox txtPass

- Nhấp button Đăng nhập thì thực hiện kiểm tra

+ Nếu txtUser = "teonv" và txtPass = "123" thì chuyển sang Form1.

+ Ngược lại thì thông báo "Không đúng tên người dùng / mật khẩu !!!"

- Nhấp button Thoát thì hiển thị thông báo "Chắc không? "

+ Nếu chọn Yes thì kết thúc chương trình.

+ Ngược lại thì trở lại màn hình Đăng nhập hệ thống.

Bài 11: Khởi tạo một ứng dụng Windows Forms Application

- Form1: được thiết kế như sau



- Form2: được thiết kế như sau



(txtUser, txtPass, btnDangnhap, btnThoat)

- Khi chạy chương trình thì Form2 (Đăng nhập hệ thống) được mở lên trước.

- Nhập:

+ Tên người dùng vào TextBox txtUser

+ Mật khẩu vào TextBox txtPass

- Nhấp button Đăng nhập thì thực hiện kiểm tra

+ Nếu txtUser = "teonv" và txtPass = "123" thì chuyển sang Form1.

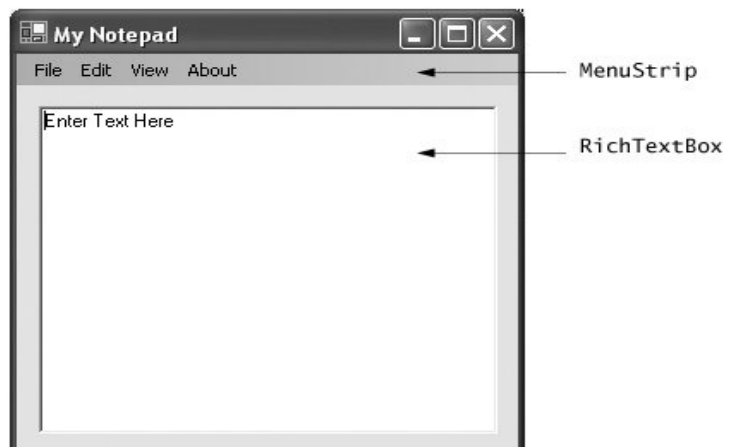
+ Ngược lại thì thông báo "Không đúng tên người dùng / mật khẩu !!!"

- Nhấp button Thoát thì hiển thị thông báo "Chắc không? "

+ Nếu chọn Yes thì kết thúc chương trình.

+ Ngược lại thì trở lại màn hình Đăng nhập hệ thống.

Bài 12: Tạo một giao diện cho ứng dụng soạn thảo mô phỏng ứng dụng Microsoft Word sử dụng MDI Form bao gồm các Menu: File, Edit, View, Insert, Format, Window, Help



BÀI 3: LƯU TRỮ VÀ QUẢN LÝ DỮ LIỆU

Giới thiệu:

Lưu trữ và quản lý dữ liệu là một nội dung quan trọng trong quá trình phát triển phần mềm, giúp chương trình có khả năng lưu giữ, cập nhật và xử lý thông tin một cách hiệu quả. Trong lập trình C#, dữ liệu có thể được lưu trữ dưới nhiều hình thức như mảng, danh sách, tập tin hoặc cơ sở dữ liệu nhằm phục vụ cho việc quản lý và khai thác thông tin trong các ứng dụng thực tế.

Thông qua nội dung này, người học sẽ hiểu được cách tổ chức dữ liệu, thao tác thêm, sửa, xóa và tìm kiếm dữ liệu trong chương trình. Đồng thời, sinh viên được rèn luyện kỹ năng xử lý dữ liệu bằng các cấu trúc phù hợp, góp phần xây dựng các ứng dụng quản lý có tính thực tiễn và hiệu quả cao.

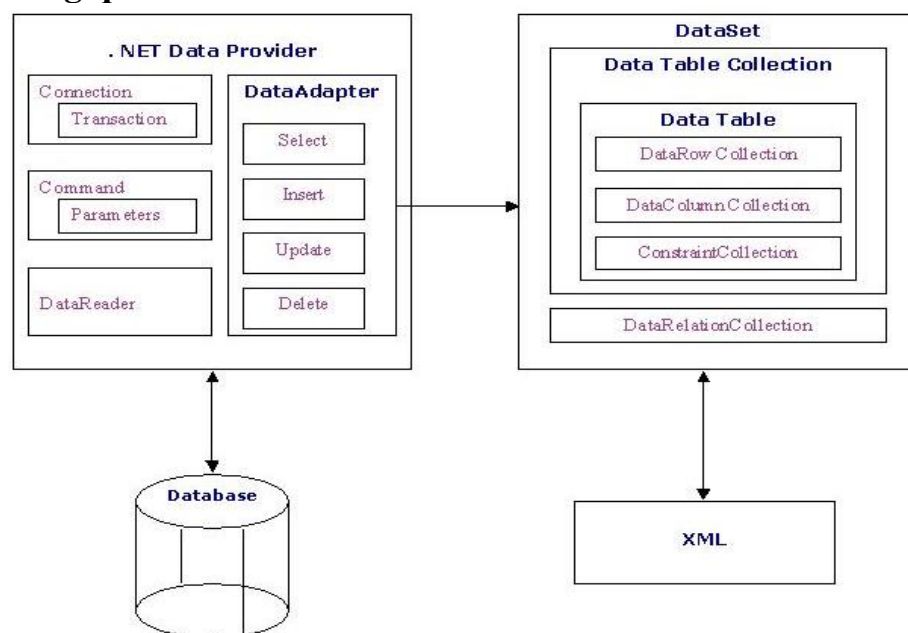
Mục tiêu:

Học xong bài này, sinh viên có khả năng:

- Trình bày được về ADO.NET.
- Trình bày được công cụ tạo báo cáo đi theo Visual Studio.NET.
- Vận dụng được các đối tượng kết nối cơ sở dữ liệu.
- Vận dụng được các câu lệnh SQL cũng như các thủ tục của SQL để thao tác dữ liệu.
- Tạo được công cụ báo cáo.
- Nghiêm túc trong học tập và thực hiện tốt các yêu cầu được giao.
- Luôn động não suy nghĩ, thường xuyên luyện tập tư duy trong việc học.

Nội dung chính:

1. Tổng quan về ADO.NET



ADO .NET Data Architecture

Ngày nay, những ứng dụng thương mại cần thiết để quản lý khối lượng dữ liệu khổng lồ. Dữ liệu thường được lưu trữ trong những bảng quan hệ trong cơ sở dữ liệu. Nhận dữ liệu và thao tác dữ liệu trực tiếp từ một cơ sở dữ liệu đòi hỏi kiến thức về những câu lệnh thao tác trên cơ sở dữ liệu để truy cập đến dữ liệu. Và ứng dụng cần giao tiếp với Cơ sở dữ liệu để thực hiện những công việc sau:

Nhận dữ liệu được lưu trữ trong cơ sở dữ liệu và hiển thị chúng ra giao diện người dùng. Cập nhật dữ liệu, thực hiện thêm, hiệu chỉnh, và xóa dữ liệu.

ADO.NET là một mô hình được những ứng dụng .NET sử dụng để giao tiếp với cơ sở dữ liệu cho việc nhận, truy cập, và cập nhật dữ liệu.

Data Provider

Một data provider được sử dụng cho việc kết nối đến cơ sở dữ liệu, nhận, lưu trữ dữ liệu trong dataset, đọc, nhận và cập nhật dữ liệu trong cơ sở dữ liệu.

Có hai loại data provider:

- OleDb data provider - loại này làm việc với tất cả OleDb Provider như Sql OleDb Provider, Oracle OleDb provider, và Jet OleDb Provider. Được biết đến trong môi trường .NET với namespace System.Data.OleDb.

- Sql Server data provider - loại này làm việc chỉ với Microsoft SQL Server. Cho phép thao tác với tốc độ tối ưu với hệ cơ sở dữ liệu MSSQL Server, được biết đến với namespace System.Data.SqlClient.

2. Các công cụ kết nối dữ liệu

Kết nối là một component được sử dụng để thiết lập một kết nối đến cơ sở dữ liệu từ một data source. Có hai kiểu đối tượng kết nối thường sử dụng nhất là OleDbConnection và SqlConnection. Bảng sau đây hiển thị những thuộc tính và phương thức hay sử dụng nhất của một đối tượng kết nối.

Tên	Ý nghĩa
ConnectionString	Cung cấp thông tin như datasource, tên cơ sở dữ liệu, được sử dụng để thiết lập kết nối với một cơ sở dữ liệu
Open()	Mở một kết nối với datasource được khai báo tại ConnectionString
Close()	Được sử dụng để đóng kết nối với data source
State	Được sử dụng để kiểm tra trạng thái của một kết nối : 0:kết nối đang đóng, 1: kết nối đang mở.

Ví dụ về việc sử dụng đối tượng kết nối

```
string connectionString="PROVIDER=SQLOLEDB;
```

```
server=(local):uid=netpwd=;database=NET";
```

```
OleDbConnction conObj=new OleDbConnection(connectionString);
```

```
conObj.Open();
```

Data Adapter

Data Adapter là thành phần của ADO.NET, có tác dụng chuyển tiếp dữ liệu từ và đến cơ sở dữ liệu. Data Adapter nhận dữ liệu từ cơ sở dữ liệu vào một DataSet. Khi chúng ta thay đổi dữ liệu trong dataset, cũng là thay đổi trong cơ sở dữ liệu bởi dataadapter. Có hai kiểu data adapter thường dùng để cấu hình kết nối đến cơ sở dữ liệu trong Visual Studio .NET:

- SqlDataAdapter: Làm việc chỉ với hệ cơ sở dữ liệu MS SQL Server
- OleDbDataAdapter: Kiểu này được cấu hình để làm việc với hầu hết các hệ cơ sở dữ liệu được hỗ trợ bởi OleDb data provider.

Data Adapter sử dụng đối tượng kết nối OleDbConnection và SqlConnection được cung cấp bởi data provider để giao tiếp với cơ sở dữ liệu.

Thuộc tính và phương thức của Data Adapter

Data Adapter giao tiếp với cơ sở dữ liệu trong khi nhận, thêm mới, xóa và cập nhật dữ liệu. Những thuộc tính sau đây được thiết lập để thực hiện những tác vụ khác nhau trên một hệ cơ sở dữ liệu

- SelectCommand: Nhận dữ liệu từ CSDL thông qua một câu truy vấn hoặc stored procedures
- InsertCommand: Thêm dữ liệu vào CSDL qua câu lệnh insert.
- UpdateCommand: Cập nhật cơ sở dữ liệu với câu lệnh update
- DeleteCommand: Xóa dữ liệu khỏi cơ sở dữ liệu thông qua câu lệnh delete
- Fill(): Là phương thức đẩy dữ liệu từ CSDL vào một dataset.
- Update(): Là phương thức thực thi InsertCommand, Update Command, hoặc DeleteCommand cho mỗi câu lệnh thêm, hiệu chỉnh hoặc xóa dòng để thực hiện thay đổi trong CSDL.

Data Command

DataCommand là đối tượng thực thi những câu lệnh SQL hoặc stored procedure được sử dụng để thao tác với CSDL. Data Command là đối tượng của lớp SqlCommand và OleDbCommand.

Ví dụ sử dụng Data Command

```
string connectionstring="PROVIDER=SQLOLEDB;
server=(local):uid=netpwd=;database=NET";
OleDbConnction conObj=new OleDbConnection(connectionstring);
conObj.Open();
OleDbCommand cmd=new OleDbConnection("delete from books where
ibookid='003'",conObj);
int result=cmd.ExecuteNonQuery();
conObj.Close();
```

Data Reader

Data reader được sử dụng để chỉ nhận dữ liệu từ một data source. Sử dụng data reader nhận được kết quả nhanh hơn và không tốn bộ nhớ tại mọi thời điểm vì chỉ một hàng dữ liệu thực sự được lưu trữ trong bộ nhớ.

Các thuộc tính của đối tượng DataReader:

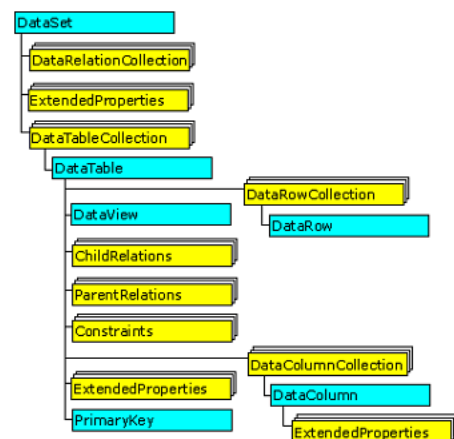
Tên	Ý nghĩa
Read()	Phương thức được sử dụng để đọc một dòng
Close()	Được sử dụng để đóng đối tượng DataReader.
NextResult	Được sử dụng để di chuyển đến hàng dữ liệu tiếp theo trong trường hợp câu lệnh truy vấn trả lại nhiều dòng kết quả

Ví dụ 7.1: Sử dụng Data Reader

```
private void button2_Click(object sender, EventArgs e)
{
    string connectionstring="PROVIDER=SQLOLEDB;
server=(local);uid=netpwd=;database=NET";
OleDbConnction conObj=new OleDbConnection(connectionstring);
conObj.Open();
OleDbCommand cmd=new OleDbConnection("Select * from
books",conObj);
OleDbDataReader dr;
dr=cmd.ExecuteReader();
while(dr.Read())
{
    lbltestdatareader.Text=dr["BookID"].ToString();
}
conObj.Close();
}
```

DataSet : Là đối tượng nhận dữ liệu từ cơ sở dữ liệu thông qua DataAdapter, DataSet hoạt động như một cơ sở dữ liệu ảo chứa những bảng, dòng, và cột.

Mô hình DataSet như sau:



Các đối tượng thuộc tính cần quan tâm với đối tượng DataSet bao gồm: DataTableCollection, DataRelationCollection, ExtendedProperties, DataTable, DataRelation, DataRowCollection, DataView, PrimaryKey, DataColumnCollection. Chi tiết những đối tượng thuộc tính này sẽ được đề cập trong các phần sau.

Ví dụ 7.2 Sử dụng DataAdapter lấy dữ liệu từ cơ sở dữ liệu vào DataSet.

```
private void button3_Click(object sender, EventArgs e)
{
    string connectionString="PROVIDER=SQLOLEDB;
    server=(local);uid=netpwd=;database=NET";
    OleDbConnction conObj=new OleDbConnection(connectionString);
    conObj.Open();
    OleDbCommand cmd=new OleDbConnection("Select * from
    books",conObj);
    OleDbDataAdapter da=new OleDbDataAdapter(cmd);
    DataSet ds=new DataSet();
    da.Fill(ds,"books");
    conObj.Close();
}
```

Data Binding: Sau khi nhận dữ liệu từ cơ sở dữ liệu, dữ liệu cần được hiển thị trên những control của Windows Form để người sử dụng thấy. Quá trình đó gọi là Data Binding.

3. Hiển thị dữ liệu, lọc dữ liệu

a. Hiển thị dữ liệu

Data Binding

Khái niệm Data Binding :

Sau khi nhận dữ liệu từ cơ sở dữ liệu, dữ liệu cần được hiển thị trên những control của Windows Form để người sử dụng thấy. Quá trình đó gọi là Data Binding.

Bước 1: Tạo ra một đối tượng SqlConnection để kết nối tới Cơ sở dữ liệu trong phương thức Main() tạo ra một đối tượng SqlConnection đặt tên mySqlConnection để kết nối tới cơ sở dữ liệu Northwind máy chủ phục vụ SQL:

```
SqlConnection mySqlConnection = new SqlConnection(
"server=localhost;database=Northwind;uid=sa;pwd=sa");
```

Chuỗi được chuyển đến bộ khởi dựng SqlConnection được biết như connection string và chứa những phần tử sau đây:

Server: chỉ rõ tên của máy tính trên đó SQL Server đang chạy- trong ví dụ này là localhost ; localhost là một tên chung tham chiếu tới máy tính mà trên đó chương trình của chúng ta chạy. Nếu cơ sở dữ liệu của chúng ta đang chạy lưu trữ trên một máy tính khác với máy tính mà chương trình hiện thời chúng ta đang chạy, thì chúng ta sẽ cần phải thay thế localhost với tên của máy tính đó.

Database: chỉ rõ tên của cơ sở dữ liệu - trong ví dụ này là Northwind.

uid : chỉ rõ tên tài khoản của người sử dụng cơ sở dữ liệu : trong ví dụ này là sa; sa là một tài khoản của người dùng cơ sở dữ liệu chung được quản lý bởi người quản trị cơ sở dữ liệu "database administrator" (DBA). Chúng ta có thể sử dụng bất kỳ tài khoản người sử dụng cơ sở dữ liệu nào miễn là nó có quyền truy cập tới cơ sở dữ liệu Northwind.

pwd: chỉ rõ mật khẩu cho người sử dụng. Mật khẩu cho người sử dụng “sa” trong cơ sở dữ liệu của tôi cũng là sa. Chúng ta sẽ cần thay đổi pwd thành mật khẩu cho tài khoản sa của chúng ta, hay bất cứ tài khoản nào chúng ta chỉ định ở uid.

Tạo ra một đối tượng SqlCommand

Bước 2: Tạo ra một đối tượng SqlCommand đặt tên mySqlCommand được sử dụng sau đó để gửi một phát biểu SELECT cho cơ sở dữ liệu cho sự thực thi.

```
SqlCommand mySqlCommand = mySqlConnection.CreateCommand();
```

Thiết đặt thuộc tính CommandText của Đối tượng SqlCommand

Bước 3: Gán thuộc tính CommandText của mySqlCommand tạo ra trong bước trước đây tới một phát biểu SELECT. Phát biểu này sẽ truy xuất những cột CustomerID, CompanyName, ContactName, và Address từ hàng trong bảng những khách hàng có CustomerID là ALFKI:

```
mySqlCommand.CommandText =
```

```
"SELECT CustomerID, CompanyName, ContactName, Address "+
```

```
"FROM Customers "+
```

```
"WHERE CustomerID = 'ALFKI'";
```

Mở đối tượng SqlConnection

Bước 4: Mở kết nối cơ sở dữ liệu sử dụng phương thức Open() của đối tượng SqlConnection tạo ra trong Bước 1:

```
mySqlConnection.Open();
```

Một khi kết nối tới cơ sở dữ liệu được mở, chúng ta có thể gửi những lệnh cho cơ sở dữ liệu cho sự thực thi.

Chạy phát biểu SELECT

Chúng ta chạy phát biểu SELECT trước đó được gán trong mySqlCommand bởi việc gọi phương thức ExecuteReader(). Phương thức này trả về một đối tượng SqlDataReader và rồi chúng ta sẽ dùng để đọc dữ liệu hàng được

trả về bởi phát biểu SELECT.

Bước 5: Tạo ra một đối tượng SqlDataReader và gọi phương thức ExecuteReader() của đối tượng MySqlCommand để chạy phát biểu SELECT.

Đọc hàng sử dụng đối tượng SqlDataReader

Bước 6: Đọc hàng trong mySqlDataReader sử dụng phương thức Read():

mySqlDataReader.Read();

Trình bày những giá trị cột từ đối tượng SqlDataReader

Chúng ta có thể đọc giá trị cho một cột từ mySqlDataReader bằng cách chuyển qua tên của cột trong cặp dấu ngoặc vuông. Chẳng hạn, mySqlDataReader[" CustomerID "] trả về giá trị của cột CustomerID.

Bước 7: Trình bày những giá trị cột cho CustomerID, CompanyName, ContactName, và Address:

```
Console.WriteLine("mySqlDataReader[\" CustomerID\"] = "+
mySqlDataReader["CustomerID"]);
```

```
Console.WriteLine("mySqlDataReader[\" CompanyName\"] = "+
mySqlDataReader["CompanyName"]);
```

```
Console.WriteLine("mySqlDataReader[\" ContactName\"] = "+
mySqlDataReader["ContactName"]);
```

```
Console.WriteLine("mySqlDataReader[\"      Address\"]      =      "+
mySqlDataReader["Address"]);
```

Đóng đối tượng SqlDataReader

Khi chúng ta kết thúc việc đọc những hàng từ một đối tượng SqlDataReader, đóng nó sử dụng phương thức Close()

Bước 8: Gọi phương thức Close() để đóng mySqlDataReader:

```
mySqlDataReader.Close();
```

Đóng đối tượng SqlConnection

Khi chúng ta kết thúc truy cập cơ sở dữ liệu, đóng đối tượng SqlConnection của chúng ta sử dụng phương thức Close().

Bước 9: Gọi phương thức Close() để đóng mySqlConnection:

```
mySqlConnection.Close();
```

b. Lọc dữ liệu

Dữ liệu từ một table trong cơ sở dữ liệu được lát toàn bộ và đổ vào Dataset, sau đó tiến hành lọc những dòng dữ liệu cần thiết.

Ví dụ 7.3 sau mô tả cách thực hiện này.

```
private void button4_Click(object sender, EventArgs e)
```

```

{
string connectionstring="PROVIDER=SQLOLEDB;
server=(local):uid=netpwd=;database=NET";
OleDbConnction conObj=new OleDbConnection(connectionstring);
conObj.Open();
OleDbCommand cmd=new OleDbConnection("Select * from
books",conObj);
OleDbDataAdapter da=new OleDbDataAdapter(cmd);
DataSet ds=new DataSet();
da.Fill(ds,"books");
MessageBox.Show(ds.Tables["books"].Rows.Count.ToString());
DataView dv=new DataView(ds.Tables["books"]);
dv.RowFilter="fprice>20000";
dv.Sort="fprice Desc";
MessageBox.Show(dv.Table.Rows.Count.ToString());
conObj.Close();
}

```

Ví dụ 7.4 sẽ lấy về danh sách những quyển sách có nhà xuất bản là công ty phát hành sách nhà xuất bản trẻ. (mã số inxb=2);

```

private void button5_Click(object sender, EventArgs e)
{
string connectionstring="PROVIDER=SQLOLEDB;
server=(local):uid=netpwd=;database=NET";
OleDbConnction conObj=new OleDbConnection(connectionstring);
conObj.Open();
OleDbCommand cmd=new OleDbConnection("Select * from books where
inxb=?",conObj);
cmd.CommandType=CommandType.Text;
cmd.Parameters.Add("@ibookid",2);
OleDbDataAdapter da=new OleDbDataAdapter(cmd);
DataSet ds=new DataSet();
da.Fill(ds,"books");
MessageBox.Show(ds.Tables["books"].Rows.Count.ToString());
conObj.Close();
}

```

```
}
```

Chúng ta lưu ý với dòng lệnh

```
cmd.Parameters.Add("@ibookid", 2);
```

Đây là cách sử dụng ngắn gọn khi truyền tham số mà những chuyên gia phát triển phần mềm thực tế thường sử dụng, nguyên mẫu câu lệnh có công thức như sau:

```
OleDbParameter param = new  
OleDbParameter("@paramname",OleDbType.VarChar);  
param.Value = "value";
```

Trong đó, OleDbType là thư viện những kiểu dữ liệu tương ứng với những kiểu dữ liệu lưu trữ trong hệ cơ sở dữ liệu.

4. Thao tác với dữ liệu

a. Thêm mới dữ liệu vào cơ sở dữ liệu

```
private void btnthem_Click(object sender, EventArgs e)  
{  
string connectionstring="PROVIDER=SQLOLEDB;  
server=(local):uid=netpwd=;database=NET";  
OleDbConnction conObj=new OleDbConnection(connectionstring);  
conObj.Open();  
OleDbCommand cmd=new OleDbCommand("insert books  
values(?,?,?,?)",conObj);  
cmd.CommandType=CommandType.Text;  
cmd.Parameters.Add("@inxb",2);  
cmd.Parameters.Add("@vmasach","MS0002");  
cmd.Parameters.Add("@vtensach","Lập trình .NET cho cuộc sống");  
cmd.Parameters.Add("@fgia",75000);  
cmd.ExcuteNonQuery();  
MessageBox.Show("Đã thêm một sách vào CSDL");  
conObj.Close();  
}
```

Ở ví dụ trên, câu lệnh SQL insert được thực hiện kết hợp với đối tượng tham số Parameters để đẩy dữ liệu vào cơ sở dữ liệu thông qua đối tượng Command.

b. Cập nhật, xóa dữ liệu trong cơ sở dữ liệu

Quá trình cập nhật và xóa dữ liệu tương tự như với thêm mới cơ sở dữ liệu.

```

private void btncapnhat_Click(object sender, EventArgs e)
{
string connectionstring="PROVIDER=SQLOLEDB;
server=(local):uid=netpwd=;database=NET";
OleDbConnction conObj=new OleDbConnection(connectionstring);
conObj.Open();
OleDbCommand cmd=new OleDbCommand("update books set
vtensach=?,fgia=? where vmasach=?",conObj);
cmd.CommandType=CommandType.Text;
cmd.Parameters.Add("@vmasach","MS0002");
cmd.Parameters.Add("@vtensach",".NET for read life");
cmd.Parameters.Add("@fgia",85000);
cmd.ExcuteNonQuery();
MessageBox.Show("Đã sửa đổi nội dung sách trong CSDL");
conObj.Close();
}

```

Xóa dữ liệu

```

private void btncapnhat_Click(object sender, EventArgs e)
{
string connectionstring="PROVIDER=SQLOLEDB;
server=(local):uid=netpwd=;database=NET";
OleDbConnction conObj=new OleDbConnection(connectionstring);
conObj.Open();
OleDbCommand cmd=new OleDbCommand("delete from books where
vmasach=?",conObj);
cmd.CommandType=CommandType.Text;
cmd.Parameters.Add("@vmasach","MS0002");
cmd.ExcuteNonQuery();
MessageBox.Show("Đã xóa sách trong CSDL");
conObj.Close();
}

```

5. Lập báo cáo

Bước 1: Tạo DataSet (Nguồn dữ liệu):

Chuột phải vào Project > **Add > New Item > DataSet** (đặt tên ví dụ: dsReport.xsd).

Mở DataSet, chuột phải chọn Add > DataTable, thêm các cột dữ liệu tương ứng với báo cáo.

Bước 2: Thiết kế Báo cáo (.rdlc):

Chuột phải vào Project > Add > New Item > Report (đặt tên Report1.rdlc).

Mở report, dùng Report Data pane để thêm DataSet đã tạo ở bước 1.

Kéo thả Table, Textbox để thiết kế giao diện.

Bước 3: Hiển thị Báo cáo với ReportViewer:

Mở Form, tìm control ReportViewer trong Toolbox và kéo vào.

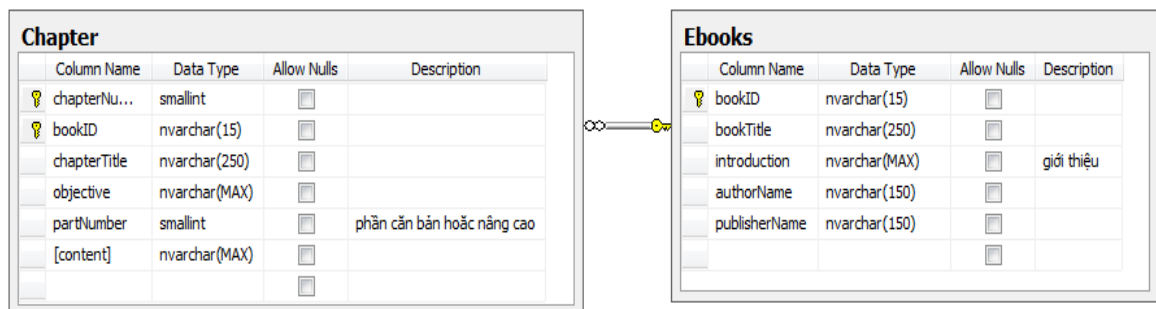
Bấm mũi tên đen trên góc phải control ReportViewer > chọn Choose Report (Report1.rdlc).

Trong code, nạp dữ liệu vào DataSet và refresh báo cáo:

```
var reportDataSource = new ReportDataSource("DataSet1", dataTable);
reportViewer1.LocalReport.DataSources.Clear();
reportViewer1.LocalReport.DataSources.Add(reportDataSource);
reportViewer1.LocalReport.ReportPath = "Report1.rdlc";
reportViewer1.RefreshReport();
```

Bài tập:

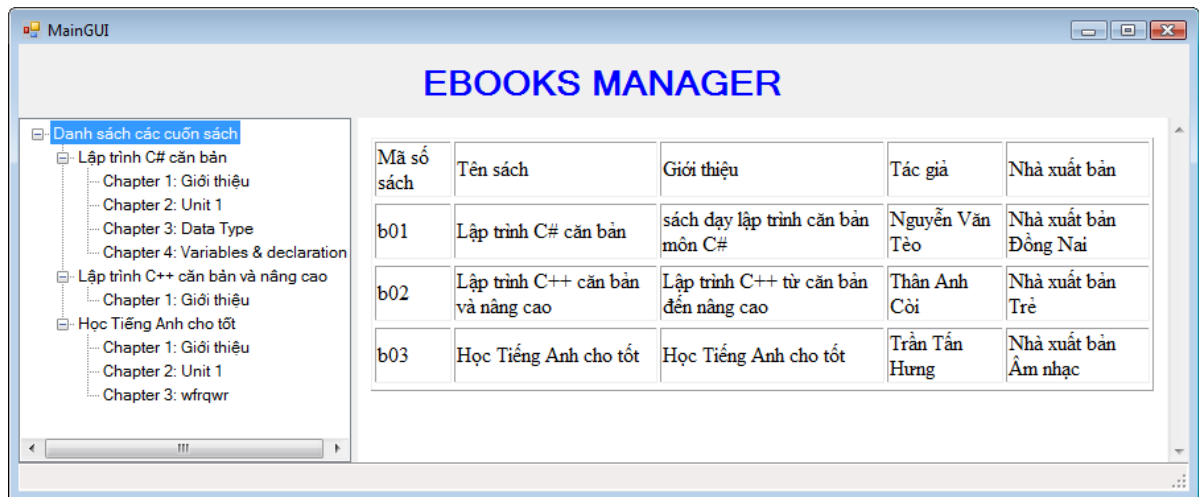
Để quản lý sách, người ta cho 1 phần lược đồ quan hệ được mô tả như hình sau:



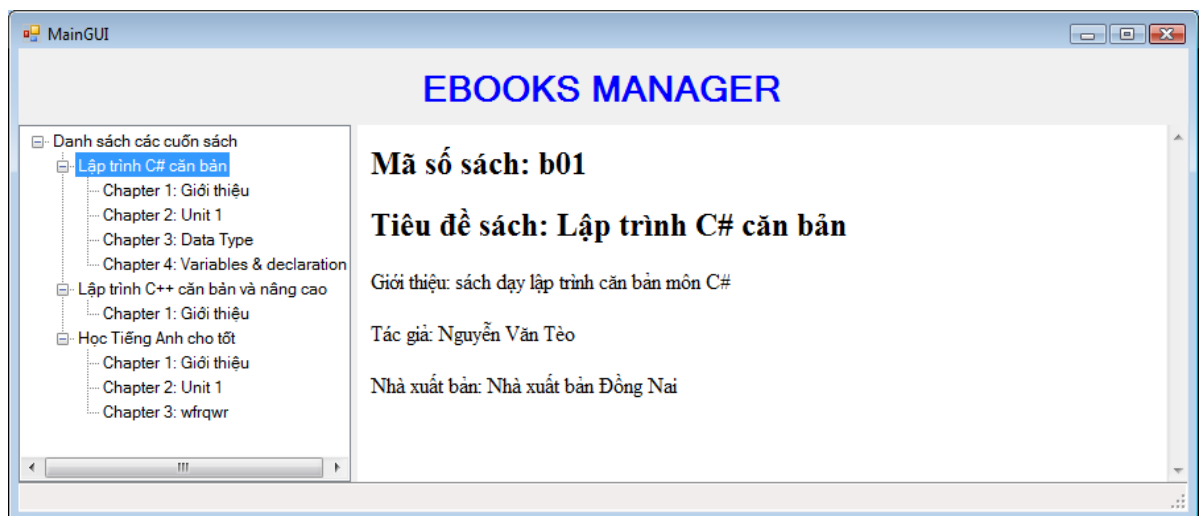
Viết chương trình thực hiện các yêu cầu sau:

Khi chương trình khởi động sẽ xây dựng TreeView hiển thị danh sách các cuốn sách cùng các chương thuộc về cuốn sách đó.

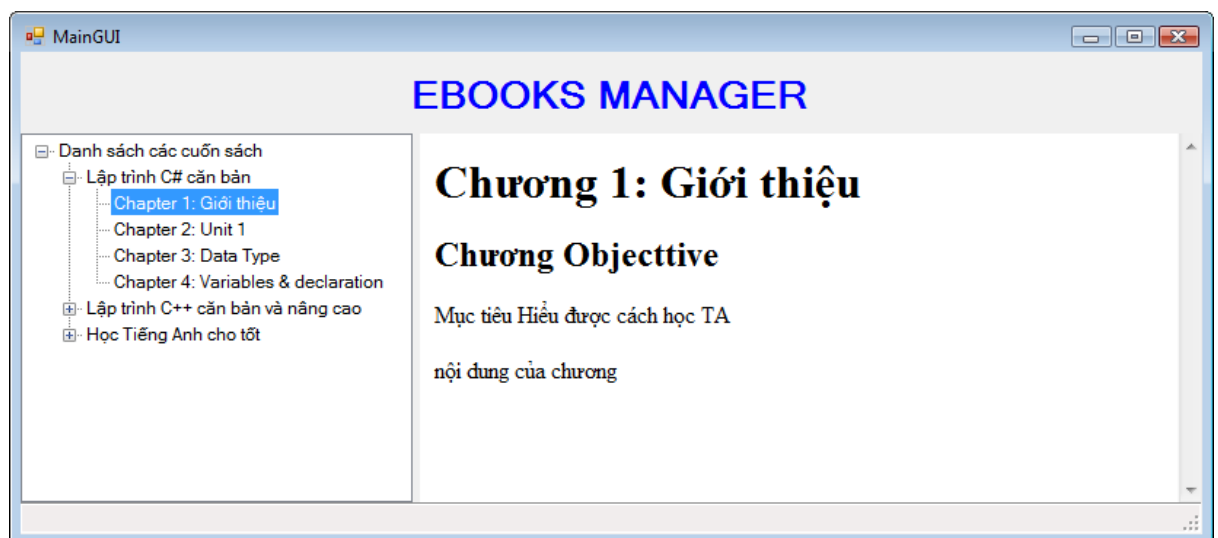
Khi chọn danh sách các cuốn sách, webbrowser sẽ hiển thị danh sách sách như hình vẽ



Khi chọn 1 cuốn sách, thông tin chi tiết của cuốn sách sẽ được hiển thị

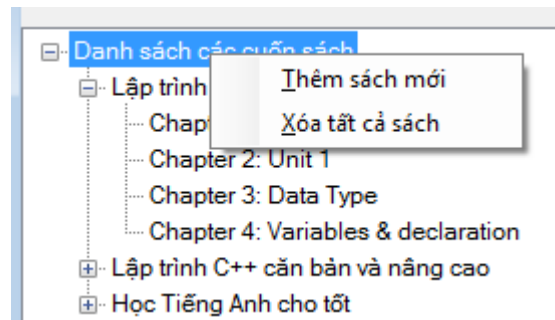


Khi chọn 1 chương, nội dung của chương sẽ được hiển thị như hình vẽ

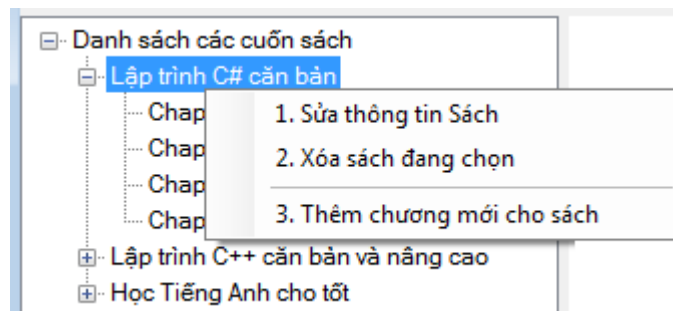


Khi nhấn chuột phải lên danh sách các cuốn sách, context menu sẽ hiển thị

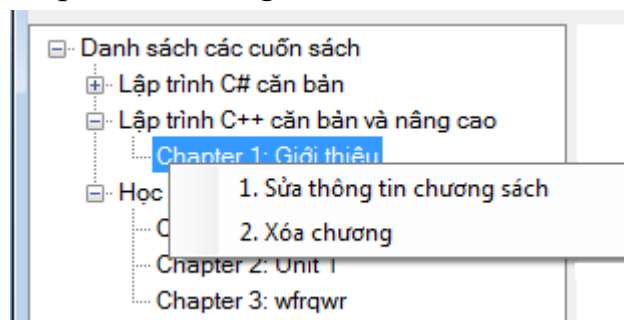
như hình sau



Khi nhấn chuột phải 1 cuốn sách được chọn, context menu sẽ hiển thị như hình sau



Khi nhấn chuột phải 1 chương sách được chọn, context menu sẽ hiển thị như hình sau



Form Thêm/ Sửa thông tin sách

A screenshot of a Windows Form titled 'Ebooks_WinForm'. The form has a title bar with standard Windows controls. The main content area has a title 'BOOK INFORMATION' in blue. Below the title are several input fields: 'Mã sách:' with a text box containing 'b01', 'Tựa đề:' with a text box containing 'Lập trình C# căn bản', 'Giới thiệu:' with a text area containing 'sách dạy lập trình căn bản môn C#', 'Tác giả:' with a text box containing 'Nguyễn Văn Tèo', and 'Nhà xuất bản:' with a text box containing 'Nhà xuất bản Đồng Nai'. At the bottom right are two buttons: 'Lưu' and 'Quay về'.

Form Thêm/sửa chương sách

CHAPTER INFORMATION

Số thứ tự chương: Mã số sách: b02

Tên chương:

Mục tiêu chương:

Phần:

Nội dung:

TÀI LIỆU THAM KHẢO

- 1/ C# 2005 – Lập trình cơ bản. Nhà xuất bản lao động và xã hội
- 2/ MSDN Library.
- 3/ Programming C#. Tác giả: Jesse Liberty. Nhà xuất bản: O'Reilly
- 4/ C# in 21 days. Tác giả: Bradley L.Jones. Nhà xuất bản: SAMS
- 5/ .Net toàn tập (tập 4: Lập trình CSDL dùng ADO.NET và C#) - Dương Quang Thiện.
- 6/ .Net toàn tập (tập 6: Lập báo cáo dùng RystalReport và C#) - Dương Quang Thiện.
- 7/ Lập trình windows nâng cao trường khoa học kỹ thuật máy tính HCM.
- 8/ Giáo trình lập trình Windows với c# .NET của nhà xuất bản Lao Động Xã Hội
- 9/ Windows Forms Programming with C#, Erik Brown, Manning Publications Microsoft Visual C# 2005 Step by Step, Microsoft Press
- 10/ Website www.codeproject.com